



US 20250285295A1

(19) **United States**

(12) **Patent Application Publication**

Kale et al.

(10) **Pub. No.: US 2025/0285295 A1**

(43) **Pub. Date: Sep. 11, 2025**

(54) **BIDIRECTIONAL OBJECT TRACKING IN
COMPUTER VISION APPLICATIONS**

(71) Applicant: **NVIDIA Corporation**, Santa Clara, CA
(US)

(72) Inventors: **Amit Kale**, Pune (IN); **Bhushan
Rupde**, Maharashtra (IN); **Kaustubh
Purandare**, San Jose, CA (US)

(21) Appl. No.: **18/597,451**

(22) Filed: **Mar. 6, 2024**

Publication Classification

(51) **Int. Cl.**
G06T 7/246 (2017.01)
G06T 7/277 (2017.01)
G06T 7/60 (2017.01)
G06T 7/73 (2017.01)
G06V 10/74 (2022.01)
G06V 10/75 (2022.01)

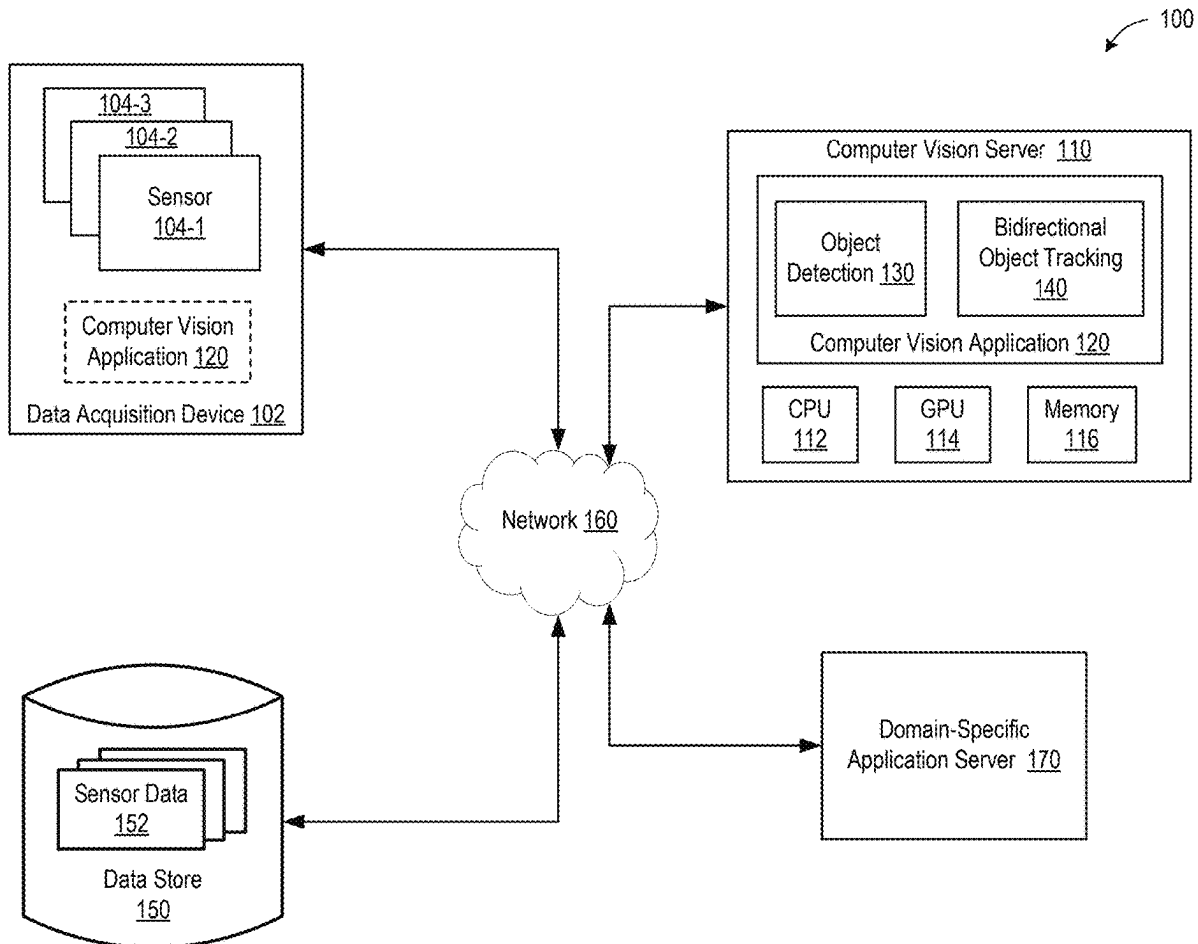
(52) **U.S. Cl.**

CPC **G06T 7/248** (2017.01); **G06T 7/277**
(2017.01); **G06T 7/60** (2013.01); **G06T 7/73**
(2017.01); **G06V 10/751** (2022.01); **G06V**
10/761 (2022.01); **G06T 2207/10016**
(2013.01); **G06T 2207/20081** (2013.01); **G06T**
2207/30241 (2013.01)

(57)

ABSTRACT

Disclosed are apparatuses, systems, and techniques for implementing bidirectional tracking in computer vision applications. In one embodiment, the techniques include obtaining digital representations of an object depicted in video frames and for each of a forward direction (FD) of tracking and a reverse direction (RD) of tracking, obtaining, using (i) a current state of the object associated with an upstream video frame and (ii) the digital representation of the object for a downstream video frame, an updated state of the object associated with the downstream video frame. The techniques further include obtaining, using the updated state of the object for the FD and/or the updated state of the object for the RD, a bidirectional state of the object, and determining, using the bidirectional state of the object, a trajectory of the object across the video frames.



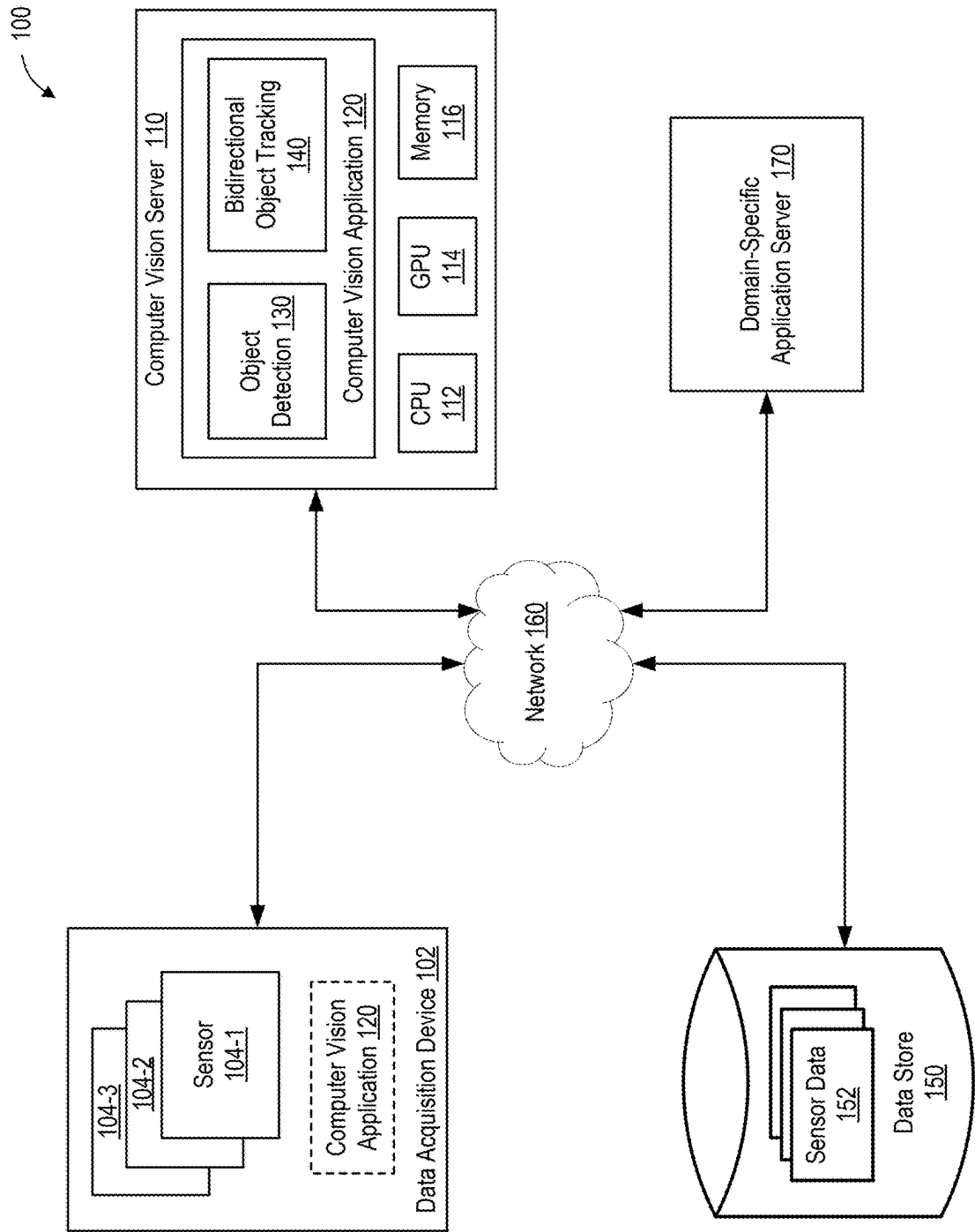


FIG. 1

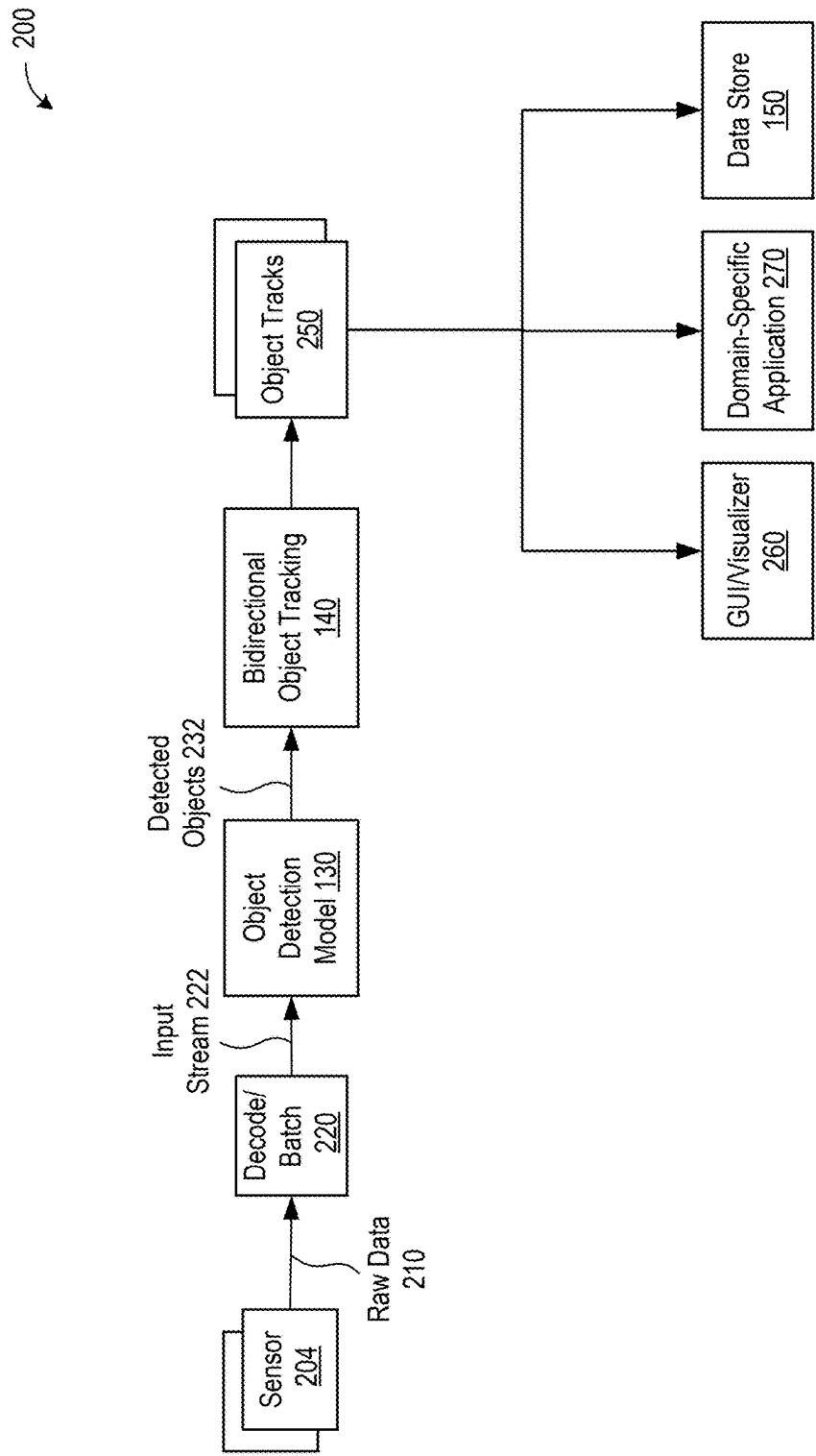


FIG. 2

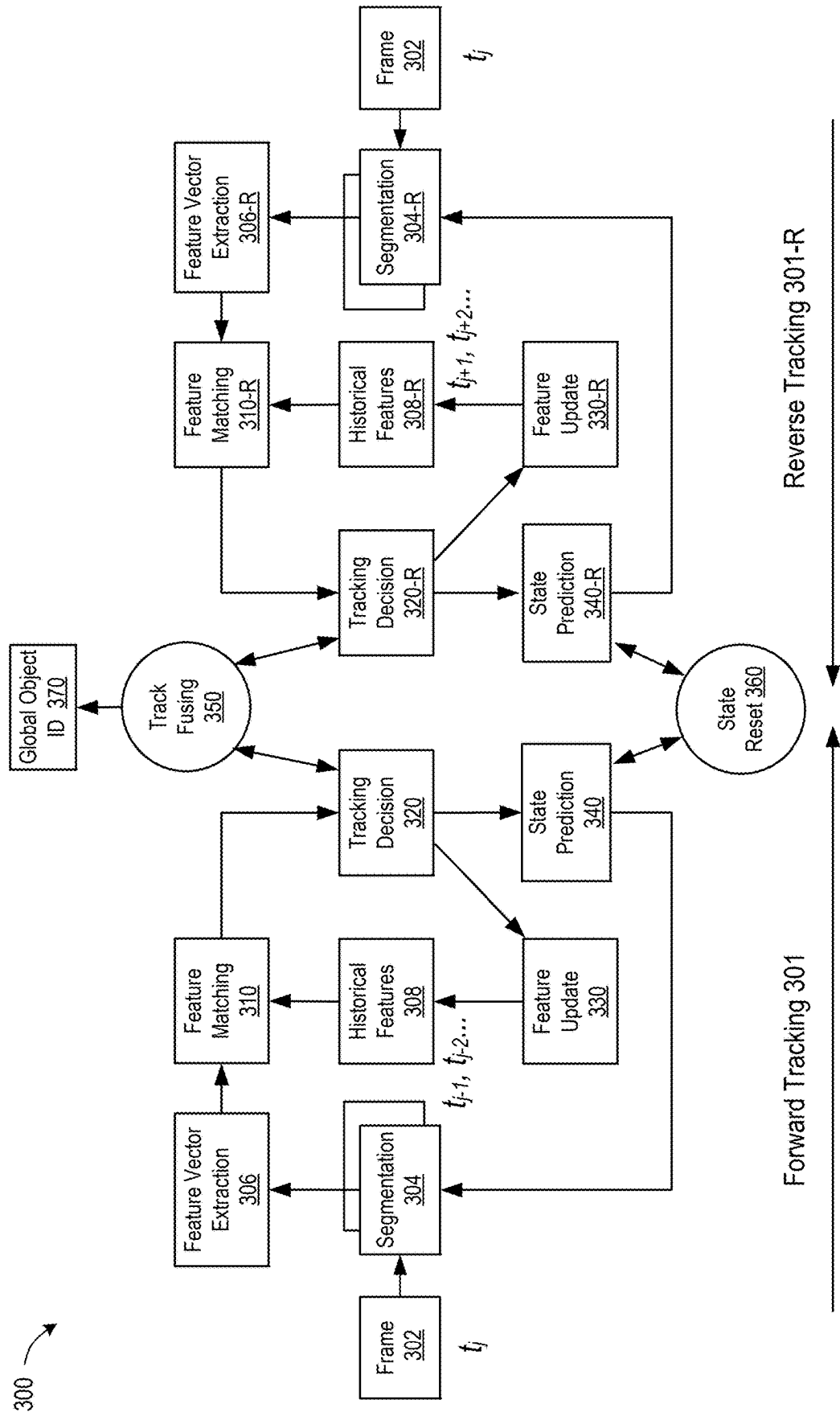


FIG. 3

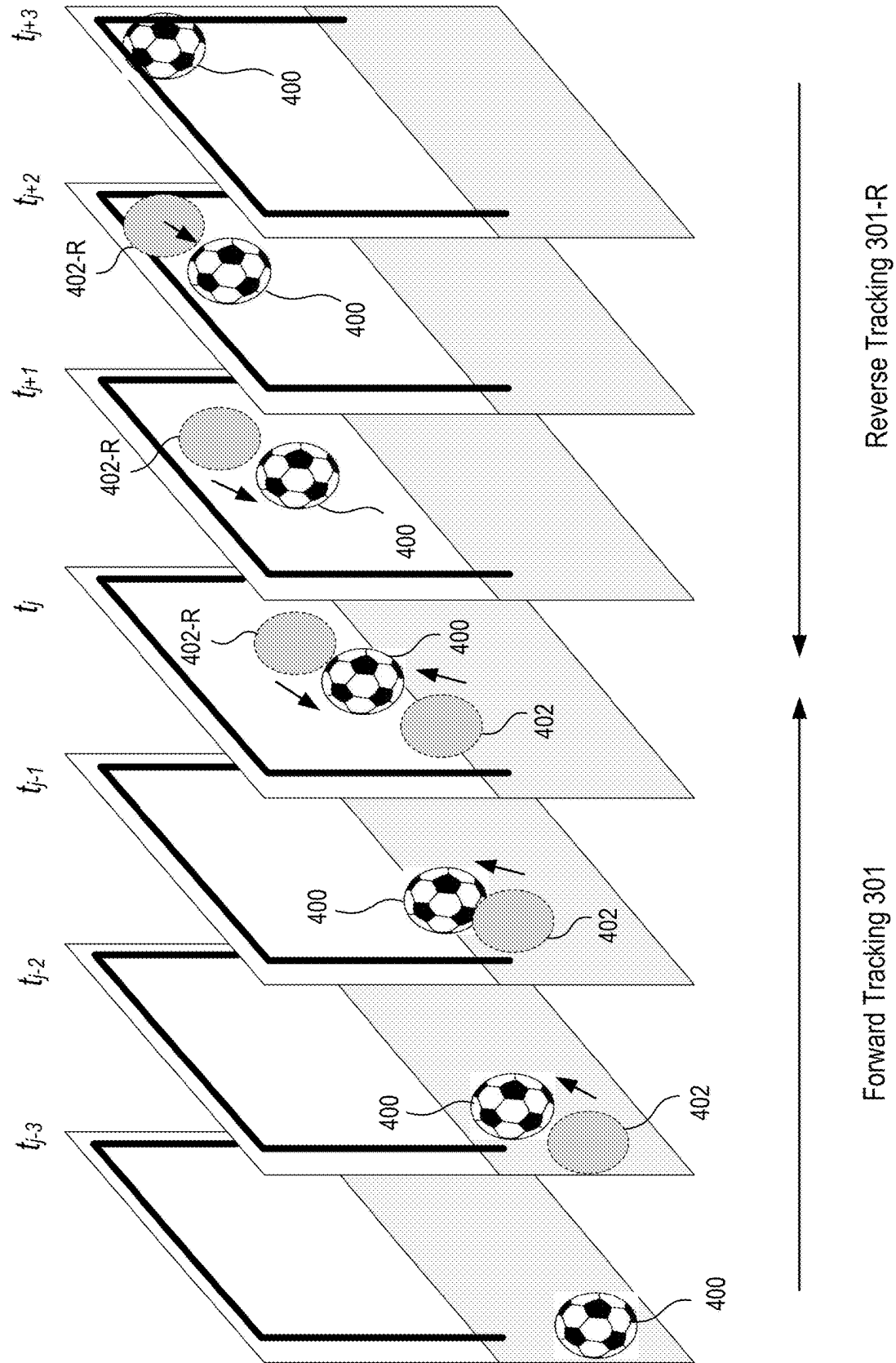
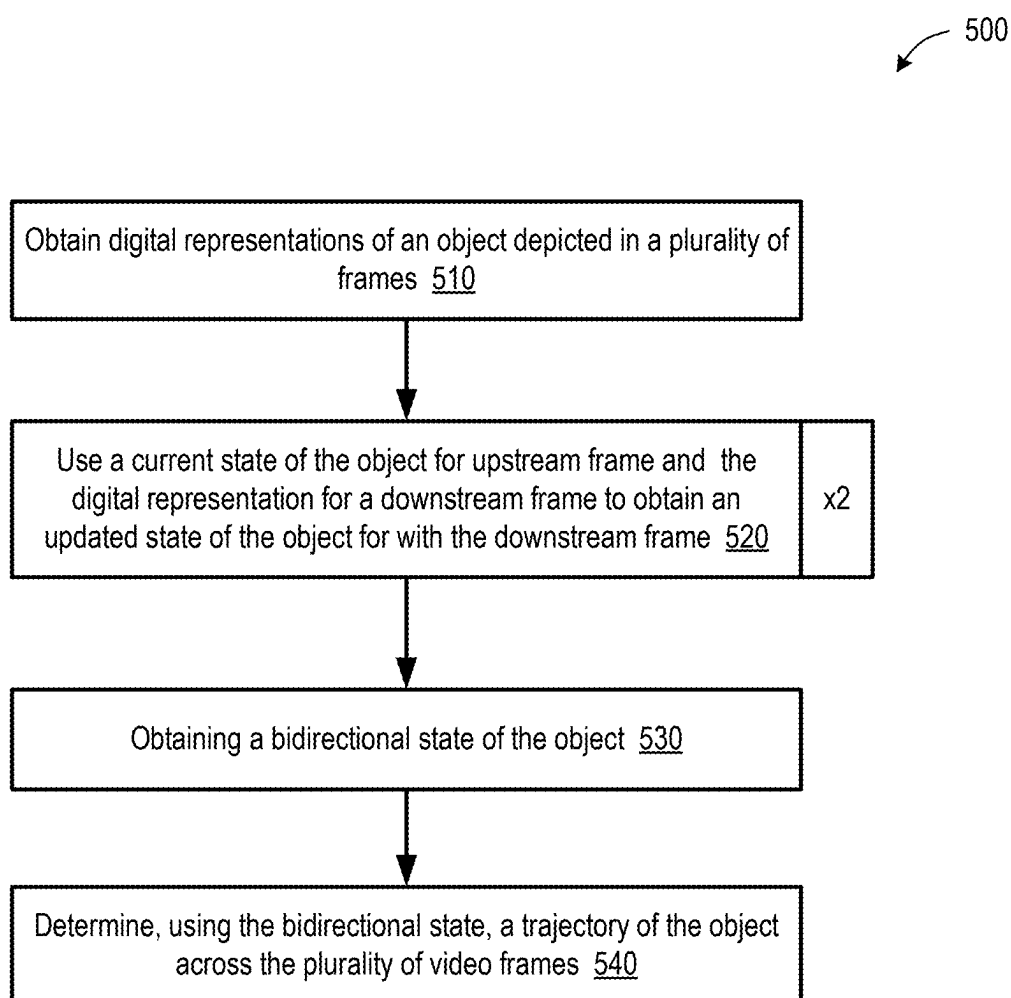


FIG. 4

**FIG. 5A**

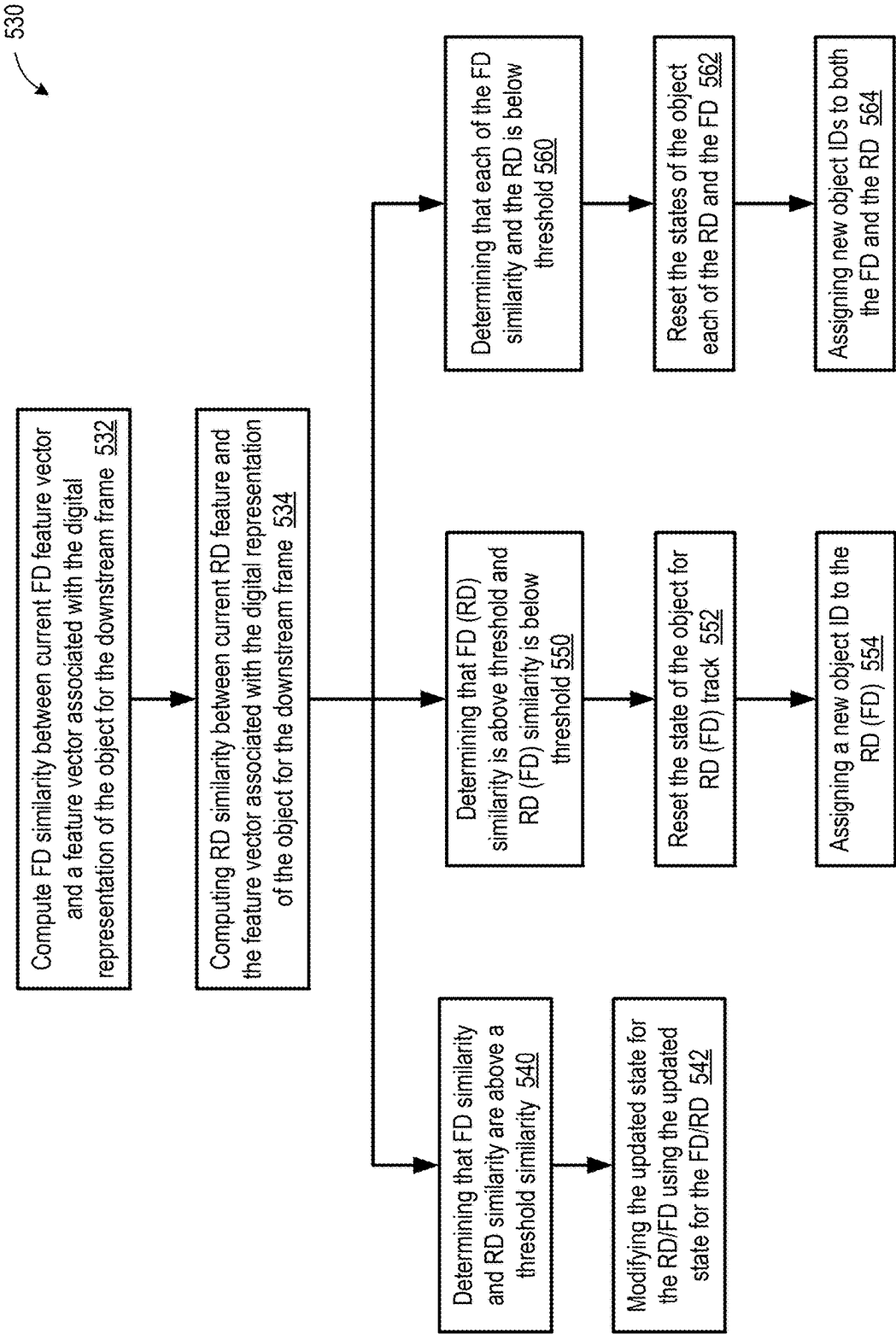


FIG. 5B

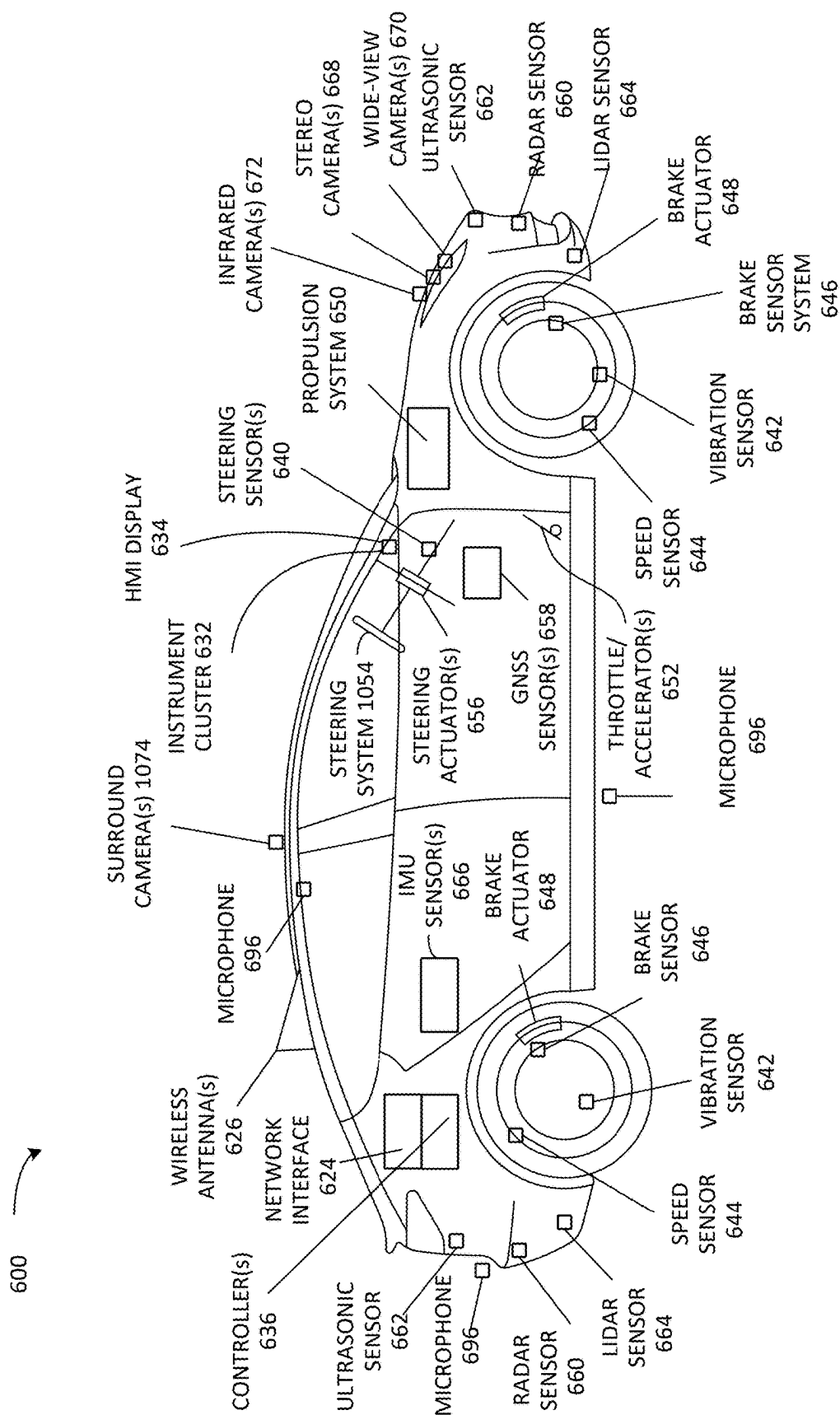


FIG. 6A

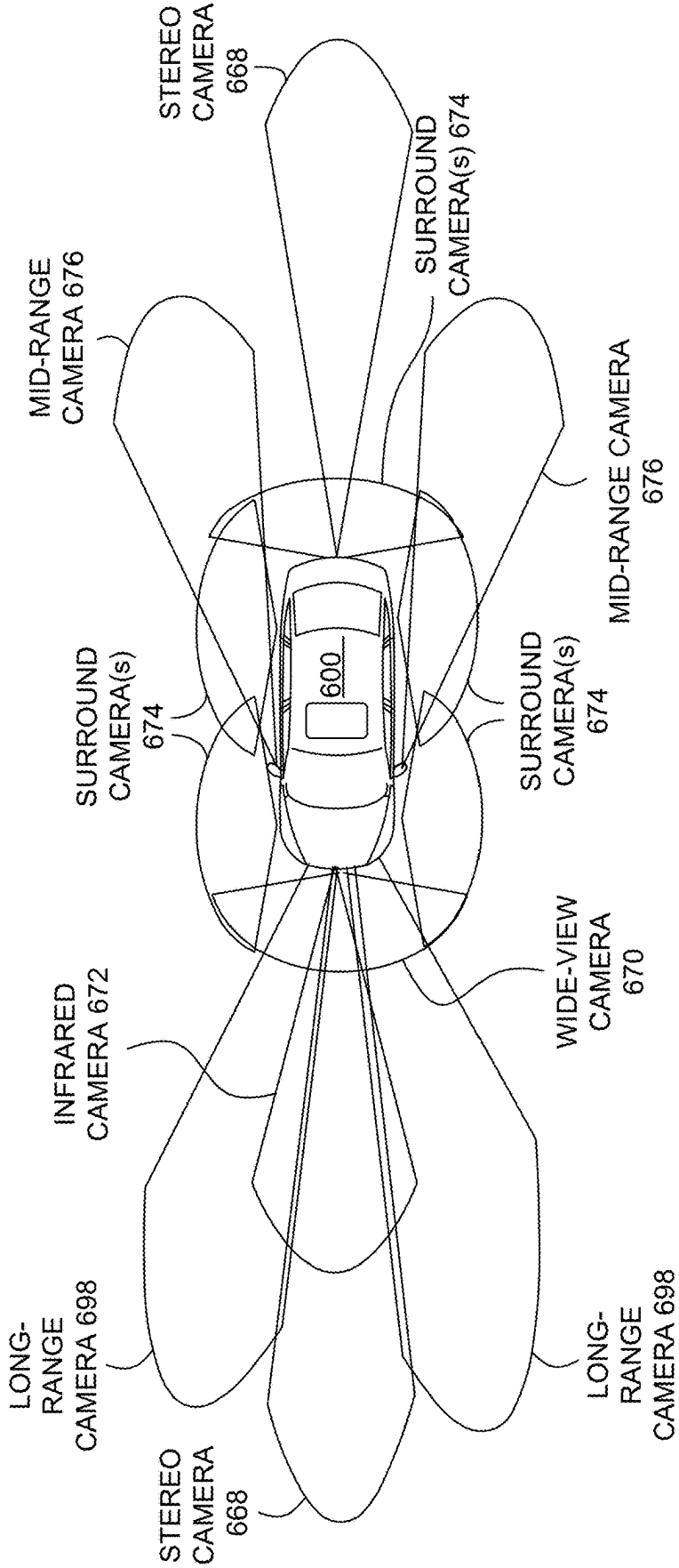


FIG. 6B

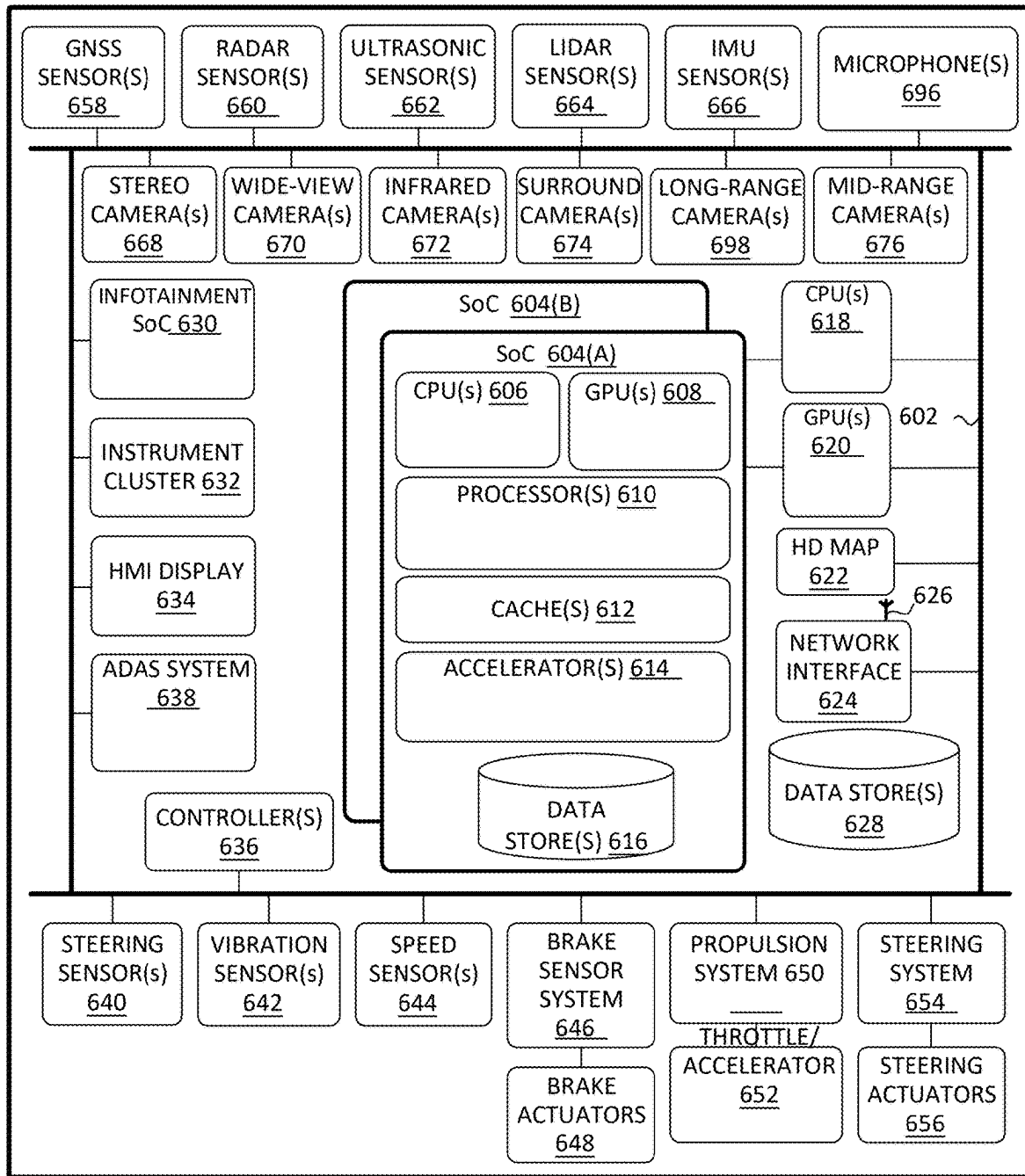


FIG. 6C

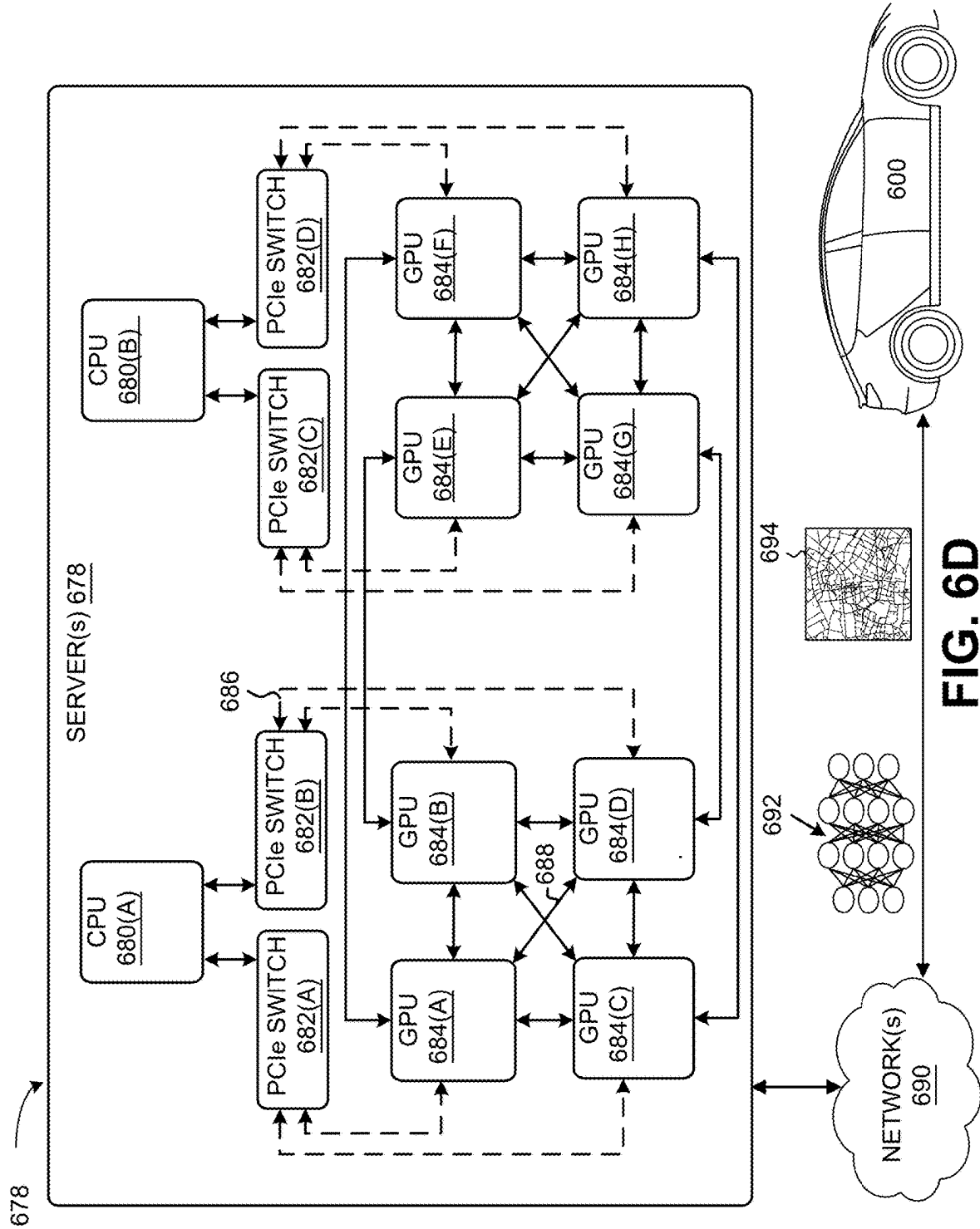


FIG. 6D

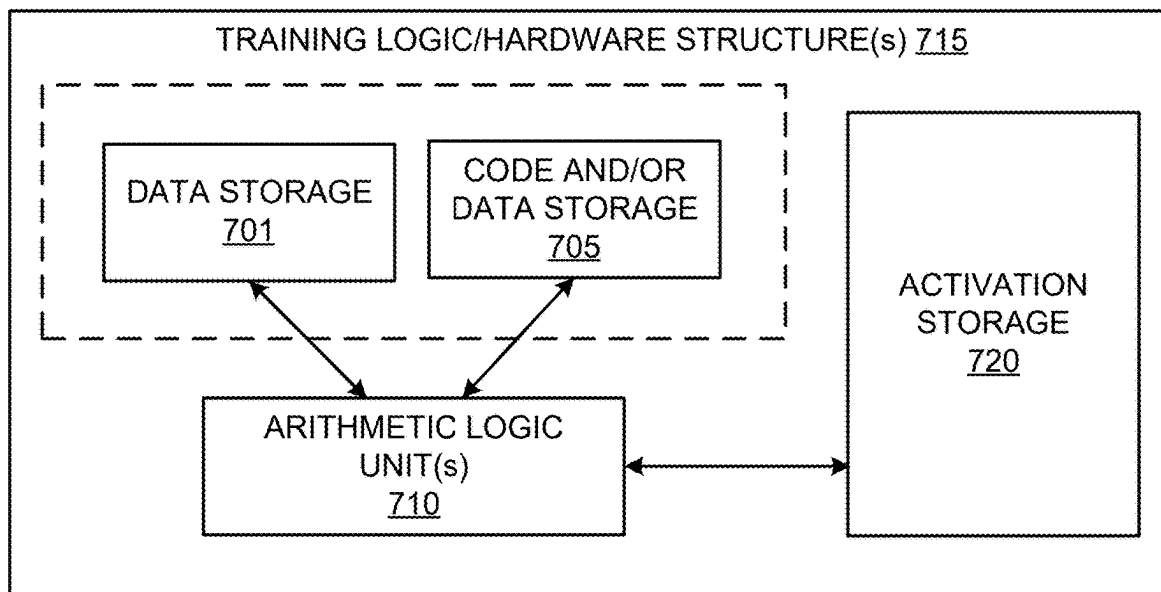


FIG. 7A

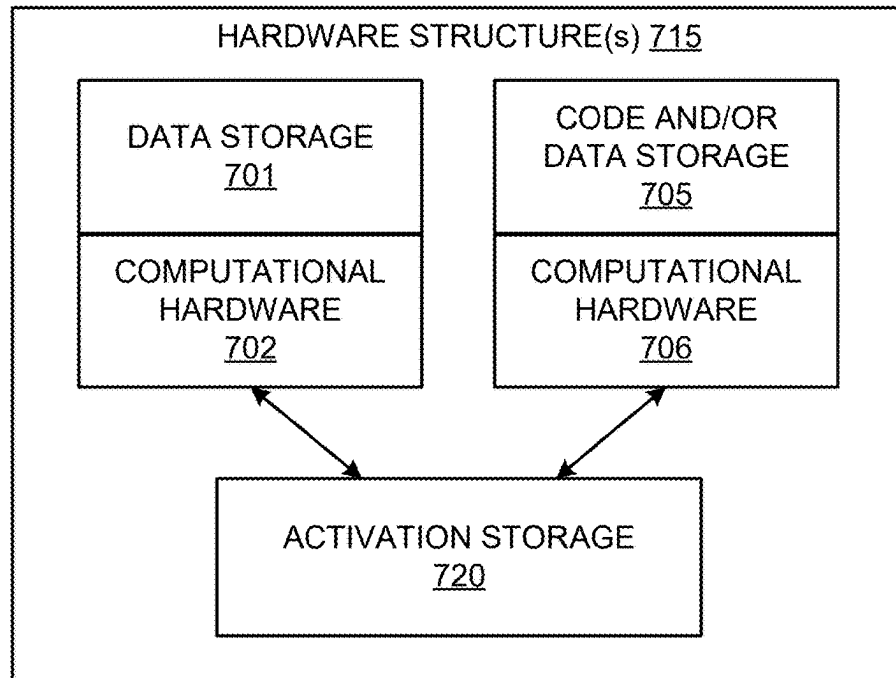


FIG. 7B

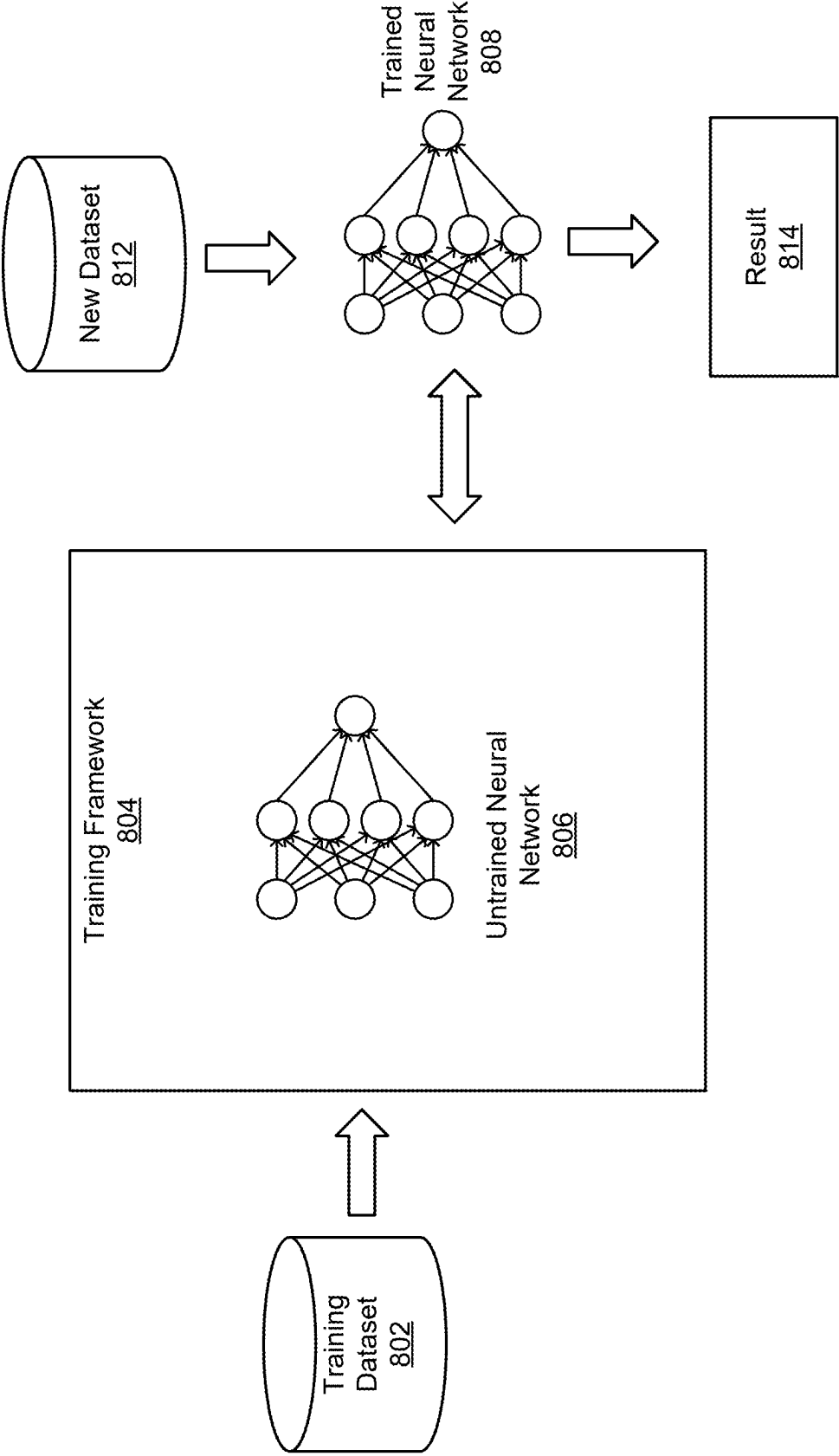


FIG. 8

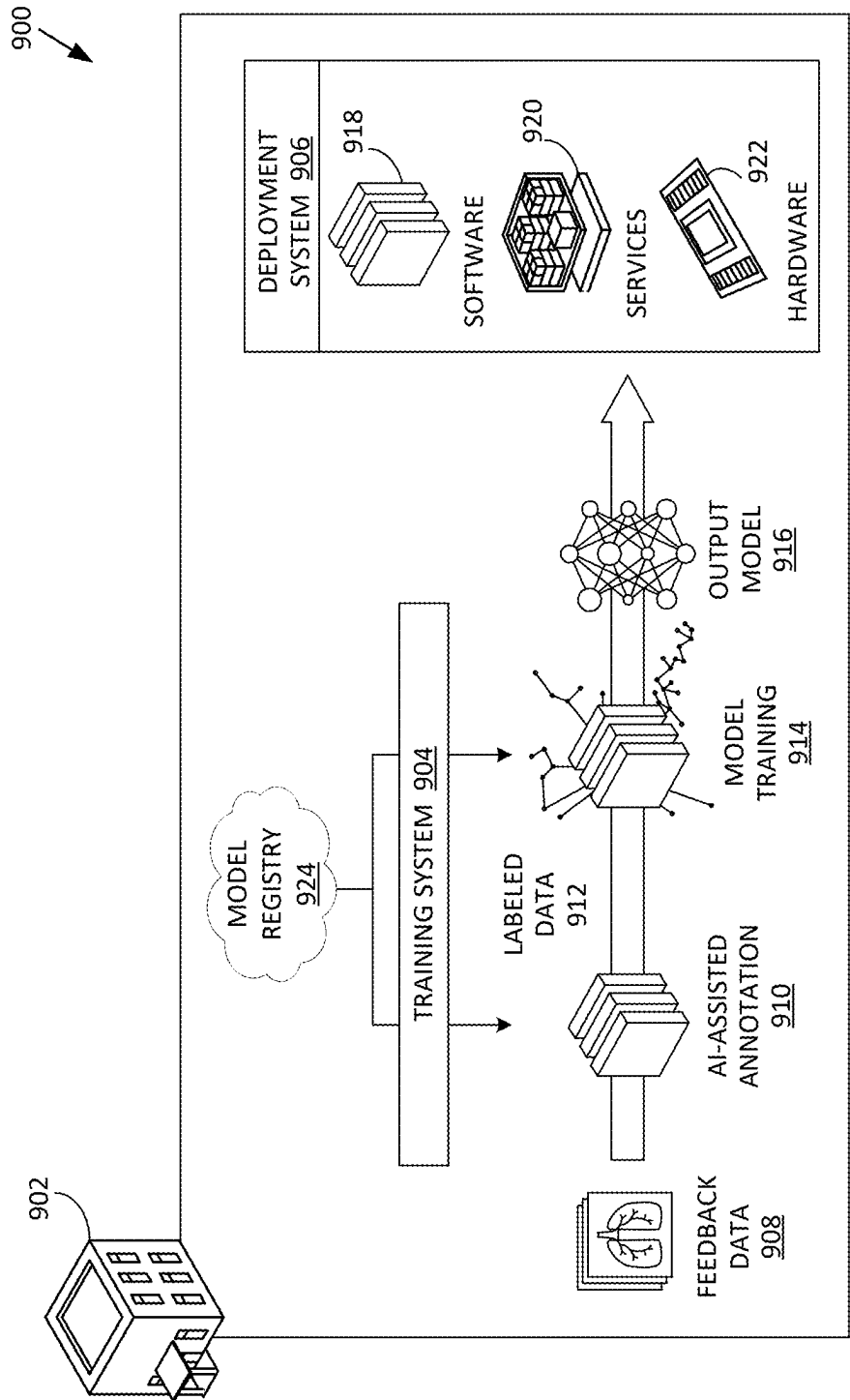


FIG. 9

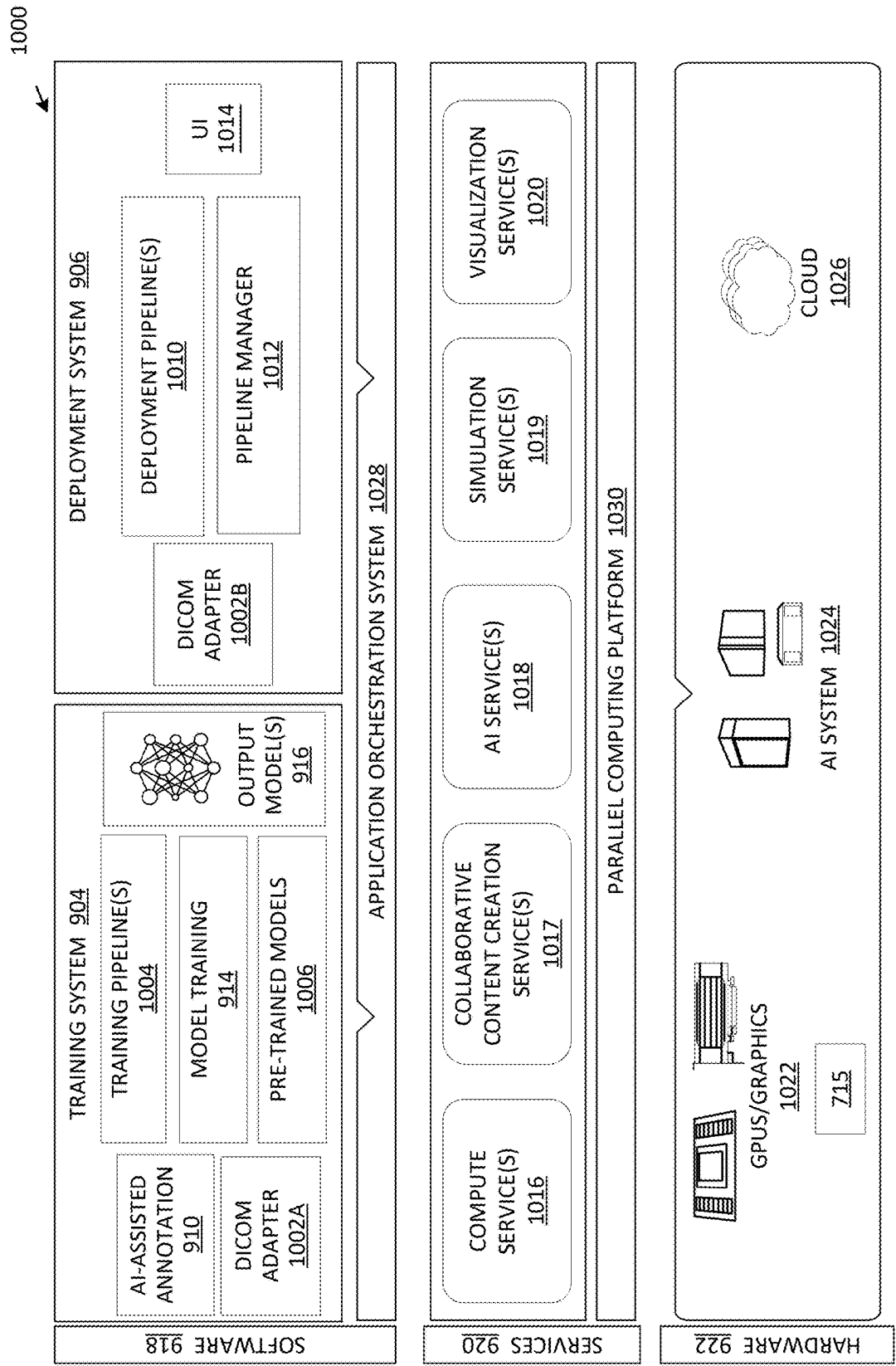


FIG. 10

BIDIRECTIONAL OBJECT TRACKING IN COMPUTER VISION APPLICATIONS

TECHNICAL FIELD

[0001] At least one embodiment pertains to processing resources used to execute and facilitate tasks performed using artificial intelligence. For example, at least one embodiment pertains to improving accuracy of computer vision applications that use machine learning models for detection, identification, and tracking of objects.

BACKGROUND

[0002] Machine learning techniques are often used in office and hospital environments, medical imaging, robotic automation, security applications, autonomous transportation, law enforcement, and many other settings. Computer vision is one field where machine learning systems and techniques are deployed to extract information from digital images or videos (streams of images). Computer vision automates many of the tasks that the human vision can do, such as detecting presence of objects in a field of view, identifying types of the objects, recognizing new and previously seen object, etc., and also performs tasks that the human cannot do (or can accomplish only approximately), e.g., accurately determining coordinates and velocities of objects. Computer vision can process video data collected by a variety of data-sensing devices, e.g., digital cameras, scanners, lidars, radars, and/or the like. Applications of computer vision include transportation (autonomous vehicle and driver-assist systems), public safety and surveillance systems, sports analytics, manufacturing product tracking, warehouse and retail tracking, personal identification, mood identification, and/or the like. Computer vision systems can be trained using training data (images, actions, facial expressions, and/or other data, such as sounds) to identify patterns in the data that may facilitate various computer vision functions, e.g., existence of an object within a training image/video (object detection), identification of a type of the object (object identification), following the object and predicting future locations of the object (object tracking), and/or the like. Training can be supervised or unsupervised. After a deployment of a successfully trained computer vision, new data (images/videos) is supplied to the model during an inference stage and various information is extracted from the new data using patterns and features learned by the model during training.

BRIEF DESCRIPTION OF DRAWINGS

[0003] FIG. 1 is a block diagram of an example computing architecture capable of implementing bidirectional tracking of objects in computer vision applications, according to at least one embodiment;

[0004] FIG. 2 illustrates schematically a data flow in an example computer vision system that deploys bidirectional tracking of objects, according to at least one embodiment;

[0005] FIG. 3 illustrates operations of a bidirectional object tracking deployed by a computer vision system, according to at least one embodiment;

[0006] FIG. 4 illustrates schematically forward tracking and reverse tracking of an example object, according to at least one embodiment;

[0007] FIGS. 5A-5B depict a flow diagram of an example method of bidirectional tracking of objects in computer vision applications, according to at least one embodiment;

[0008] FIG. 6A illustrates an example of an autonomous vehicle, according to at least one embodiment;

[0009] FIG. 6B illustrates an example of camera locations and fields of view for the autonomous vehicle of FIG. 6A, according to at least one embodiment;

[0010] FIG. 6C is a block diagram illustrating an example system architecture for the autonomous vehicle of FIG. 6A, according to at least one embodiment;

[0011] FIG. 6D is a diagram illustrating a system for communication between cloud-based server(s) and the autonomous vehicle of FIG. 6A, according to at least one embodiment;

[0012] FIG. 7A illustrates inference and/or training logic, according to at least one embodiment;

[0013] FIG. 7B illustrates inference and/or training logic, according to at least one embodiment;

[0014] FIG. 8 illustrates training and deployment of a neural network, according to at least one embodiment;

[0015] FIG. 9 is an example data flow diagram for an advanced computing pipeline, according to at least one embodiment;

[0016] FIG. 10 is a system diagram for an example system for training, adapting, instantiating and deploying machine learning models in an advanced computing pipeline, according to at least one embodiment.

DETAILED DESCRIPTION

[0017] Tracking of objects is an important function of computer vision, which may be used to track a vehicle on a street (intersection, parking area, etc.), a person in a secure area (airport, industrial facility, etc.), players and a ball in a ballgame (to collect statistics for sports analytics), and the like. Computer vision systems attempt to separate objects of interest from background in video frames, determine boundaries of the objects (e.g., via bounding boxes, contours, and/or the like), a type/class of the objects (e.g., vehicle, person, ball, etc.), determine direction and speed of the objects' motion, and predict the objects' future trajectory. As objects move within the field of view of a video-capturing sensor (e.g., one or more cameras), a visible size, orientation, appearance, colors, and/or other visible characteristics of the object change between different frames. Good-quality computer vision models/algorithms need to be capable of identifying those changes and consistently tracking correct objects despite their changing appearances. High resolution and high frame rate video data makes object identification and tracking more reliable by ensuring that object features are informative, distinctive, and experience small changes between consecutive frames, which minimizes the likelihood of misrecognizing an object's depiction in a new frame as a depiction of a new object or a depiction of some other objects that are being tracked. However, computer vision applications often have to detect, classify, and track multiple objects using video data acquired and processed under less-than-ideal conditions. For example, a video may include a large number of small objects that appear similar. Objects may be moving towards or away from the camera resulting in quick and significant change of the visible dimensions of the objects. A video may have one or more frames missing (e.g., as a result of a decoding error or poor network conditions for streaming scenarios). In some instances, a

combination of these factors (and/or other factors) can occur simultaneously. In such instances, object misidentifications can occur, e.g., a previously identified and tracked object can be misrecognized as a new object and assigned a new tracking identification (ID), a situation referred to as an ID switch herein.

[0018] Aspects and embodiments of the instant disclosure address these and other technological challenges by disclosing methods and systems that, on one hand, reduce the likelihood of occurrence of ID switches and, on the other hand, provide a mechanism for identifying and correcting falsely attempted ID switches. More specifically, the disclosed techniques include tracking objects in both the forward direction of time and the reverse direction of time. For example, a time series of video frames $t_1, t_2, \dots, t_N$ can depict any number of objects (which may be identified using a suitable trained object detection model) and the state of the object $S(t_j)$ can be tracked in the forward direction, $S(t_1) \rightarrow \dots \rightarrow S(t_{j-1}) \rightarrow S(t_j) \rightarrow S(t_{j+1}) \rightarrow \dots \rightarrow S(t_N)$. The available frames $t_1, t_2, \dots, t_N$ need not be equidistant in time, as some of the frames may be missing. The state of the object may include an object ID, the object's state of motion $X(t_j)$ (which may include object's coordinates, velocity, acceleration, etc.), and a feature vector $F(t_j)$ representing a combined historical visual appearance of the object through frames $t_1, \dots, t_j$: $S(t_j) = \{ID, X(t_j), F(t_j)\}$. The object's state of motion (which may include the object's size) $X(t_j)$ is used to predict (project) a new state of motion and size (e.g., using a suitable statistical filter, e.g., a Kalman filter, a particle filter, and/or the like) of the object $X_{pred}(t_{j+1})$ for new frame, e.g., t_{j+1} . The object's observed (based on the location in the frame) state of motion $X_{obs}(t_{j+1})$ is then used to update the object's tracked state of motion, $X_{pred}(t_{j+1}), X_{obs}(t_{j+1}) \rightarrow X(t_{j+1})$. Similarly, a new feature vector $F_{obs}(t_{j+1})$ representing observed appearance of the object in frame t_{j+1} is obtained and used—in comparison with historical appearance feature vector $F(t_j)$ —to determine whether the object in frame t_{j+1} is the same as (or different than) the object in frame t_j . If the two vectors are sufficiently similar (e.g., based on their cosine similarity) and are given a high confidence value by a tracking component, the object ID from frame t_j is maintained for frame t_{j+1} . The historical feature vector may then be updated, $F(t_j) \rightarrow F(t_{j+1}) = M(F(t_j), F_{obs}(t_{j+1}))$, using a suitable mapping function $M(\cdot)$. If the two vectors are dissimilar, a new object ID_{new} may be started for the observed object indicating that this new object (or a previously occluded object) has entered the field of view. Such an ID switch may also include initializing a new object's state and starting a new track for new object ID_{new} based on the observed feature vector $F_{obs}(t_{j+1})$ and observed state of motion $X_{obs}(t_{j+1})$. The object ID and the most recent state of the previously tracked object $S(t_j)$ may be stored, e.g., to account for a possibility that the object may be temporarily occluded. In such instances, the reappearing object may be given the same object ID and a state that may continue from the same state $S(t_j)$. If a sufficiently long time has passed, a new state for object ID may be initialized. In some instances, the object ID may be switched to one of other existing object IDs, provided that the newly observed feature vector $F_{obs}(t_{j+1})$ is sufficiently similar to the historical appearance feature vector $F(t_j)$ for that other object. In such instances, the observed feature vector $F_{obs}(t_{j+1})$ and the observed

position in frame t_{j+1} can be used to update the state of the other object, followed by tracking the object for subsequent frames, t_{j+2}, t_{j+3} , et.

[0019] The addition of the reverse tracking eliminates significantly false ID switches. The reverse track $S(t_N) \rightarrow \dots \rightarrow S(t_{j+1}) \rightarrow S(t_j) \rightarrow S(t_{j-1}) \rightarrow \dots \rightarrow S(t_1)$ tracks the state of the object in the reverse (anti-chronological) order of frames, in a way that is similar to the forward tracking, e.g., by defining the reverse state: $\bar{S}(t_j) = \{ID, \bar{X}(t_j), \bar{F}(t_j)\}$. The object's state of motion $\bar{X}(t_j)$ for the reverse track predicts the state of motion $\bar{X}_{pred}(t_{j-1})$ for earlier frame t_{j-1} and, taken together with the observed state of motion $\bar{X}_{obs}(t_{j-1})$, is used to update the object's tracked state of motion, $\bar{X}(t_{j-1})$. A new feature vector $\bar{F}_{obs}(t_{j-1})$ representing appearance of the object in frame t_{j-1} is then used to determine if the object in frame t_{j-1} is the same as the object in frame t_j . If the two vectors $\bar{F}(t_j)$ and $\bar{F}_{obs}(t_{j-1})$ are sufficiently similar and have high tracker confidence values, the reverse track object ID from frame t_j is maintained for frame t_{j-1} and the historical feature vector is updated, $\bar{F}(t_j) \rightarrow \bar{F}(t_{j-1}) = M(\bar{F}(t_j), \bar{F}_{obs}(t_{j-1}))$. If the two vectors are dissimilar, the reverse track object ID may be stored together with the most recent state of the object $\bar{S}(t_{j-1})$, to account for a possibility that the object may be temporarily occluded. In such instances, the reappearing object may be given the same object ID and a state that may continue from the same state $\bar{S}(t_j)$. If a sufficiently long time has passed, a new state for object ID may be initialized. In the meantime, a new object ID_{new} may be assigned to an object that is associated with the observed vector $\bar{F}_{obs}(t_{j-1})$.

[0020] The existence of the parallel forward tracking and reverse tracking prevents at least some of the ID switches. For example, if an ID switch occurs between frames t_{j-1} and t_j in the forward direction, but no matching ID switch occurs between frames t_j and t_{j-1} in the reverse direction, the forward ID switch may be deemed false, the object ID from frame t_{j-1} may be maintained for frame t_j , and the backward feature vector $\bar{F}(t_j)$ and the observed state of motion $\bar{X}_{obs}(t_j)$ may be used to update the state of the object $\bar{S}(t_{j-1})$ to obtain the new state $\bar{S}(t_j)$. Similarly, if an ID switch occurs between frames t_j and t_{j-1} in the reverse direction, but no matching ID switch occurs between frames t_{j-1} and t_j in the forward direction, the reverse ID switch may be deemed false, the object ID from frame t_j may be maintained for frame t_{j-1} , and the forward feature vector $F(t_{j-1})$ and the observed state of motion $X_{obs}(t_{j-1})$ may be used to update the state of the object $S(t_j)$ to obtain the new state $S(t_{j-1})$. The reverse object ID may also be matched to the forward object ID. If an ID switch occurs between frames t_{j-1} and t_j in the forward direction and a matching ID switch occurs between frames t_j and t_{j-1} in the reverse direction, the ID switch may be deemed true (meaning that the object tracked prior to frame t_j becomes occluded while a different object becomes visible starting from frame t_j). New IDs are then assigned to the objects for both directions of tracking and the states of the objects are reset. Finally, if no ID switch occurs between frames t_{j-1} and t_j in the forward direction or between frames t_j and t_{j-1} in the reverse direction, the reverse object ID may be matched to the forward object ID, and feature vectors may be updated for both directions, e.g., an average (or a weighted average) of $F(t_j)$ and $\bar{F}(t_j)$ (and, similarly, of $F(t_{j-1})$ and $\bar{F}(t_{j-1})$) may be taken to update feature vectors in both directions (data from the two directions of tracking is fused). Such updates may efficiently prevent future false ID switches.

[0021] The advantages of the disclosed techniques include, but are not limited to, preventing false ID switches from occurring as well as correcting false switches that are about to occur. Prevention of false ID switches comes as a result of better-quality feature vectors fused from the feature vectors for the forward direction and the reverse direction. Such fused feature vectors are capable of more accurate representation of visual appearances of tracked objects and, therefore, are less susceptible to misrecognizing the same object as a new object. Correcting false switches occurring in one direction (e.g., forward) is facilitated by the tracking of the objects in the second direction (e.g., backward), thus providing a check on whether the ID switch is real (true) or false.

[0022] The systems and methods described herein may be used for a variety of purposes, by way of example and without limitation, for machine control, machine locomotion, machine driving, synthetic data generation, model training, perception, augmented reality, virtual reality, mixed reality, robotics, security and surveillance, simulation and digital twinning, autonomous or semi-autonomous machine applications, deep learning, environment simulation, data center processing, conversational AI, generative AI, light transport simulation (e.g., ray-tracing, path tracing, etc.), collaborative content creation for 3D assets, cloud computing and/or any other suitable applications.

[0023] Disclosed embodiments may be comprised in a variety of different systems such as automotive systems (e.g., a control system for an autonomous or semi-autonomous machine, a perception system for an autonomous or semi-autonomous machine), systems implemented using a robot, aerial systems, medial systems, boating systems, smart area monitoring systems, systems for performing deep learning operations, systems for performing simulation operations, systems for performing digital twin operations, systems implemented using an edge device, systems for generating or presenting at least one of augmented reality content, virtual reality content, mixed reality content, systems incorporating one or more virtual machines (VMs), systems for performing synthetic data generation operations, systems implemented at least partially in a data center, systems for performing conversational AI operations, systems for performing light transport simulation, systems for performing collaborative content creation for 3D assets, systems implementing one or more language models, such as large language models (LLMs) (which may process text, voice, image, and/or other data types to generate outputs in one or more formats), systems implemented at least partially using cloud computing resources, and/or other types of systems.

System Architecture

[0024] FIG. 1 is a block diagram of an example computing architecture 100 capable of implementing bidirectional tracking of objects in computer vision applications, according to at least one embodiment. As depicted in FIG. 1, computing architecture 100 may include a data acquisition device 102, a computer vision server 110, a data store 150, a domain-specific application server 170, and/or other devices. Data acquisition device 102 may include one or more sensors 104-1, 104-2, 104-3, etc. For example, sensors 104-*k* may be photographic/video cameras producing a stream of image/video data, and/or any other suitable imaging sensors, including but not limited to light detection and

ranging (lidar) sensors, radio detection and ranging (radar) sensors, sonars, and/or the like. In some embodiments, sensors 104-*k* may additionally include audio data detection devices, e.g., microphones, producing audio data. In some embodiments, data generated by sensors 104-*k* may be streaming (runtime) data. In some embodiments, data generated by sensors 104-*k* may first be collected and stored and may undergo computer vision processing later, e.g., offline.

[0025] Sensors 104-*k* may generate sensor data in any suitable processed, unprocessed, or minimally processed raw data format, which may be sensor-specific and/or proprietary. In some embodiments, sensor data generated by sensors 104-*k* may be collected periodically with some frequency, which may correspond to a camera acquisition rate, lidar scanning frequency, and/or the like. In some embodiments, sensor data may be mosaicked pixel data obtained using a number of color filters, e.g., Bayer filter. Data generated by sensors 104-*k* may be serialized, deserialized, processed by any suitable image processor, compressed, and/or the like. Data generated by sensors 104-*k* may be streamed via any number of channels, including color channels, e.g., a red channel, a green channel, a blue channel, and/or the like. Data generated by sensors 104-*k* may be apportioned into frames (for video data) or any other units (for other types of data). The frames may be accompanied by various metadata, e.g., metadata indicating a type of sensor 104-*k* that generated the data, a time of data collection/generation, settings used by sensor 104-*k* to generate the data, a type of post-acquisition processing of the data, a format of the data, and/or the like.

[0026] In some embodiments, data streamed by sensors 104-*k* may be delivered to computer vision server 110 over a network 160. Network 160 may be a public network (e.g., the Internet), a private network (e.g., a local area network (LAN), or wide area network (WAN)), a wireless network, a personal area network (PAN), a mesh network, and/or any combination thereof. In some embodiments, data acquisition device 102 may be connected to computer vision server 110 over a high-speed bus (in lieu of network 160). For example, in traffic surveillance applications, sensors 104-*k* may be part of a sensing system that include lidars, radars, cameras, sonars, etc., whereas computer vision server 110 may process raw data generated by the sensing system and perform object detection, classification, and tracking. In such instances, data may be delivered between sensors 104-*k* and computer vision server 110 over a Gigabit Multimedia Serial Link (GMSL) or a similar high-speed bus.

[0027] Computer vision server 110 may support a computer vision application 120 that receives (e.g., over network 160 or a bus) data obtained by one or more sensors 104-*k*. Computer vision application 120 may include multiple stages of processing of the data, including one or more pre-processing operations: denoising, enhancement, changing resolution and contrast, binarization, cropping, aggregation, re-formatting, de-archiving, de-compression, batching, and/or the like. After pre-processing, the data may be processed by an object detection model 130. Object detection model 130 may be or include one or more deep neural networks having one or more hidden layers, e.g., convolutional neural networks, recurrent neural networks (RNN), fully-connected neural networks, transformer neural networks, and/or any other networks or a combination thereof. Various neurons of object detection model 130 may receive inputs from other neurons or from an external source and

may produce an output by computing a sum of weighted inputs and a bias value; the sum may then be used as an input into a non-linear activation function. In one illustrative example, weights and biases may initially be assigned random values that are modified during training of object detection model **130**.

[0028] Object detection model **130** may output any suitable digital representation of objects detected in the processed data, e.g., in video frames. The representations may include bounding boxes, contours, or some other pointers indicating locations of objects within the video frames. The representations may further include feature vectors representative of the visual appearance of the objects. In some implementations, the feature vectors may also represent some of the objects' surroundings (e.g., a portion of a background providing context for the objects).

[0029] Computer vision application **120** may further include bidirectional object tracking **140** that processes the representations of objects output by object detection model **130**. Bidirectional object tracking **140** tracks various detected objects in the normal chronological direction of frames (forward direction) in the inverse chronological direction (reverse direction) and compares, at individual frames, objects' trajectories and visual appearances to verify that correct objects are tracked in both directions. Bidirectional object tracking **140** may include estimating coordinates, velocities, visible sizes of the objects, and accelerations of the objects for different frames, and/or other information associated with motion and appearance of the objects.

[0030] Computer vision server **110** may be or include a desktop computer, a laptop computer, a smartphone, a tablet computer, a server, a computing device that accesses a remote server, a computing device that utilizes a virtualized computing environment, a gaming console, a wearable computer, a smart TV, and/or any combination thereof. Computer vision server **110** may have any number of central processing units (CPUs) **112**, graphics processing units (GPUs) **114**, parallel processing units (PPUs), data processing units (DPUs), or accelerators, and/or other suitable processing devices capable of performing the techniques described herein. Computer vision server **110** may include any number of memory devices, also referred to simply as memory **116** herein. Computer vision server **110** may store executable codes, libraries, and various dependencies of computer vision application **120** including bidirectional object tracking **140**. Computer vision application **120** may be executed by CPU **112**, GPU **114**, and/or other processing devices, or any combination thereof. CPU **112** and/or GPU **114** (or other processing devices) may support any number of virtual CPUs and/or virtual GPUs. For example, GPU **114** may include multiple cores, each core being capable of executing multiple GPU threads. Each core may run multiple threads concurrently (e.g., in parallel). In at least one embodiment, threads may have access to registers. Some or all cores may include a scheduler to distribute computational tasks and processes among different threads of the respective core. A dispatch unit may implement scheduled tasks on appropriate threads using various private registers and shared registers. In at least one embodiment, GPU **114** may have a (high-speed) cache, access to which may be shared by multiple cores. Furthermore, GPU **114** may include a GPU memory to store intermediate and/or final results (outputs) of various computations performed by GPU **114**. In some

embodiments, computer vision server **110** may also include network controllers, peripheral devices, and the like. Peripheral devices may include cameras (e.g., video cameras) for capturing images (or sequences of images), microphones for capturing sounds, scanners, sensors, or any other devices for intake of data.

[0031] In some embodiments, at least some portion of computer vision application **120** may be located (implemented or executed) directly on data acquisition device **102**, as illustrated by the respective dashed box in FIG. **1**. For example, pre-processing of data acquired by sensors **104-k** and/or object detection **130** may be fully or partially performed on data acquisition device **102**.

[0032] In some embodiments, computing architecture **100** may include a domain-specific application server **170** that receives, e.g., over network **160**, outputs of computer vision application **120**, e.g., tracks of detected objects. Domain-specific application server **170** may support one or more of a surveillance application, a public safety application, a sports analytics application, a manufacturing product tracking application, a warehouse application, a retail merchandise tracking application, a personal identification application, a mood/facial feature identification application, or the like, and/or any other application that uses object tracking.

[0033] In some embodiments, computer vision server **110** may process sensor data **152** obtained by data acquisition device **102** and stored in data store **150**. Data store **150** may be hosted by one or more storage devices, such as main memory, magnetic or optical storage disks, tapes, or hard drives, network-attached storage (NAS), storage area network (SAN), and so forth. Although depicted as separate from computer vision server **110**, in at least one embodiment, data store **150** may be a part of computer vision server **110**. In at least some embodiments, data store **150** may be a network-attached file server. In other embodiments, data store **150** may be some other type of persistent storage, such as an object-oriented database, a relational database, and so forth, that may be hosted by one or more other machines coupled to computer vision server **110** via network **160**.

[0034] FIG. **2** illustrates schematically a data flow **200** in an example computer vision system that deploys bidirectional tracking of objects, according to at least one embodiment. As depicted in FIG. **2**, one or more sensors **204**, e.g., video cameras, may generate one or more streams of raw data **210** (or minimally processed data). In some embodiments, raw video data may be compressed, e.g., using one or more video codecs, e.g., H.264, H.265, VP8, VP9, AV1, or some other video codec(s). Raw data **210** may include accompanying audio data compressed using one or more audio codecs, e.g., AAC, Opus, .mp3, etc. Raw data **210** may include two-dimensional (2D) images, e.g., projections of a three-dimensional environment onto a 2D projection surface (e.g., a detector array of a digital camera). In some embodiments, raw data **210** may include 3D images, e.g., images taken by lidars and/or radars (as part of sensors **204**) that include a distance to objects.

[0035] A decode/batch stage **220** may pre-process the stream(s) of raw data **210**, including but not limited to decoding the raw data, authenticating the raw data, denoising the raw data, filtering the raw data, enhancing the raw data, and/or the like. Decode/batch stage **220** may further aggregate, combine, batch, etc., multiple streams of raw data into an aggregated input stream **222**. Input stream **222** may include frames of a specific size, e.g., 1920×1080 pixels,

1280×720 pixels, and/or frames of any other supported size. In some embodiments, decode/batch stage **220** may generate frames of sizes that are specified as part of object detection model **130** settings. Available frame sizes may be determined by a size of an input data that the object detection model **130** may be configured (e.g., via a number of neurons in an input layer) and trained to process. To change the size of the input data, decode/batch stage **220** may apply one or more filters capable of downsampling or upsampling raw data **210**. Filters may apply various selectable interpolation or extrapolation schemes to compute pixel intensities of downsampled/upsampled pixels.

[0036] Object detection model **130** detects a presence of depictions of objects in video frames of input stream **222**. Detected objects **232** may be represented using geometric features and visual features. Geometric features may include bounding boxes (which may be 3D boxes, in the instances of 3D images), convex hulls, contours, or any other suitable pointers indicating locations of objects within the video frames. Visual features, referred to as feature vectors herein, may include any representations of visual appearance of the objects. In some embodiments, feature vectors may be obtained using histograms characterizing distribution of pixels of various intensities and/or colors associated with corresponding detected objects **232**. In some embodiments, feature vectors may be or include embeddings generated by a suitable neural network model (deployed as part of object detection model **130**), including but not limited to convolutional neural networks, fully-connected neural networks, recurrent neural networks, long short-term memory neural networks, networks with self-attention and/or cross attention mechanisms, including transformer networks, and/or the like. In some embodiments, feature vectors may represent not only the visual appearance of detected objects **232** themselves (e.g., content of bounding boxes) but may also characterize at least a portion of the environment, e.g., an area of a background and/or other neighboring objects visible near a given object.

[0037] Bidirectional object tracking **140** may process the geometric features (e.g., bounding boxes) and feature vectors and output object tracks **250**. In one example non-limiting embodiment, an object track $\text{Track}[\text{ID}, \{X(t_j), \text{Box}(t_j)\}]$ may refer to a data structure that is indexed by an object ID and includes the object's state of motion $X(t_j)$ and bounding box $\text{Box}(t_j)$ for a set of frames $\{t_j\}$ in which the object is visible. In some embodiments, the object track may further include an identification of frames in which the object is not visible, e.g., occluded by other objects. The state of motion $X(t_j)$ may refer to coordinates of a suitable reference point of the object (e.g., a geometric center/center of mass of the object), velocity, and/or acceleration of such reference point. $\text{Box}(t_j)$ may refer to a suitable geometric figure that compactly encloses the object during its motion. A size of $\text{Box}(t_j)$ may change with frame t_j as the object approaches or departs from sensor(s) **204**.

[0038] Object tracks **250** may be displayed on a graphical user interface (GUI) or any other visualizer **260** to a user/operator, directed to domain-specific application **270** (which may be supported by computer vision server **110** and/or domain-specific server **270** of FIG. **1**) for further processing, such as statistical analysis, decision-making, and/or the like, and/or stored in data store **150**.

[0039] FIG. **3** illustrates operations **300** of a bidirectional object tracking deployed by a computer vision system,

according to at least one embodiment. Operations **300** include operations of forward tracking **301** (also referred to as the forward direction herein) and reverse tracking **301-R** (referred to as the reverse direction herein). Various processing operations of reverse tracking **301-R** that are similar to respective operations of forward tracking **301** are indicated by character “-R” placed after the same numeral. Forward and direct tracking operations denoted with the same numeral may be performed by the same (or similar) software modules and/or hardware devices of the computer vision system. Operations **300** may include processing a frame **302**, e.g., frame t_j from a series of video frames $t_1, t_2, \dots, t_N$. Processing may include segmentation **304** of the frame into individual objects. In some implementations, segmentation **304** may be performed by object detection model **130** of FIG. **1** and FIG. **2**. Segmentation **304** may identify any number of objects in frame **302** and determine locators (e.g., bounding boxes) for individual identified objects. Feature vector extraction **306** may process the outputs of segmentation **304** and compute feature vectors for various identified objects. Although in FIG. **3**, segmentation **304-R** and feature vector extraction **306-R** are depicted as separate operations, in some implementations, these operations may be the same as corresponding operations of forward tracking **301**.

[0040] FIG. **4** illustrates schematically forward tracking **301** and reverse tracking **301-R** of an example object, according to at least one embodiment. FIG. **4** depicts motion of an object **400** (ball) captured in seven frames $t_{j-3}, \dots, t_{j+3}$ (the motion is not necessarily to scale), with frames $t_{j-3}, t_{j-2}, t_{j-1}$ leading up to frame t_j in the forward direction of tracking and frames $t_{j+3}, t_{j+2}, t_{j+1}$ leading up to frame t_j in the reverse direction of tracking. Although, for simplicity and conciseness, FIG. **4** depicts a single ball, a number of tracked objects need not be limited.

[0041] The forward direction and the reverse direction may maintain separate states of the object that are updated with each additional frame. The state of the object in the forward direction is denoted with $S(t_j)$. The state of the object may include an object ID, which may be a unique identifier (e.g., an alphanumeric, symbolic, or any other suitable identifier) assigned to an object. As long as the same object is being tracked, the same object ID is maintained for the object. New objects entering the field of view may receive different unique object IDs. The state of the object $S(t_j)$ may further include a state of motion of the object, $X(t_j)$, which may include object's coordinates, $x(t_j), y(t_j)$, velocity $v_x(t_j), v_y(t_j)$, acceleration, $a_x(t_j), a_y(t_j)$, bounding box, $\text{Box}(t_j)$, and/or the like. (In some implementations, the coordinates/velocities/acceleration may be 3D vectors with three rather than two components.) The state of the object $S(t_j)$ can also include a feature vector $F(t_j)$ representing a combined historical evolution of visual appearance of the object across the frames that precede frame t_j (e.g., frames $t_{j-3}, t_{j-2}, t_{j-1}$ and/or earlier frames not depicted in FIG. **4**). FIG. **4** illustrates schematically, for various frames, current locations of object **400** and previous locations **402** (for the forward direction of tracking) and **402-R** (for the reverse direction of tracking).

[0042] Referring again to FIG. **3**, feature vector extraction **306** may obtain feature vector $F_{obs}(t_j)$ associated with the object being tracked and describing observed appearance of the object in frame t_j . Historical features **308** store a feature vector $F(t_{j-1})$ obtained after processing the preceding frames

(up to frame t_{j-1}). Feature matching **310** may compare feature vector $F(t_{j-1})$ with the observed feature vector $F_{obs}(t_j)$ to determine whether the object in frame t_j is the same as (or different from) the object in frame t_{j-1} (and/or other preceding frames). The comparison may be performed using the cosine similarity (dot product) of the two vectors, $Sim = F(t_{j-1}) \cdot F_{obs}(t_j)$. If the two vectors are sufficiently similar, e.g., $Sim \geq Sim_T$, tracking decision module **320** may maintain the current object ID (from the state of the object for frame t_{j-1}) for current frame t_j . Feature update **330** may then update the historical feature vector, $F(t_{j-1}) \rightarrow F(t_j) = M(F(t_{j-1}), F_{obs}(t_j))$, using a suitable mapping function $M(\cdot)$. In some implementations, the mapping function may be a linear combination of the two feature vectors, e.g., $F(t_j) = (1-w)F(t_{j-1}) + wF_{obs}(t_j)$, where w is a weight given to the new observation (e.g., $w=0.1, 0.2, 0.5$, or any other empirically selected number) and $1-w$ is the weight given to the historical feature vector. The new value $F(t_j)$ is then used to update (e.g., replace) the previous value $F(t_{j-1})$ in historical features **308**.

[0043] If the two vectors $F(t_{j-1})$ and $F_{obs}(t_j)$ are dissimilar ($Sim < Sim_T$), a new object ID_{new} may be initiated for the observed object indicating that this new object (or a previously occluded object) has entered (re-entered) the field of view. Such an ID switch may also include initializing a new object's state and starting a new track for the new object ID_{new} based on the observed feature vector $F_{obs}(t_j)$ and the observed state of motion $X_{obs}(t_j)$. Initially, the observed state of motion may include just the coordinates of the new object and its bounding box. (If lidar/radar observations are available, the initial observed state of the new object's motion may also include the velocity of the new object.) The object ID and the most recent state of the previously tracked object $S(t_{j-1})$ may be stored, e.g., to account for a possibility that the object may be temporarily occluded. In such instances, the reappearing object may be given the same object ID and a state that may continue from the same state $S(t_{j-1})$. If a sufficiently long time will have passed, a new state for object ID may be initialized.

[0044] In some instances, the object ID may be switched to one of other existing object IDs, provided that the newly observed feature vector $F_{obs}(t_j)$ is sufficiently similar to the historical appearance feature vector $F(t_{j-1})$ for that other object in frame t_{j-1} (or one of earlier frames). In such instances, the observed feature vector $F_{obs}(t_j)$ and the observed position in frame t_j may be used to update the state of the other object, followed by tracking the object for subsequent frames, t_{j+2}, t_{j+3} , etc.

[0045] The object's state of motion $X(t_{j-1})$ may be used to predict (project), using state prediction module **340**, a new state of motion of the object $X_{pred}(t_j)$ for new frame t_j . This may be performed kinematically, based on known (tracked) coordinates, velocity, and (in some embodiments) acceleration of the object. The prediction $X_{pred}(t_j)$ may be used to assist cropping **304** in identifying one or more candidate objects in frame t_j that correspond to a specific object ID from frame t_{j-1} . Disambiguation from the candidate object may then be performed using feature matching module **310**, which selects the candidate object with the observed feature vector $F_{obs}(t_j)$ having the highest similarity to the historical feature vector $F(t_{j-1})$ for the object ID.

[0046] The predicted state of motion $X_{pred}(t_j)$ may be combined with the observed state $X_{obs}(t_j)$, which may be based on coordinates of the object in frame t_j , to estimate the

actual (the (e.g., the most probable) state of motion $X(t_j)$ of the object. In some embodiments, the actual state of motion may be obtained using a suitable statistical filter, e.g., a Kalman filter, a particle filter, and/or the like. The statistical filter obtains the most probable actual state of motion $X(t_j)$ by taking into account statistical corrections between observed and estimated actual state of motion (as a result of noise/errors of the measurements), e.g., by defining and using a covariance matrix for various quantities being tracked (coordinates, velocities, etc.). The covariance matrix may also be updated for each consecutive frame.

[0047] The reverse tracking **301-R** may be performed similarly, but in the opposite direction of time. The reverse tracking defines a separate reverse state, $S(t_j) = \{ID, \bar{X}(t_j), F(t_j)\}$, with object ID that may be different from object ID of the forward tracking **301**. (An advantage of having a reverse object ID for the reverse track that is different from a forward object ID—even though both IDs refer to the same object arises from the fact that the two tracks start from opposite ends of a series of frames where it may not be easy to determine that the objects tracked by the two tracks relate to the same object.) The object's state of motion $\bar{X}(t_{j+1})$ for the reverse track is used to predict, using state prediction **340-R**, the state of motion $\bar{X}_{pred}(t_j)$ for frame t_j (which, for the reverse track, represents a downstream frame relative to upstream frame t_{j+1}). Taken together with the observed state of motion $\bar{X}_{obs}(t_j)$, the predicted state of motion $\bar{X}_{pred}(t_j)$ is then used to update the object's tracked state of motion, $\bar{X}(t_j)$. As in the case of forward tracking, a suitable statistical filter may be used to make this prediction. Additionally, feature matching **310-R** and feature update **330-R** are performed. For example, a new feature vector $F_{obs}(t_j)$ representing appearance of the object in frame t_j (which may be the same as new feature vector for the forward track, $F_{obs}(t_j) = F_{obs}(t_j)$), as these feature vectors may correspond to and obtained by segmentation of the same frame t_j). The reverse track may update (provided that the same object is observed in frame t_j as was tracked in frames $t_{j+1}, t_{j+2}, \dots$) the historical feature vector, $F(t_{j+1}) \rightarrow F(t_j) = M(F(t_{j+1}), F_{obs}(t_j))$, e.g., using the same mapping function $M(\cdot)$ as for the forward tracking **301**.

[0048] Feature matching **310-R** may determine whether the same object is being tracked across different frames. For example, feature matching **310-R** may compute similarly of historical feature vector $F(t_{j+1})$ to the observed feature vector $F_{obs}(t_j)$. If the two vectors are sufficiently similar, e.g., with the corresponding similarity at or above Sim_T , the same object ID may be maintained for frame t_j as for frame t_{j+1} . If the two vectors $F(t_{j+1})$ and $F_{obs}(t_j)$ are dissimilar, the reverse track object ID may be stored together with the most recent state of the object $S(t_{j+1})$, to account for a possibility that the object may be temporarily occluded. In such instances, the reappearing object may be given the same object ID and a state that may continue from the same state $S(t_{j+1})$. If a sufficiently long time has passed, a new state for object ID may be initialized. In the meantime, a new object ID_{new} may be assigned to an object that is associated with the observed vector $F_{obs}(t_{j-1})$.

[0049] If tracking decision **320** determines that feature matching **310** indicates the same object is observed in frame t_j as in frame t_{j+1} , and tracking decision **320-R** determines that feature matching **310-R** indicates the same object is observed in frame t_j as in frame t_{j+1} , the continuity of tracking across frame t_j is established. Track fusion **350** may

then set object ID equal (or otherwise map object ID, e.g., by updating a mapping table) to object ID. In some implementations, mapping of object ID to object ID may be performed by mapping each of object ID and object ID to a global object ID 370. The global object ID may be used as an output of track fusion 350 for a given frame and may be provided to a computer vision application (e.g., computer vision application 120 of FIG. 1), e.g., as part of object tracking data. Track fusion 350 may additionally modify feature vectors $F(t_j)$ and $F(t_j)$, e.g., by computing a simple average $F_F(t_j)=[F(t_j)+F(t_j)]/2$, and use the fused feature vector $F_F(t_j)$ as part of the state of the object for both the forward direction and the reverse direction. In some implementations, the fused feature vector may be computed as a weighted average of the two feature vectors $F_F(t_j)=w_1F(t_j)+w_2F(t_j)$, with some suitably chosen weights w_1 and w_2 . In some embodiments, the weights may be proportional with a confidence level (e.g., probability) with which the respective feature matching 310 and 310-R have determined that the observed feature vector $F_{obs}(t_j)$ corresponds to the object being tracked in the two directions of tracking respectively.

[0050] Once it is determined that object ID for the forward tracking and object ID for the reverse tracking correspond to the same object, track fusion 350 may include mapping the two object IDs to each other, $ID \leftrightarrow ID$, and further mapping the two object IDs to a bidirectional (global) object ID₂: $ID \leftrightarrow ID_2 \leftrightarrow ID$. The bidirectional object ID₂ may be included in a final output of the bidirectional tracking (e.g., provided to a suitable domain-specific application 270 of FIG. 2). In some implementations, the bidirectional object ID₂ may be taken the same as object ID for the forward tracking.

[0051] In some implementations, if feature matching 310 for the forward tracking determines that the historical feature vector $F(t_{j-1})$ is dissimilar to observed feature vector $F_{obs}(t_j)$, this may potentially indicate that a new object (or a previously occluded object) has entered the field of view. Feature matching 310-R of the reverse tracking may then be used to verify if this potential ID switch is real or false. For example, if feature matching 310-R indicates that no respective ID switch occurs in the reverse direction, the forward ID switch may be deemed false. In such instances, the object ID from frame t_{j-1} may be maintained for frame t_j , and the observed feature vector $F_{obs}(t_j)$ and the observed state of motion $X_{obs}(t_j)$ may be used to update the state of the object $S(t_{j-1})$ to obtain the new state $S(t_j)$. Similarly, if feature matching 310-R indicates that a potential ID switch may be occurring in the reverse direction, but no matching ID switch occurs between frames in the forward direction, the reverse ID switch may be deemed false. In such instances, the object ID from frame t_{j+1} may be maintained for frame t_j , and the observed feature vector $F_{obs}(t_j)$ and the state of motion $X_{obs}(t_j)$ may be used to update the state of the object $S(t_{j-1})$ to obtain the new state $S(t_j)$. The reverse object ID may also be matched to the forward object ID.

[0052] If an ID switch is detected both in the forward direction and in the reverse direction, the ID switch may be deemed true. New identifications may then be assigned to the objects for both directions of tracking and a state reset 360 may be performed. For example, the object ID may be switched to a new identification, $ID \rightarrow ID_{new}$, indicating that a new object is now tracked in the forward direction. State reset 360 may include resetting the state of the object $S(t_{j-1})$, e.g., discarding (or storing) previous historical feature vector $F(t_{j-1})$ and object's state of motion $X(t_{j-1})$ and starting a new

track for new object ID_{new} based on the observed feature vector $F_{obs}(t_j)$ and the observed state of motion $X_{obs}(t_j)$. In some instances, the object ID may be switched to one of other existing object IDs, if the newly observed feature vector $F_{obs}(t_j)$ is sufficiently similar to the historical appearance feature vector $F(t_j)$ for that other object. In such instances, the observed feature vector $F_{obs}(t_j)$ and the observed position in frame t_j can be used to update the state of the other object, followed by tracking the object for subsequent frames, t_{j+2} , t_{j+3} , etc. A similar identification reset $ID \rightarrow ID_{new}$ and a state reset may be performed in the reverse direction.

[0053] FIGS. 5A-5B depict a flow diagram of an example method 500 of bidirectional tracking of objects in computer vision applications, according to at least one embodiment. Method 500 may be performed in the context of autonomous driving applications, industrial control applications, provisioning of streaming services, video monitoring services, surveillance and public/private safety services, artificial intelligence and machine learning services, mapping services, gaming services, virtual reality or augmented reality services, and many other contexts, and/or in systems and applications that deploy computer vision techniques. Method 500 may be performed using one or more processing units (e.g., CPUs, GPUs, accelerators, PPUs, DPUs, etc.), which may include (or communicate with) one or more memory devices. In at least one embodiment, method 500 may be performed using various components and devices illustrated in FIG. 1, e.g., computer vision server 110, and/or the like. Method 500 may be performed to process data received from data acquisition device 102 (e.g., data generated by one or more sensors 104-k). In at least one embodiment, processing units performing method 500 may be executing instructions stored on a non-transient computer-readable storage media. In at least one embodiment, method 500 may be performed using multiple processing threads (e.g., CPU threads and/or GPU threads), individual threads executing one or more individual functions, routines, subroutines, or operations of the method. In at least one embodiment, processing threads implementing method 500 may be synchronized (e.g., using semaphores, critical sections, and/or other thread synchronization mechanisms). Alternatively, processing threads implementing method 500 may be executed asynchronously with respect to each other. Various operations of method 500 may be performed in a different order compared with the order shown in FIGS. 5A-5B. Some operations of method 500 may be performed concurrently with other operations. In at least one embodiment, one or more operations shown in FIGS. 5A-5B may not always be performed.

[0054] As illustrated in FIG. 5A, method 500 may include, at block 510, obtaining digital representations of an object depicted in a plurality of video frames (e.g., video frame of input stream 222 in FIG. 2). The digital representations of the object may include feature vectors generated by an object detection machine learning model (e.g., object detection model 130) for respective video frames of the plurality of video frames. For example, feature vectors may be generated by feature vector extraction 306 in FIG. 3.

[0055] With blocks 520-540, method 500 may perform a bidirectional tracking of the object across the plurality of video frames. Some operations of the bidirectional tracking (e.g., operations of block 520) may be performed for both directions of bidirectional tracking, a forward direction (FD)

and a reverse direction (RD). The terms “upstream frame” and “downstream frame” should be understood as referring to a particular track. For example, for the FD of bidirectional tracking, a frame t_{j-1} may be an upstream frame and frame t_j may be a downstream frame. Similarly, for the RD of bidirectional tracking, a frame t_{j+1} may be an upstream frame and frame t_j may be the downstream frame.

[0056] At block 520, method 500 may include obtaining, using (i) a current state of the object associated with an upstream video frame (e.g., state $S(t_{j-1})$ for FD and/or state $\bar{S}(t_{j+1})$ for RD) and (ii) the digital representation of the object for a downstream video frame (e.g., observed feature vector $F_{obs}(t_j)$ for FD and/or observed feature vector $F_{obs}(t_j)$ for RD), an updated state of the object associated with the downstream video frame (e.g., state $S(t_j)$ for FD and/or state $\bar{S}(t_j)$ for RD). In some implementations, the current state of the object may include a feature vector representative of a depiction of the object in at least the upstream video frame, and a location of the object associated with the upstream video frame. For example, the current state $S(t_{j-1})$ for FD may include feature vector $F(t_{j-1})$ that is built using one or more feature vectors observed in previous video frames, $F_{obs}(t_{j-1})$, $F_{obs}(t_{j-2})$, etc. In some implementations, the current state of the object $S(t_{j-1})$ may further include a size of the object. For example, the location and the size of the object associated with the upstream video frame may be represented via a bounding box Box_{j-1} for the object associated with the upstream video frame t_{j-1} . In some implementations, the current state of the object $S(t_{j-1})$ may further include a velocity of the object $V(t_{j-1})$ associated with the upstream video frame. In some implementations, the location of the object, the size of the object and the velocity of the object may be determined using a statistical filter (such as a Kalman filter).

[0057] At block 530, method 500 may include obtaining a bidirectional state of the object. As described below in conjunction with FIG. 5B, the bidirectional state may be obtained using the updated state of the object for the FD (e.g., state $S(t_j)$), or the updated state of the object for the RD (e.g., state $\bar{S}(t_j)$), or both. At block 540, method 500 may include determining, using the bidirectional state of the object, a trajectory of the object across the plurality of video frames. In some implementations, method 500 may continue with generating a human-perceivable report that is based, at least in part, on the trajectory of the object, generating a statistical report that is based, at least in part, on the trajectory of the object, or generating a video that depicts, at least a portion of the trajectory of the object (e.g., segmenting, from a larger video, a video clip depicting the motion of the object).

[0058] Operations of block 530 may include a number of operations and scenarios that are depicted in FIG. 5B. In some implementations, obtaining the bidirectional state of the object may include using a FD similarity and a RD similarity. More specifically, at block 532, method 500 may include computing the FD similarity between an FD feature vector associated with the current state of the object for the FD (e.g., feature vector $F(t_{j-1})$ for frame t_{j-1}) of the bidirectional tracking and a feature vector associated with the digital representation of the object for the downstream frame (e.g., observed feature vector $F_{obs}(t_j)$ for frame t_j). Similarly, at block 534, method 500 may include computing the RD similarity between an RD feature vector associated with the current state of the object for the RD (e.g., feature vector

$F(t_{j+1})$ for frame t_{j+1}) of the bidirectional tracking and the feature vector associated with the digital representation of the object for the downstream frame (e.g., observed feature vector $F_{obs}(t_j)$ for frame t_j).

[0059] As illustrated by the left branch in FIG. 5B, in some instances, method 500 may include, at block 540 determining that each of the FD similarity and the RD similarity is above a threshold similarity. At block 542, method 500 may include modifying the updated state for the FD (e.g., $F(t_{j-1})$) using the updated state for the RD (e.g., $F(t_j)$), and may further include modifying the updated state for the RD using the updated state for the FD. In some implementations, the modified update states for the FD and the RD may be the same, e.g., equal to an average of both updated states, $[F(t_j) + F(t_{j-1})]/2$. In some implementations, the modified update states for the FD and the RD may be different, e.g., the modified updated states may be a weighted averages of the updated states, $w_{FD}F(t_j) + (1 - w_{FD})F(t_{j-1})$, and $(1 - w_{RD})F(t_j) + w_{RD}F(t_{j-1})$, with suitably chosen weights w_{FD} and w_{RD} .

[0060] As illustrated by the central branch in FIG. 5B, in some instances, method 500 may include, at block 550, determining that the FD similarity (or the RD similarity) is above a threshold similarity and that the RD similarity (or the FD similarity) is below the threshold similarity. At block 552, method 500 may include resetting the state of the object for the RD (or the FD), which may include storing the current state of the object for the RD, $\bar{S}(t_{j+1})$ (or the current state of the object for the FD, $S(t_{j-1})$). At block 554, method 500 may include assigning a new object identification (ID) for the RD (or the FD) of the bidirectional tracking.

[0061] As illustrated by the right branch in FIG. 5B, in some instances, method 500 may include, at block 560, determining that each of the FD similarity and the RD similarity is below a threshold similarity. At block 562, method 500 may continue with resetting the updated state of the object for the FD and the RD. At block 564, method 500 may include assigning new object IDs to each of the FD and RD of the bidirectional tracking.

Autonomous Vehicle

[0062] FIG. 6A illustrates an example of an autonomous vehicle 600, according to at least one embodiment. In at least one embodiment, autonomous vehicle 600 (alternatively referred to herein as “vehicle 600”) may be, without limitation, a passenger vehicle, such as a car, a truck, a bus, and/or another type of vehicle that accommodates one or more passengers. In at least one embodiment, vehicle 600 may be a semi-tractor-trailer truck used for hauling cargo. In at least one embodiment, vehicle 600 may be an airplane, robotic vehicle, or other kind of vehicle.

[0063] Autonomous vehicles may be described in terms of automation levels, defined by National Highway Traffic Safety Administration (“NHTSA”), a division of US Department of Transportation, and Society of Automotive Engineers (“SAE”) “Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles” (e.g., Standard No. J3016-201806, published on Jun. 15, 2018, Standard No. J3016-201609, published on Sep. 30, 2016, and previous and future versions of this standard). In at least one embodiment, vehicle 600 may be capable of functionality in accordance with one or more of Level 1 through Level 5 of autonomous driving levels. For example, in at least one embodiment, vehicle 600 may be

capable of conditional automation (Level 3), high automation (Level 4), and/or full automation (Level 5), depending on embodiment.

[0064] In at least one embodiment, vehicle **600** may include, without limitation, components such as a chassis, a vehicle body, wheels (e.g., 2, 4, 6, 8, 18, etc.), tires, axles, and other components of a vehicle. In at least one embodiment, vehicle **600** may include, without limitation, a propulsion system **650**, such as an internal combustion engine, hybrid electric power plant, an all-electric engine, and/or another propulsion system type. In at least one embodiment, propulsion system **650** may be connected to a drive train of vehicle **600**, which may include, without limitation, a transmission, to enable propulsion of vehicle **600**. In at least one embodiment, propulsion system **650** may be controlled in response to receiving signals from a throttle/accelerator(s) **652**.

[0065] In at least one embodiment, a steering system **654**, which may include, without limitation, a steering wheel, is used to steer vehicle **600** (e.g., along a desired path or route) when propulsion system **650** is operating (e.g., when vehicle **600** is in motion). In at least one embodiment, steering system **654** may receive signals from steering actuator(s) **656**. In at least one embodiment, a steering wheel may be optional for full automation (Level 5) functionality. In at least one embodiment, a brake sensor system **646** may be used to operate vehicle brakes in response to receiving signals from brake actuator(s) **648** and/or brake sensors.

[0066] In at least one embodiment, controller(s) **636**, which may include, without limitation, one or more system on chips (“SoCs”) (not shown in FIG. 6A) and/or graphics processing unit(s) (“GPU(s)”), provide signals (e.g., representative of commands) to one or more components and/or systems of vehicle **600**. For instance, in at least one embodiment, controller(s) **636** may send signals to operate vehicle brakes via brake actuator(s) **648**, to operate steering system **654** via steering actuator(s) **656**, to operate propulsion system **650** via throttle/accelerator(s) **652**. In at least one embodiment, controller(s) **636** may include one or more onboard (e.g., integrated) computing devices that process sensor signals, and output operation commands (e.g., signals representing commands) to enable autonomous driving and/or to assist a human driver in driving vehicle **600**. In at least one embodiment, controller(s) **636** may include a first controller for autonomous driving functions, a second controller for functional safety functions, a third controller for artificial intelligence functionality (e.g., computer vision), a fourth controller for infotainment functionality, a fifth controller for redundancy in emergency conditions, and/or other controllers. In at least one embodiment, a single controller may handle two or more of above functionalities, two or more controllers may handle a single functionality, and/or any combination thereof.

[0067] In at least one embodiment, controller(s) **636** provide signals for controlling one or more components and/or systems of vehicle **600** in response to sensor data received from one or more sensors (e.g., sensor inputs). In at least one embodiment, sensor data may be received from, for example and without limitation, global navigation satellite systems (“GNSS”) sensor(s) **658** (e.g., Global Positioning System sensor(s)), RADAR sensor(s) **660**, ultrasonic sensor(s) **662**, LIDAR sensor(s) **664**, inertial measurement unit (“IMU”) sensor(s) **666** (e.g., accelerometer(s), gyroscope(s), a magnetic compass or magnetic compasses, magnetometer(s),

etc.), microphone(s) **696**, stereo camera(s) **668**, wide-view camera(s) **670** (e.g., fisheye cameras), infrared camera(s) **672**, surround camera(s) **674** (e.g., 360 degree cameras), long-range cameras (not shown in FIG. 6A), mid-range camera(s) (not shown in FIG. 6A), speed sensor(s) **644** (e.g., for measuring speed of vehicle **600**), vibration sensor(s) **642**, steering sensor(s) **640**, brake sensor(s) (e.g., as part of brake sensor system **646**), and/or other sensor types.

[0068] In at least one embodiment, one or more of controller(s) **636** may receive inputs (e.g., represented by input data) from an instrument cluster **632** of vehicle **600** and provide outputs (e.g., represented by output data, display data, etc.) via a human-machine interface (“HMI”) display **634**, an audible annunciator, a loudspeaker, and/or via other components of vehicle **600**. In at least one embodiment, outputs may include information such as vehicle velocity, speed, time, map data (e.g., a High Definition map (not shown in FIG. 6A), location data (e.g., vehicle’s **600** location, such as on a map), direction, location of other vehicles (e.g., an occupancy grid), information about objects and status of objects as perceived by controller(s) **636**, etc. For example, in at least one embodiment, HMI display **634** may display information about presence of one or more objects (e.g., a street sign, caution sign, traffic light changing, etc.), and/or information about driving maneuvers vehicle has made, is making, or will make (e.g., changing lanes now, taking exit **34B** in two miles, etc.).

[0069] In at least one embodiment, vehicle **600** further includes a network interface **624** which may use wireless antenna(s) **626** and/or modem(s) to communicate over one or more networks. For example, in at least one embodiment, network interface **624** may be capable of communication over Long-Term Evolution (“LTE”), Wideband Code Division Multiple Access (“WCDMA”), Universal Mobile Telecommunications System (“UMTS”), Global System for Mobile communication (“GSM”), IMT-CDMA Multi-Carrier (“CDMA2000”) networks, etc. In at least one embodiment, wireless antenna(s) **626** may also enable communication between objects in environment (e.g., vehicles, mobile devices, etc.), using local area network(s), such as Bluetooth, Bluetooth Low Energy (“LE”), Z-Wave, ZigBee, etc., and/or low power wide-area network(s) (“LPWANs”), such as LoRaWAN, SigFox, etc. protocols.

[0070] FIG. 6B illustrates an example of camera locations and fields of view for autonomous vehicle **600** of FIG. 6A, according to at least one embodiment. In at least one embodiment, cameras and respective fields of view are one example embodiment and are not intended to be limiting. For instance, in at least one embodiment, additional and/or alternative cameras may be included and/or cameras may be located at different locations on vehicle **600**.

[0071] In at least one embodiment, camera types for cameras may include, but are not limited to, digital cameras that may be adapted for use with components and/or systems of vehicle **600**. In at least one embodiment, camera(s) may operate at automotive safety integrity level (“ASIL”) B and/or at another ASIL. In at least one embodiment, camera types may be capable of any image capture rate, such as 60 frames per second (fps), 120 fps, 240 fps, etc., depending on embodiment. In at least one embodiment, cameras may be capable of using rolling shutters, global shutters, another type of shutter, or a combination thereof. In at least one embodiment, color filter array may include a red clear clear clear (“RCCC”) color filter array, a red clear clear blue

(“RCCB”) color filter array, a red blue green clear (“RBGC”) color filter array, a Foveon X3 color filter array, a Bayer sensors (“RGGB”) color filter array, a monochrome sensor color filter array, and/or another type of color filter array. In at least one embodiment, clear pixel cameras, such as cameras with an RCCC, an RCCB, and/or an RBGC color filter array, may be used in an effort to increase light sensitivity.

[0072] In at least one embodiment, one or more of camera(s) may be used to perform advanced driver assistance systems (“ADAS”) functions (e.g., as part of a redundant or fail-safe design). For example, in at least one embodiment, a Multi-Function Mono Camera may be installed to provide functions including lane departure warning, traffic sign assist and intelligent headlamp control. In at least one embodiment, one or more of camera(s) (e.g., all cameras) may record and provide image data (e.g., video) simultaneously.

[0073] In at least one embodiment, one or more camera may be mounted in a mounting assembly, such as a custom designed (three-dimensional (“3D”) printed) assembly, in order to cut out stray light and reflections from within vehicle 600 (e.g., reflections from dashboard reflected in windshield mirrors) which may interfere with camera image data capture abilities. With reference to wing-mirror mounting assemblies, in at least one embodiment, wing-mirror assemblies may be custom 3D printed so that a camera mounting plate matches a shape of a wing-mirror. In at least one embodiment, camera(s) may be integrated into wing-mirrors. In at least one embodiment, for side-view cameras, camera(s) may also be integrated within four pillars at each corner of a cabin.

[0074] In at least one embodiment, cameras with a field of view that include portions of an environment in front of vehicle 600 (e.g., front-facing cameras) may be used for surround view, to help identify forward facing paths and obstacles, as well as aid in, with help of one or more of controller(s) 636 and/or control SoCs, providing information critical to generating an occupancy grid and/or determining preferred vehicle paths. In at least one embodiment, front-facing cameras may be used to perform many similar ADAS functions as LIDAR, including, without limitation, emergency braking, pedestrian detection, and collision avoidance. In at least one embodiment, front-facing cameras may also be used for ADAS functions and systems including, without limitation, Lane Departure Warnings (“LDW”), Autonomous Cruise Control (“ACC”), and/or other functions such as traffic sign recognition.

[0075] In at least one embodiment, a variety of cameras may be used in a front-facing configuration, including, for example, a monocular camera platform that includes a CMOS (“complementary metal oxide semiconductor”) color imager. In at least one embodiment, a wide-view camera 670 may be used to perceive objects coming into view from a periphery (e.g., pedestrians, crossing traffic or bicycles). Although only one wide-view camera 670 is illustrated in FIG. 6B, in other embodiments, there may be any number (including zero) wide-view cameras on vehicle 600. In at least one embodiment, any number of long-range camera(s) 698 (e.g., a long-view stereo camera pair) may be used for depth-based object detection, especially for objects for which a neural network has not yet been trained. In at least

one embodiment, long-range camera(s) 698 may also be used for object detection and classification, as well as basic object tracking.

[0076] In at least one embodiment, any number of stereo camera(s) 668 may also be included in a front-facing configuration. In at least one embodiment, one or more of stereo camera(s) 668 may include an integrated control unit comprising a scalable processing unit, which may provide a programmable logic (“FPGA”) and a multi-core micro-processor with an integrated Controller Area Network (“CAN”) or Ethernet interface on a single chip. In at least one embodiment, such a unit may be used to generate a 3D map of an environment of vehicle 600, including a distance estimate for all points in an image. In at least one embodiment, one or more of stereo camera(s) 668 may include, without limitation, compact stereo vision sensor(s) that may include, without limitation, two camera lenses (one each on left and right) and an image processing chip that may measure distance from vehicle 600 to target object and use generated information (e.g., metadata) to activate autonomous emergency braking and lane departure warning functions. In at least one embodiment, other types of stereo camera(s) 668 may be used in addition to, or alternatively from, those described herein.

[0077] In at least one embodiment, cameras with a field of view that include portions of environment to sides of vehicle 600 (e.g., side-view cameras) may be used for surround view, providing information used to create and update an occupancy grid, as well as to generate side impact collision warnings. For example, in at least one embodiment, surround camera(s) 674 (e.g., four surround cameras as illustrated in FIG. 6B) could be positioned on vehicle 600. In at least one embodiment, surround camera(s) 674 may include, without limitation, any number and combination of wide-view cameras, fisheye camera(s), 360 degree camera(s), and/or similar cameras. For instance, in at least one embodiment, four fisheye cameras may be positioned on a front, a rear, and sides of vehicle 600. In at least one embodiment, vehicle 600 may use three surround camera(s) 674 (e.g., left, right, and rear), and may leverage one or more other camera(s) (e.g., a forward-facing camera) as a fourth surround-view camera.

[0078] In at least one embodiment, cameras with a field of view that include portions of an environment behind vehicle 600 (e.g., rear-view cameras) may be used for parking assistance, surround view, rear collision warnings, and creating and updating an occupancy grid. In at least one embodiment, a wide variety of cameras may be used including, but not limited to, cameras that are also suitable as a front-facing camera(s) (e.g., long-range cameras 698 and/or mid-range camera(s) 676, stereo camera(s) 668), infrared camera(s) 672, etc.), as described herein.

[0079] FIG. 6C is a block diagram illustrating an example system architecture for autonomous vehicle 600 of FIG. 6A, according to at least one embodiment. In at least one embodiment, each of components, features, and systems of vehicle 600 in FIG. 6C is illustrated as being connected via a bus 602. In at least one embodiment, bus 602 may include, without limitation, a CAN data interface (alternatively referred to herein as a “CAN bus”). In at least one embodiment, a CAN may be a network inside vehicle 600 used to aid in control of various features and functionality of vehicle 600, such as actuation of brakes, acceleration, braking, steering, windshield wipers, etc. In at least one embodiment,

bus 602 may be configured to have dozens or even hundreds of nodes, each with its own unique identifier (e.g., a CAN ID). In at least one embodiment, bus 602 may be read to find steering wheel angle, ground speed, engine revolutions per minute (“RPMs”), button positions, and/or other vehicle status indicators. In at least one embodiment, bus 602 may be a CAN bus that is ASIL B compliant.

[0080] In at least one embodiment, in addition to, or alternatively from CAN, FlexRay and/or Ethernet protocols may be used. In at least one embodiment, there may be any number of busses forming bus 602, which may include, without limitation, zero or more CAN busses, zero or more FlexRay busses, zero or more Ethernet busses, and/or zero or more other types of busses using different protocols. In at least one embodiment, two or more busses may be used to perform different functions, and/or may be used for redundancy. For example, a first bus may be used for collision avoidance functionality and a second bus may be used for actuation control. In at least one embodiment, each bus of bus 602 may communicate with any of components of vehicle 600, and two or more busses of bus 602 may communicate with corresponding components. In at least one embodiment, each of any number of system(s) on chip(s) (“SoC(s)”) 604 (such as SoC 604(A) and SoC 604(B), each of controller(s) 636, and/or each computer within vehicle may have access to same input data (e.g., inputs from sensors of vehicle 600), and may be connected to a common bus, such as CAN bus.

[0081] In at least one embodiment, vehicle 600 may include one or more controller(s) 636, such as those described herein with respect to FIG. 6A. In at least one embodiment, controller(s) 636 may be used for a variety of functions. In at least one embodiment, controller(s) 636 may be coupled to any of various other components and systems of vehicle 600, and may be used for control of vehicle 600, artificial intelligence of vehicle 600, infotainment for vehicle 600, and/or other functions.

[0082] In at least one embodiment, vehicle 600 may include any number of SoCs 604. In at least one embodiment, each of SoCs 604 may include, without limitation, central processing units (“CPU(s)”) 606, graphics processing units (“GPU(s)”) 608, processor(s) 610, cache(s) 612, accelerator(s) 614, data store(s) 616, and/or other components and features not illustrated. In at least one embodiment, SoC(s) 604 may be used to control vehicle 600 in a variety of platforms and systems. For example, in at least one embodiment, SoC(s) 604 may be combined in a system (e.g., system of vehicle 600) with a High Definition (“HD”) map 622 which may obtain map refreshes and/or updates via network interface 624 from one or more servers (not shown in FIG. 6C).

[0083] In at least one embodiment, CPU(s) 606 may include a CPU cluster or CPU complex (alternatively referred to herein as a “CCPLEX”). In at least one embodiment, CPU(s) 606 may include multiple cores and/or level two (“L2”) caches. For instance, in at least one embodiment, CPU(s) 606 may include eight cores in a coherent multiprocessor configuration. In at least one embodiment, CPU(s) 606 may include four dual-core clusters where each cluster has a dedicated L2 cache (e.g., a 2 megabyte (MB) L2 cache). In at least one embodiment, CPU(s) 606 (e.g., CCPLEX) may be configured to support simultaneous cluster operations enabling any combination of clusters of CPU(s) 606 to be active at any given time.

[0084] In at least one embodiment, one or more of CPU(s) 606 may implement power management capabilities that include, without limitation, one or more of following features: individual hardware blocks may be clock-gated automatically when idle to save dynamic power; each core clock may be gated when such core is not actively executing instructions due to execution of Wait for Interrupt (“WFI”)/Wait for Event (“WFE”) instructions; each core may be independently power-gated; each core cluster may be independently clock-gated when all cores are clock-gated or power-gated; and/or each core cluster may be independently power-gated when all cores are power-gated. In at least one embodiment, CPU(s) 606 may further implement an enhanced algorithm for managing power states, where allowed power states and expected wakeup times are specified, and hardware/microcode determines which best power state to enter for core, cluster, and CCPLEX. In at least one embodiment, processing cores may support simplified power state entry sequences in software with work offloaded to microcode.

[0085] In at least one embodiment, GPU(s) 608 may include an integrated GPU (alternatively referred to herein as an “iGPU”). In at least one embodiment, GPU(s) 608 may be programmable and may be efficient for parallel workloads. In at least one embodiment, GPU(s) 608 may use an enhanced tensor instruction set. In at least one embodiment, GPU(s) 608 may include one or more streaming microprocessors, where each streaming microprocessor may include a level one (“L1”) cache (e.g., an L1 cache with at least 96 KB storage capacity), and two or more streaming microprocessors may share an L2 cache (e.g., an L2 cache with a 512 KB storage capacity). In at least one embodiment, GPU(s) 608 may include at least eight streaming microprocessors. In at least one embodiment, GPU(s) 608 may use compute application programming interface(s) (API(s)). In at least one embodiment, GPU(s) 608 may use one or more parallel computing platforms and/or programming models (e.g., NVIDIA’s CUDA model).

[0086] In at least one embodiment, one or more of GPU(s) 608 may be power-optimized for best performance in automotive and embedded use cases. For example, in at least one embodiment, GPU(s) 608 could be fabricated on Fin field-effect transistor (“FinFET”) circuitry. In at least one embodiment, each streaming microprocessor may incorporate a number of mixed-precision processing cores partitioned into multiple blocks. For example, and without limitation, 64 PF32 cores and 32 PF64 cores could be partitioned into four processing blocks. In at least one embodiment, each processing block could be allocated 16 FP32 cores, 8 FP64 cores, 16 INT32 cores, two mixed-precision NVIDIA Tensor cores for deep learning matrix arithmetic, a level zero (“L0”) instruction cache, a warp scheduler, a dispatch unit, and/or a 64 KB register file. In at least one embodiment, streaming microprocessors may include independent parallel integer and floating-point data paths to provide for efficient execution of workloads with a mix of computation and addressing calculations. In at least one embodiment, streaming microprocessors may include independent thread scheduling capability to enable finer-grain synchronization and cooperation between parallel threads. In at least one embodiment, streaming microprocessors may include a combined L1 data cache and shared memory unit in order to improve performance while simplifying programming.

[0087] In at least one embodiment, one or more of GPU(s) 608 may include a high bandwidth memory (“HBM”) and/or a 16 GB high-bandwidth memory second generation (“HBM2”) memory subsystem to provide, in some examples, about 900 GB/second peak memory bandwidth. In at least one embodiment, in addition to, or alternatively from, HBM memory, a synchronous graphics random-access memory (“SGRAM”) may be used, such as a graphics double data rate type five synchronous random-access memory (“GDDR5”).

[0088] In at least one embodiment, GPU(s) 608 may include unified memory technology. In at least one embodiment, address translation services (“ATS”) support may be used to allow GPU(s) 608 to access CPU(s) 606 page tables directly. In at least one embodiment, when a GPU of GPU(s) 608 memory management unit (“MMU”) experiences a miss, an address translation request may be transmitted to CPU(s) 606. In response, 2 CPU of CPU(s) 606 may look in its page tables for a virtual-to-physical mapping for an address and transmit translation back to GPU(s) 608, in at least one embodiment. In at least one embodiment, unified memory technology may allow a single unified virtual address space for memory of both CPU(s) 606 and GPU(s) 608, thereby simplifying GPU(s) 608 programming and porting of applications to GPU(s) 608.

[0089] In at least one embodiment, GPU(s) 608 may include any number of access counters that may keep track of frequency of access of GPU(s) 608 to memory of other processors. In at least one embodiment, access counter(s) may help ensure that memory pages are moved to physical memory of a processor that is accessing pages most frequently, thereby improving efficiency for memory ranges shared between processors.

[0090] In at least one embodiment, one or more of SoC(s) 604 may include any number of cache(s) 612, including those described herein. For example, in at least one embodiment, cache(s) 612 could include a level three (“L3”) cache that is available to both CPU(s) 606 and GPU(s) 608 (e.g., that is connected to CPU(s) 606 and GPU(s) 608). In at least one embodiment, cache(s) 612 may include a write-back cache that may keep track of states of lines, such as by using a cache coherence protocol (e.g., MEI, MESI, MSI, etc.). In at least one embodiment, a L3 cache may include 4 MB of memory or more, depending on embodiment, although smaller cache sizes may be used.

[0091] In at least one embodiment, one or more of SoC(s) 604 may include one or more accelerator(s) 614 (e.g., hardware accelerators, software accelerators, or a combination thereof). In at least one embodiment, SoC(s) 604 may include a hardware acceleration cluster that may include optimized hardware accelerators and/or large on-chip memory. In at least one embodiment, large on-chip memory (e.g., 4 MB of SRAM), may enable a hardware acceleration cluster to accelerate neural networks and other calculations. In at least one embodiment, a hardware acceleration cluster may be used to complement GPU(s) 608 and to off-load some of tasks of GPU(s) 608 (e.g., to free up more cycles of GPU(s) 608 for performing other tasks). In at least one embodiment, accelerator(s) 614 could be used for targeted workloads (e.g., perception, convolutional neural networks (“CNNs”), recurrent neural networks (“RNNs”), etc.) that are stable enough to be amenable to acceleration. In at least one embodiment, a CNN may include a region-based or

regional convolutional neural networks (“RCNNs”) and Fast RCNNs (e.g., as used for object detection) or other type of CNN.

[0092] In at least one embodiment, accelerator(s) 614 (e.g., hardware acceleration cluster) may include one or more deep learning accelerator (“DLA”). In at least one embodiment, DLA(s) may include, without limitation, one or more Tensor processing units (“TPUs”) that may be configured to provide an additional ten trillion operations per second for deep learning applications and inferencing. In at least one embodiment, TPUs may be accelerators configured to, and optimized for, performing image processing functions (e.g., for CNNs, RCNNs, etc.). In at least one embodiment, DLA(s) may further be optimized for a specific set of neural network types and floating point operations, as well as inferencing. In at least one embodiment, design of DLA(s) may provide more performance per millimeter than a typical general-purpose GPU, and typically vastly exceeds performance of a CPU. In at least one embodiment, TPU(s) may perform several functions, including a single-instance convolution function, supporting, for example, INT8, INT16, and FP16 data types for both features and weights, as well as post-processor functions. In at least one embodiment, DLA(s) may quickly and efficiently execute neural networks, especially CNNs, on processed or unprocessed data for any of a variety of functions, including, for example and without limitation: a CNN for object identification and detection using data from camera sensors; a CNN for distance estimation using data from camera sensors; a CNN for emergency vehicle detection and identification and detection using data from microphones; a CNN for facial recognition and vehicle owner identification using data from camera sensors; and/or a CNN for security and/or safety related events.

[0093] In at least one embodiment, DLA(s) may perform any function of GPU(s) 608, and by using an inference accelerator, for example, a designer may target either DLA(s) or GPU(s) 608 for any function. For example, in at least one embodiment, a designer may focus processing of CNNs and floating point operations on DLA(s) and leave other functions to GPU(s) 608 and/or accelerator(s) 614.

[0094] In at least one embodiment, accelerator(s) 614 may include programmable vision accelerator (“PVA”), which may alternatively be referred to herein as a computer vision accelerator. In at least one embodiment, PVA may be designed and configured to accelerate computer vision algorithms for advanced driver assistance system (“ADAS”) 638, autonomous driving, augmented reality (“AR”) applications, and/or virtual reality (“VR”) applications. In at least one embodiment, PVA may provide a balance between performance and flexibility. For example, in at least one embodiment, each PVA may include, for example and without limitation, any number of reduced instruction set computer (“RISC”) cores, direct memory access (“DMA”), and/or any number of vector processors.

[0095] In at least one embodiment, RISC cores may interact with image sensors (e.g., image sensors of any cameras described herein), image signal processor(s), etc. In at least one embodiment, each RISC core may include any amount of memory. In at least one embodiment, RISC cores may use any of a number of protocols, depending on embodiment. In at least one embodiment, RISC cores may execute a real-time operating system (“RTOS”). In at least one embodiment, RISC cores may be implemented using one or more

integrated circuit devices, application specific integrated circuits (“ASICs”), and/or memory devices. For example, in at least one embodiment, RISC cores could include an instruction cache and/or a tightly coupled RAM.

[0096] In at least one embodiment, DMA may enable components of PVA to access system memory independently of CPU(s) **606**. In at least one embodiment, DMA may support any number of features used to provide optimization to a PVA including, but not limited to, supporting multi-dimensional addressing and/or circular addressing. In at least one embodiment, DMA may support up to six or more dimensions of addressing, which may include, without limitation, block width, block height, block depth, horizontal block stepping, vertical block stepping, and/or depth stepping.

[0097] In at least one embodiment, vector processors may be programmable processors that may be designed to efficiently and flexibly execute programming for computer vision algorithms and provide signal processing capabilities. In at least one embodiment, a PVA may include a PVA core and two vector processing subsystem partitions. In at least one embodiment, a PVA core may include a processor subsystem, DMA engine(s) (e.g., two DMA engines), and/or other peripherals. In at least one embodiment, a vector processing subsystem may operate as a primary processing engine of a PVA, and may include a vector processing unit (“VPU”), an instruction cache, and/or vector memory (e.g., “VMEM”). In at least one embodiment, VPU core may include a digital signal processor such as, for example, a single instruction, multiple data (“SIMD”), very long instruction word (“VLIW”) digital signal processor. In at least one embodiment, a combination of SIMD and VLIW may enhance throughput and speed.

[0098] In at least one embodiment, each of vector processors may include an instruction cache and may be coupled to dedicated memory. As a result, in at least one embodiment, each of vector processors may be configured to execute independently of other vector processors. In at least one embodiment, vector processors that are included in a particular PVA may be configured to employ data parallelism. For instance, in at least one embodiment, plurality of vector processors included in a single PVA may execute a common computer vision algorithm, but on different regions of an image. In at least one embodiment, vector processors included in a particular PVA may simultaneously execute different computer vision algorithms, on one image, or even execute different algorithms on sequential images or portions of an image. In at least one embodiment, among other things, any number of PVAs may be included in hardware acceleration cluster and any number of vector processors may be included in each PVA. In at least one embodiment, PVA may include additional error correcting code (“ECC”) memory, to enhance overall system safety.

[0099] In at least one embodiment, accelerator(s) **614** may include a computer vision network on-chip and static random-access memory (“SRAM”), for providing a high-bandwidth, low latency SRAM for accelerator(s) **614**. In at least one embodiment, on-chip memory may include at least 4 MB SRAM, comprising, for example and without limitation, eight field-configurable memory blocks, that may be accessible by both a PVA and a DLA. In at least one embodiment, each pair of memory blocks may include an advanced peripheral bus (“APB”) interface, configuration circuitry, a controller, and a multiplexer. In at least one embodiment,

any type of memory may be used. In at least one embodiment, a PVA and a DLA may access memory via a backbone that provides a PVA and a DLA with high-speed access to memory. In at least one embodiment, a backbone may include a computer vision network on-chip that interconnects a PVA and a DLA to memory (e.g., using APB).

[0100] In at least one embodiment, a computer vision network on-chip may include an interface that determines, before transmission of any control signal/address/data, that both a PVA and a DLA provide ready and valid signals. In at least one embodiment, an interface may provide for separate phases and separate channels for transmitting control signals/addresses/data, as well as burst-type communications for continuous data transfer. In at least one embodiment, an interface may comply with International Organization for Standardization (“ISO”) 26262 or International Electrotechnical Commission (“IEC”) 61508 standards, although other standards and protocols may be used.

[0101] In at least one embodiment, one or more of SoC(s) **604** may include a real-time ray-tracing hardware accelerator. In at least one embodiment, real-time ray-tracing hardware accelerator may be used to quickly and efficiently determine positions and extents of objects (e.g., within a world model), to generate real-time visualization simulations, for RADAR signal interpretation, for sound propagation synthesis and/or analysis, for simulation of SONAR systems, for general wave propagation simulation, for comparison to LIDAR data for purposes of localization and/or other functions, and/or for other uses.

[0102] In at least one embodiment, accelerator(s) **614** can have a wide array of uses for autonomous driving. In at least one embodiment, a PVA may be used for key processing stages in ADAS and autonomous vehicles. In at least one embodiment, a PVA’s capabilities are a good match for algorithmic domains needing predictable processing, at low power and low latency. In other words, a PVA performs well on semi-dense or dense regular computation, even on small data sets, which might require predictable run-times with low latency and low power. In at least one embodiment, such as in vehicle **600**, PVAs might be designed to run classic computer vision algorithms, as they can be efficient at object detection and operating on integer math.

[0103] For example, according to at least one embodiment of technology, a PVA is used to perform computer stereo vision. In at least one embodiment, a semi-global matching-based algorithm may be used in some examples, although this is not intended to be limiting. In at least one embodiment, applications for Level 3-5 autonomous driving use motion estimation/stereo matching on-the-fly (e.g., structure from motion, pedestrian recognition, lane detection, etc.). In at least one embodiment, a PVA may perform computer stereo vision functions on inputs from two monocular cameras.

[0104] In at least one embodiment, a PVA may be used to perform dense optical flow. For example, in at least one embodiment, a PVA could process raw RADAR data (e.g., using a 4D Fast Fourier Transform) to provide processed RADAR data. In at least one embodiment, a PVA is used for time of flight depth processing, by processing raw time of flight data to provide processed time of flight data, for example.

[0105] In at least one embodiment, a DLA may be used to run any type of network to enhance control and driving safety, including for example and without limitation, a

neural network that outputs a measure of confidence for each object detection. In at least one embodiment, confidence may be represented or interpreted as a probability, or as providing a relative “weight” of each detection compared to other detections. In at least one embodiment, a confidence measure enables a system to make further decisions regarding which detections should be considered as true positive detections rather than false positive detections. In at least one embodiment, a system may set a threshold value for confidence and consider only detections exceeding threshold value as true positive detections. In an embodiment in which an automatic emergency braking (“AEB”) system is used, false positive detections would cause vehicle to automatically perform emergency braking, which is obviously undesirable. In at least one embodiment, highly confident detections may be considered as triggers for AEB. In at least one embodiment, a DLA may run a neural network for regressing confidence value. In at least one embodiment, neural network may take as its input at least some subset of parameters, such as bounding box dimensions, ground plane estimate obtained (e.g., from another subsystem), output from IMU sensor(s) 666 that correlates with vehicle 600 orientation, distance, 3D location estimates of object obtained from neural network and/or other sensors (e.g., LIDAR sensor(s) 664 or RADAR sensor(s) 660), among others.

[0106] In at least one embodiment, one or more of SoC(s) 604 may include data store(s) 616 (e.g., memory). In at least one embodiment, data store(s) 616 may be on-chip memory of SoC(s) 604, which may store neural networks to be executed on GPU(s) 608 and/or a DLA. In at least one embodiment, data store(s) 616 may be large enough in capacity to store multiple instances of neural networks for redundancy and safety. In at least one embodiment, data store(s) 616 may comprise L2 or L3 cache(s).

[0107] In at least one embodiment, one or more of SoC(s) 604 may include any number of processor(s) 610 (e.g., embedded processors). In at least one embodiment, processor(s) 66 may include a boot and power management processor that may be a dedicated processor and subsystem to handle boot power and management functions and related security enforcement. In at least one embodiment, a boot and power management processor may be a part of a boot sequence of SoC(s) 604 and may provide runtime power management services. In at least one embodiment, a boot power and management processor may provide clock and voltage programming, assistance in system low power state transitions, management of SoC(s) 604 thermals and temperature sensors, and/or management of SoC(s) 604 power states. In at least one embodiment, each temperature sensor may be implemented as a ring-oscillator whose output frequency is proportional to temperature, and SoC(s) 604 may use ring-oscillators to detect temperatures of CPU(s) 606, GPU(s) 608, and/or accelerator(s) 614. In at least one embodiment, if temperatures are determined to exceed a threshold, then a boot and power management processor may enter a temperature fault routine and put SoC(s) 604 into a lower power state and/or put vehicle 600 into a chauffeur to safe stop mode (e.g., bring vehicle 600 to a safe stop).

[0108] In at least one embodiment, processor(s) 610 may further include a set of embedded processors that may serve as an audio processing engine which may be an audio subsystem that enables full hardware support for multi-

channel audio over multiple interfaces, and a broad and flexible range of audio I/O interfaces. In at least one embodiment, an audio processing engine is a dedicated processor core with a digital signal processor with dedicated RAM.

[0109] In at least one embodiment, processor(s) 610 may further include an always-on processor engine that may provide necessary hardware features to support low power sensor management and wake use cases. In at least one embodiment, an always-on processor engine may include, without limitation, a processor core, a tightly coupled RAM, supporting peripherals (e.g., timers and interrupt controllers), various I/O controller peripherals, and routing logic.

[0110] In at least one embodiment, processor(s) 610 may further include a safety cluster engine that includes, without limitation, a dedicated processor subsystem to handle safety management for automotive applications. In at least one embodiment, a safety cluster engine may include, without limitation, two or more processor cores, a tightly coupled RAM, support peripherals (e.g., timers, an interrupt controller, etc.), and/or routing logic. In a safety mode, two or more cores may operate, in at least one embodiment, in a lockstep mode and function as a single core with comparison logic to detect any differences between their operations. In at least one embodiment, processor(s) 610 may further include a real-time camera engine that may include, without limitation, a dedicated processor subsystem for handling real-time camera management. In at least one embodiment, processor(s) 610 may further include a high-dynamic range signal processor that may include, without limitation, an image signal processor that is a hardware engine that is part of a camera processing pipeline.

[0111] In at least one embodiment, processor(s) 610 may include a video image compositor that may be a processing block (e.g., implemented on a microprocessor) that implements video post-processing functions needed by a video playback application to produce a final image for a player window. In at least one embodiment, a video image compositor may perform lens distortion correction on wide-view camera(s) 670, surround camera(s) 674, and/or on in-cabin monitoring camera sensor(s). In at least one embodiment, in-cabin monitoring camera sensor(s) are preferably monitored by a neural network running on another instance of SoC 604, configured to identify in cabin events and respond accordingly. In at least one embodiment, an in-cabin system may perform, without limitation, lip reading to activate cellular service and place a phone call, dictate emails, change a vehicle’s destination, activate or change a vehicle’s infotainment system and settings, or provide voice-activated web surfing. In at least one embodiment, certain functions are available to a driver when a vehicle is operating in an autonomous mode and are disabled otherwise.

[0112] In at least one embodiment, a video image compositor may include enhanced temporal noise reduction for both spatial and temporal noise reduction. For example, in at least one embodiment, where motion occurs in a video, noise reduction weights spatial information appropriately, decreasing weights of information provided by adjacent frames. In at least one embodiment, where an image or portion of an image does not include motion, temporal noise reduction performed by video image compositor may use information from a previous image to reduce noise in a current image.

[0113] In at least one embodiment, a video image compositor may also be configured to perform stereo rectifica-

tion on input stereo lens frames. In at least one embodiment, a video image compositor may further be used for user interface composition when an operating system desktop is in use, and GPU(s) 608 are not required to continuously render new surfaces. In at least one embodiment, when GPU(s) 608 are powered on and active doing 3D rendering, a video image compositor may be used to offload GPU(s) 608 to improve performance and responsiveness.

[0114] In at least one embodiment, one or more SoC of SoC(s) 604 may further include a mobile industry processor interface (“MIPI”) camera serial interface for receiving video and input from cameras, a high-speed interface, and/or a video input block that may be used for a camera and related pixel input functions. In at least one embodiment, one or more of SoC(s) 604 may further include an input/output controller(s) that may be controlled by software and may be used for receiving I/O signals that are uncommitted to a specific role.

[0115] In at least one embodiment, one or more of SoC(s) 604 may further include a broad range of peripheral interfaces to enable communication with peripherals, audio encoders/decoders (“codecs”), power management, and/or other devices. In at least one embodiment, SoC(s) 604 may be used to process data from cameras (e.g., connected over Gigabit Multimedia Serial Link and Ethernet channels), sensors (e.g., LIDAR sensor(s) 664, RADAR sensor(s) 660, etc. that may be connected over Ethernet channels), data from bus 602 (e.g., speed of vehicle 600, steering wheel position, etc.), data from GNSS sensor(s) 658 (e.g., connected over a Ethernet bus or a CAN bus), etc. In at least one embodiment, one or more SoC of SoC(s) 604 may further include dedicated high-performance mass storage controllers that may include their own DMA engines, and that may be used to free CPU(s) 606 from routine data management tasks.

[0116] In at least one embodiment, SoC(s) 604 may be an end-to-end platform with a flexible architecture that spans automation Levels 3-5, thereby providing a comprehensive functional safety architecture that leverages and makes efficient use of computer vision and ADAS techniques for diversity and redundancy, and provides a platform for a flexible, reliable driving software stack, along with deep learning tools. In at least one embodiment, SoC(s) 604 may be faster, more reliable, and even more energy-efficient and space-efficient than conventional systems. For example, in at least one embodiment, accelerator(s) 614, when combined with CPU(s) 606, GPU(s) 608, and data store(s) 616, may provide for a fast, efficient platform for Level 3-5 autonomous vehicles.

[0117] In at least one embodiment, computer vision algorithms may be executed on CPUs, which may be configured using a high-level programming language, such as C, to execute a wide variety of processing algorithms across a wide variety of visual data. However, in at least one embodiment, CPUs are oftentimes unable to meet performance requirements of many computer vision applications, such as those related to execution time and power consumption, for example. In at least one embodiment, many CPUs are unable to execute complex object detection algorithms in real-time, which is used in in-vehicle ADAS applications and in practical Level 3-5 autonomous vehicles.

[0118] Embodiments described herein allow for multiple neural networks to be performed simultaneously and/or sequentially, and for results to be combined together to

enable Level 3-5 autonomous driving functionality. For example, in at least one embodiment, a CNN executing on a DLA or a discrete GPU (e.g., GPU(s) 620) may include text and word recognition, allowing reading and understanding of traffic signs, including signs for which a neural network has not been specifically trained. In at least one embodiment, a DLA may further include a neural network that is able to identify, interpret, and provide semantic understanding of a sign, and to pass that semantic understanding to path planning modules running on a CPU Complex.

[0119] In at least one embodiment, multiple neural networks may be run simultaneously, as for Level 3, 4, or 5 driving. For example, in at least one embodiment, a warning sign stating “Caution: flashing lights indicate icy conditions,” along with an electric light, may be independently or collectively interpreted by several neural networks. In at least one embodiment, such warning sign itself may be identified as a traffic sign by a first deployed neural network (e.g., a neural network that has been trained), text “flashing lights indicate icy conditions” may be interpreted by a second deployed neural network, which informs a vehicle’s path planning software (preferably executing on a CPU Complex) that when flashing lights are detected, icy conditions exist. In at least one embodiment, a flashing light may be identified by operating a third deployed neural network over multiple frames, informing a vehicle’s path-planning software of a presence (or an absence) of flashing lights. In at least one embodiment, all three neural networks may run simultaneously, such as within a DLA and/or on GPU(s) 608.

[0120] In at least one embodiment, a CNN for facial recognition and vehicle owner identification may use data from camera sensors to identify presence of an authorized driver and/or owner of vehicle 600. In at least one embodiment, an always-on sensor processing engine may be used to unlock a vehicle when an owner approaches a driver door and turns on lights, and, in a security mode, to disable such vehicle when an owner leaves such vehicle. In this way, SoC(s) 604 provide for security against theft and/or carjacking.

[0121] In at least one embodiment, a CNN for emergency vehicle detection and identification may use data from microphones 696 to detect and identify emergency vehicle sirens. In at least one embodiment, SoC(s) 604 use a CNN for classifying environmental and urban sounds, as well as classifying visual data. In at least one embodiment, a CNN running on a DLA is trained to identify a relative closing speed of an emergency vehicle (e.g., by using a Doppler effect). In at least one embodiment, a CNN may also be trained to identify emergency vehicles specific to a local area in which a vehicle is operating, as identified by GNSS sensor(s) 658. In at least one embodiment, when operating in Europe, a CNN will seek to detect European sirens, and when in North America, a CNN will seek to identify only North American sirens. In at least one embodiment, once an emergency vehicle is detected, a control program may be used to execute an emergency vehicle safety routine, slowing a vehicle, pulling over to a side of a road, parking a vehicle, and/or idling a vehicle, with assistance of ultrasonic sensor(s) 662, until emergency vehicles pass.

[0122] In at least one embodiment, vehicle 600 may include CPU(s) 618 (e.g., discrete CPU(s), or dCPU(s)), that may be coupled to SoC(s) 604 via a high-speed interconnect

(e.g., PCIe). In at least one embodiment, CPU(s) **618** may include an X86 processor, for example. CPU(s) **618** may be used to perform any of a variety of functions, including arbitrating potentially inconsistent results between ADAS sensors and SoC(s) **604**, and/or monitoring status and health of controller(s) **636** and/or an infotainment system on a chip (“infotainment SoC”) **630**, for example.

[0123] In at least one embodiment, vehicle **600** may include GPU(s) **620** (e.g., discrete GPU(s), or dGPU(s)), that may be coupled to SoC(s) **604** via a high-speed interconnect (e.g., NVIDIA’s NVLINK channel). In at least one embodiment, GPU(s) **620** may provide additional artificial intelligence functionality, such as by executing redundant and/or different neural networks, and may be used to train and/or update neural networks based at least in part on input (e.g., sensor data) from sensors of a vehicle **600**.

[0124] In at least one embodiment, vehicle **600** may further include network interface **624** which may include, without limitation, wireless antenna(s) **626** (e.g., one or more wireless antennas for different communication protocols, such as a cellular antenna, a Bluetooth antenna, etc.). In at least one embodiment, network interface **624** may be used to enable wireless connectivity to Internet cloud services (e.g., with server(s) and/or other network devices), with other vehicles, and/or with computing devices (e.g., client devices of passengers). In at least one embodiment, to communicate with other vehicles, a direct link may be established between vehicle **600** and another vehicle and/or an indirect link may be established (e.g., across networks and over the Internet). In at least one embodiment, direct links may be provided using a vehicle-to-vehicle communication link. In at least one embodiment, a vehicle-to-vehicle communication link may provide vehicle **600** information about vehicles in proximity to vehicle **600** (e.g., vehicles in front of, on a side of, and/or behind vehicle **600**). In at least one embodiment, such aforementioned functionality may be part of a cooperative adaptive cruise control functionality of vehicle **600**.

[0125] In at least one embodiment, network interface **624** may include an SoC that provides modulation and demodulation functionality and enables controller(s) **636** to communicate over wireless networks. In at least one embodiment, network interface **624** may include a radio frequency front-end for up-conversion from baseband to radio frequency, and down conversion from radio frequency to baseband. In at least one embodiment, frequency conversions may be performed in any technically feasible fashion. For example, frequency conversions could be performed through well-known processes, and/or using super-heterodyne processes. In at least one embodiment, radio frequency front end functionality may be provided by a separate chip. In at least one embodiment, network interfaces may include wireless functionality for communicating over LTE, WCDMA, UMTS, GSM, CDMA2000, Bluetooth, Bluetooth LE, Wi-Fi, Z-Wave, ZigBee, LoRaWAN, and/or other wireless protocols.

[0126] In at least one embodiment, vehicle **600** may further include data store(s) **628** which may include, without limitation, off-chip (e.g., off SoC(s) **604**) storage. In at least one embodiment, data store(s) **628** may include, without limitation, one or more storage elements including RAM, SRAM, dynamic random-access memory (“DRAM”), video

random-access memory (“VRAM”), flash memory, hard disks, and/or other components and/or devices that may store at least one bit of data.

[0127] In at least one embodiment, vehicle **600** may further include GNSS sensor(s) **658** (e.g., GPS and/or assisted GPS sensors), to assist in mapping, perception, occupancy grid generation, and/or path planning functions. In at least one embodiment, any number of GNSS sensor(s) **658** may be used, including, for example and without limitation, a GPS using a Universal Serial Bus (“USB”) connector with an Ethernet-to-Serial (e.g., RS-232) bridge.

[0128] In at least one embodiment, vehicle **600** may further include RADAR sensor(s) **660**. In at least one embodiment, RADAR sensor(s) **660** may be used by vehicle **600** for long-range vehicle detection, even in darkness and/or severe weather conditions. In at least one embodiment, RADAR functional safety levels may be ASIL B. In at least one embodiment, RADAR sensor(s) **660** may use a CAN bus and/or bus **602** (e.g., to transmit data generated by RADAR sensor(s) **660**) for control and to access object tracking data, with access to Ethernet channels to access raw data in some examples. In at least one embodiment, a wide variety of RADAR sensor types may be used. For example, and without limitation, RADAR sensor(s) **660** may be suitable for front, rear, and side RADAR use. In at least one embodiment, one or more sensor of RADAR sensors(s) **660** is a Pulse Doppler RADAR sensor.

[0129] In at least one embodiment, RADAR sensor(s) **660** may include different configurations, such as long-range with narrow field of view, short-range with wide field of view, short-range side coverage, etc. In at least one embodiment, long-range RADAR may be used for adaptive cruise control functionality. In at least one embodiment, long-range RADAR systems may provide a broad field of view realized by two or more independent scans, such as within a 250 m (meter) range. In at least one embodiment, RADAR sensor(s) **660** may help in distinguishing between static and moving objects, and may be used by ADAS system **638** for emergency brake assist and forward collision warning. In at least one embodiment, sensors **660(s)** included in a long-range RADAR system may include, without limitation, monostatic multimodal RADAR with multiple (e.g., six or more) fixed RADAR antennae and a high-speed CAN and FlexRay interface. In at least one embodiment, with six antennae, a central four antennae may create a focused beam pattern, designed to record vehicle’s **600** surroundings at higher speeds with minimal interference from traffic in adjacent lanes. In at least one embodiment, another two antennae may expand field of view, making it possible to quickly detect vehicles entering or leaving a lane of vehicle **600**.

[0130] In at least one embodiment, mid-range RADAR systems may include, as an example, a range of up to 160 m (front) or 80 m (rear), and a field of view of up to 42 degrees (front) or 150 degrees (rear). In at least one embodiment, short-range RADAR systems may include, without limitation, any number of RADAR sensor(s) **660** designed to be installed at both ends of a rear bumper. When installed at both ends of a rear bumper, in at least one embodiment, a RADAR sensor system may create two beams that constantly monitor blind spots in a rear direction and next to a vehicle. In at least one embodiment, short-range RADAR systems may be used in ADAS system **638** for blind spot detection and/or lane change assist.

[0131] In at least one embodiment, vehicle 600 may further include ultrasonic sensor(s) 662. In at least one embodiment, ultrasonic sensor(s) 662, which may be positioned at a front, a back, and/or side location of vehicle 600, may be used for parking assist and/or to create and update an occupancy grid. In at least one embodiment, a wide variety of ultrasonic sensor(s) 662 may be used, and different ultrasonic sensor(s) 662 may be used for different ranges of detection (e.g., 2.5 m, 4 m). In at least one embodiment, ultrasonic sensor(s) 662 may operate at functional safety levels of ASIL B.

[0132] In at least one embodiment, vehicle 600 may include LIDAR sensor(s) 664. In at least one embodiment, LIDAR sensor(s) 664 may be used for object and pedestrian detection, emergency braking, collision avoidance, and/or other functions. In at least one embodiment, LIDAR sensor(s) 664 may operate at functional safety level ASIL B. In at least one embodiment, vehicle 600 may include multiple LIDAR sensors 664 (e.g., two, four, six, etc.) that may use an Ethernet channel (e.g., to provide data to a Gigabit Ethernet switch).

[0133] In at least one embodiment, LIDAR sensor(s) 664 may be capable of providing a list of objects and their distances for a 360-degree field of view. In at least one embodiment, commercially available LIDAR sensor(s) 664 may have an advertised range of approximately 100 m, with an accuracy of 2 cm to 3 cm, and with support for a 100 Mbps Ethernet connection, for example. In at least one embodiment, one or more non-protruding LIDAR sensors may be used. In such an embodiment, LIDAR sensor(s) 664 may include a small device that may be embedded into a front, a rear, a side, and/or a corner location of vehicle 600. In at least one embodiment, LIDAR sensor(s) 664, in such an embodiment, may provide up to a 104-degree horizontal and 35-degree vertical field-of-view, with a 200 m range even for low-reflectivity objects. In at least one embodiment, front-mounted LIDAR sensor(s) 664 may be configured for a horizontal field of view between 45 degrees and 135 degrees.

[0134] In at least one embodiment, LIDAR technologies, such as 3D flash LIDAR, may also be used. In at least one embodiment, 3D flash LIDAR uses a flash of a laser as a transmission source, to illuminate surroundings of vehicle 600 up to approximately 200 m. In at least one embodiment, a flash LIDAR unit includes, without limitation, a receptor, which records laser pulse transit time and reflected light on each pixel, which in turn corresponds to a range from vehicle 600 to objects. In at least one embodiment, flash LIDAR may allow for highly accurate and distortion-free images of surroundings to be generated with every laser flash. In at least one embodiment, four flash LIDAR sensors may be deployed, one at each side of vehicle 600. In at least one embodiment, 3D flash LIDAR systems include, without limitation, a solid-state 3D staring array LIDAR camera with no moving parts other than a fan (e.g., a non-scanning LIDAR device). In at least one embodiment, flash LIDAR device may use a 5 nanosecond class I (eye-safe) laser pulse per frame and may capture reflected laser light as a 3D range point cloud and co-registered intensity data.

[0135] In at least one embodiment, vehicle 600 may further include IMU sensor(s) 666. In at least one embodiment, IMU sensor(s) 666 may be located at a center of a rear axle of vehicle 600. In at least one embodiment, IMU sensor(s) 666 may include, for example and without limitation,

accelerometer(s), magnetometer(s), gyroscope(s), a magnetic compass, magnetic compasses, and/or other sensor types. In at least one embodiment, such as in six-axis applications, IMU sensor(s) 666 may include, without limitation, accelerometers and gyroscopes. In at least one embodiment, such as in nine-axis applications, IMU sensor(s) 666 may include, without limitation, accelerometers, gyroscopes, and magnetometers.

[0136] In at least one embodiment, IMU sensor(s) 666 may be implemented as a miniature, high performance GPS-Aided Inertial Navigation System (“GPS/INS”) that combines micro-electro-mechanical systems (“MEMS”) inertial sensors, a high-sensitivity GPS receiver, and advanced Kalman filtering algorithms to provide estimates of position, velocity, and attitude. In at least one embodiment, IMU sensor(s) 666 may enable vehicle 600 to estimate its heading without requiring input from a magnetic sensor by directly observing and correlating changes in velocity from a GPS to IMU sensor(s) 666. In at least one embodiment, IMU sensor(s) 666 and GNSS sensor(s) 658 may be combined in a single integrated unit.

[0137] In at least one embodiment, vehicle 600 may include microphone(s) 696 placed in and/or around vehicle 600. In at least one embodiment, microphone(s) 696 may be used for emergency vehicle detection and identification, among other things.

[0138] In at least one embodiment, vehicle 600 may further include any number of camera types, including stereo camera(s) 668, wide-view camera(s) 670, infrared camera(s) 672, surround camera(s) 674, long-range camera(s) 698, mid-range camera(s) 676, and/or other camera types. In at least one embodiment, cameras may be used to capture image data around an entire periphery of vehicle 600. In at least one embodiment, which types of cameras used depends on vehicle 600. In at least one embodiment, any combination of camera types may be used to provide necessary coverage around vehicle 600. In at least one embodiment, a number of cameras deployed may differ depending on embodiment. For example, in at least one embodiment, vehicle 600 could include six cameras, seven cameras, ten cameras, twelve cameras, or another number of cameras. In at least one embodiment, cameras may support, as an example and without limitation, Gigabit Multimedia Serial Link (“GMSL”) and/or Gigabit Ethernet communications. In at least one embodiment, each camera might be as described with more detail previously herein with respect to FIG. 6A and FIG. 6B.

[0139] In at least one embodiment, vehicle 600 may further include vibration sensor(s) 642. In at least one embodiment, vibration sensor(s) 642 may measure vibrations of components of vehicle 600, such as axle(s). For example, in at least one embodiment, changes in vibrations may indicate a change in road surfaces. In at least one embodiment, when two or more vibration sensors 642 are used, differences between vibrations may be used to determine friction or slippage of road surface (e.g., when a difference in vibration is between a power-driven axle and a freely rotating axle).

[0140] In at least one embodiment, vehicle 600 may include ADAS system 638. In at least one embodiment, ADAS system 638 may include, without limitation, an SoC, in some examples. In at least one embodiment, ADAS system 638 may include, without limitation, any number and combination of an autonomous/adaptive/automatic cruise

control (“ACC”) system, a cooperative adaptive cruise control (“CACC”) system, a forward crash warning (“FCW”) system, an automatic emergency braking (“AEB”) system, a lane departure warning (“LDW”) system, a lane keep assist (“LKA”) system, a blind spot warning (“BSW”) system, a rear cross-traffic warning (“RCTW”) system, a collision warning (“CW”) system, a lane centering (“LC”) system, and/or other systems, features, and/or functionality.

[0141] In at least one embodiment, ACC system may use RADAR sensor(s) 660, LIDAR sensor(s) 664, and/or any number of camera(s). In at least one embodiment, ACC system may include a longitudinal ACC system and/or a lateral ACC system. In at least one embodiment, a longitudinal ACC system monitors and controls distance to another vehicle immediately ahead of vehicle 600 and automatically adjusts speed of vehicle 600 to maintain a safe distance from vehicles ahead. In at least one embodiment, a lateral ACC system performs distance keeping, and advises vehicle 600 to change lanes when necessary. In at least one embodiment, a lateral ACC is related to other ADAS applications, such as LC and CW.

[0142] In at least one embodiment, a CACC system uses information from other vehicles that may be received via network interface 624 and/or wireless antenna(s) 626 from other vehicles via a wireless link, or indirectly, over a network connection (e.g., over the Internet). In at least one embodiment, direct links may be provided by a vehicle-to-vehicle (“V2V”) communication link, while indirect links may be provided by an infrastructure-to-vehicle (“I2V”) communication link. In general, V2V communication provides information about immediately preceding vehicles (e.g., vehicles immediately ahead of and in same lane as vehicle 600), while I2V communication provides information about traffic further ahead. In at least one embodiment, a CACC system may include either or both I2V and V2V information sources. In at least one embodiment, given information of vehicles ahead of vehicle 600, a CACC system may be more reliable and it has potential to improve traffic flow smoothness and reduce congestion on road.

[0143] In at least one embodiment, an FCW system is designed to alert a driver to a hazard, so that such driver may take corrective action. In at least one embodiment, an FCW system uses a front-facing camera and/or RADAR sensor(s) 660, coupled to a dedicated processor, digital signal processor (“DSP”), FPGA, and/or ASIC, that is electrically coupled to provide driver feedback, such as a display, speaker, and/or vibrating component. In at least one embodiment, an FCW system may provide a warning, such as in form of a sound, visual warning, vibration and/or a quick brake pulse.

[0144] In at least one embodiment, an AEB system detects an impending forward collision with another vehicle or other object, and may automatically apply brakes if a driver does not take corrective action within a specified time or distance parameter. In at least one embodiment, AEB system may use front-facing camera(s) and/or RADAR sensor(s) 660, coupled to a dedicated processor, DSP, FPGA, and/or ASIC. In at least one embodiment, when an AEB system detects a hazard, it will typically first alert a driver to take corrective action to avoid collision and, if that driver does not take corrective action, that AEB system may automatically apply brakes in an effort to prevent, or at least mitigate, an impact of a predicted collision. In at least one embodiment, an AEB

system may include techniques such as dynamic brake support and/or crash imminent braking.

[0145] In at least one embodiment, an LDW system provides visual, audible, and/or tactile warnings, such as steering wheel or seat vibrations, to alert driver when vehicle 600 crosses lane markings. In at least one embodiment, an LDW system does not activate when a driver indicates an intentional lane departure, such as by activating a turn signal. In at least one embodiment, an LDW system may use front-side facing cameras, coupled to a dedicated processor, DSP, FPGA, and/or ASIC, that is electrically coupled to provide driver feedback, such as a display, speaker, and/or vibrating component. In at least one embodiment, an LKA system is a variation of an LDW system. In at least one embodiment, an LKA system provides steering input or braking to correct vehicle 600 if vehicle 600 starts to exit its lane.

[0146] In at least one embodiment, a BSW system detects and warns a driver of vehicles in an automobile’s blind spot. In at least one embodiment, a BSW system may provide a visual, audible, and/or tactile alert to indicate that merging or changing lanes is unsafe. In at least one embodiment, a BSW system may provide an additional warning when a driver uses a turn signal. In at least one embodiment, a BSW system may use rear-side facing camera(s) and/or RADAR sensor(s) 660, coupled to a dedicated processor, DSP, FPGA, and/or ASIC, that is electrically coupled to driver feedback, such as a display, speaker, and/or vibrating component.

[0147] In at least one embodiment, an RCTW system may provide visual, audible, and/or tactile notification when an object is detected outside a rear-camera range when vehicle 600 is backing up. In at least one embodiment, an RCTW system includes an AEB system to ensure that vehicle brakes are applied to avoid a crash. In at least one embodiment, an RCTW system may use one or more rear-facing RADAR sensor(s) 660, coupled to a dedicated processor, DSP, FPGA, and/or ASIC, that is electrically coupled to provide driver feedback, such as a display, speaker, and/or vibrating component.

[0148] In at least one embodiment, conventional ADAS systems may be prone to false positive results which may be annoying and distracting to a driver, but typically are not catastrophic, because conventional ADAS systems alert a driver and allow that driver to decide whether a safety condition truly exists and act accordingly. In at least one embodiment, vehicle 600 itself decides, in case of conflicting results, whether to heed result from a primary computer or a secondary computer (e.g., a first controller or a second controller of controllers 636). For example, in at least one embodiment, ADAS system 638 may be a backup and/or secondary computer for providing perception information to a backup computer rationality module. In at least one embodiment, a backup computer rationality monitor may run redundant diverse software on hardware components to detect faults in perception and dynamic driving tasks. In at least one embodiment, outputs from ADAS system 638 may be provided to a supervisory MCU. In at least one embodiment, if outputs from a primary computer and outputs from a secondary computer conflict, a supervisory MCU determines how to reconcile conflict to ensure safe operation.

[0149] In at least one embodiment, a primary computer may be configured to provide a supervisory MCU with a confidence score, indicating that primary computer’s confidence in a chosen result. In at least one embodiment, if that confidence score exceeds a threshold, that supervisory MCU

may follow that primary computer's direction, regardless of whether that secondary computer provides a conflicting or inconsistent result. In at least one embodiment, where a confidence score does not meet a threshold, and where primary and secondary computers indicate different results (e.g., a conflict), a supervisory MCU may arbitrate between computers to determine an appropriate outcome.

[0150] In at least one embodiment, a supervisory MCU may be configured to run a neural network(s) that is trained and configured to determine, based at least in part on outputs from a primary computer and outputs from a secondary computer, conditions under which that secondary computer provides false alarms. In at least one embodiment, neural network(s) in a supervisory MCU may learn when a secondary computer's output may be trusted, and when it cannot. For example, in at least one embodiment, when that secondary computer is a RADAR-based FCW system, a neural network(s) in that supervisory MCU may learn when an FCW system is identifying metallic objects that are not, in fact, hazards, such as a drainage grate or manhole cover that triggers an alarm. In at least one embodiment, when a secondary computer is a camera-based LDW system, a neural network in a supervisory MCU may learn to override LDW when bicyclists or pedestrians are present and a lane departure is, in fact, a safest maneuver. In at least one embodiment, a supervisory MCU may include at least one of a DLA or a GPU suitable for running neural network(s) with associated memory. In at least one embodiment, a supervisory MCU may comprise and/or be included as a component of SoC(s) **604**.

[0151] In at least one embodiment, ADAS system **638** may include a secondary computer that performs ADAS functionality using traditional rules of computer vision. In at least one embodiment, that secondary computer may use classic computer vision rules (if-then), and presence of a neural network(s) in a supervisory MCU may improve reliability, safety and performance. For example, in at least one embodiment, diverse implementation and intentional non-identity makes an overall system more fault-tolerant, especially to faults caused by software (or software-hardware interface) functionality. For example, in at least one embodiment, if there is a software bug or error in software running on a primary computer, and non-identical software code running on a secondary computer provides a consistent overall result, then a supervisory MCU may have greater confidence that an overall result is correct, and a bug in software or hardware on that primary computer is not causing a material error.

[0152] In at least one embodiment, an output of ADAS system **638** may be fed into a primary computer's perception block and/or a primary computer's dynamic driving task block. For example, in at least one embodiment, if ADAS system **638** indicates a forward crash warning due to an object immediately ahead, a perception block may use this information when identifying objects. In at least one embodiment, a secondary computer may have its own neural network that is trained and thus reduces a risk of false positives, as described herein.

[0153] In at least one embodiment, vehicle **600** may further include infotainment SoC **630** (e.g., an in-vehicle infotainment system (IVI)). Although illustrated and described as an SoC, infotainment system SoC **630**, in at least one embodiment, may not be an SoC, and may include, without limitation, two or more discrete components. In at

least one embodiment, infotainment SoC **630** may include, without limitation, a combination of hardware and software that may be used to provide audio (e.g., music, a personal digital assistant, navigational instructions, news, radio, etc.), video (e.g., TV, movies, streaming, etc.), phone (e.g., hands-free calling), network connectivity (e.g., LTE, WiFi, etc.), and/or information services (e.g., navigation systems, rear-parking assistance, a radio data system, vehicle related information such as fuel level, total distance covered, brake fuel level, oil level, door open/close, air filter information, etc.) to vehicle **600**. For example, infotainment SoC **630** could include radios, disk players, navigation systems, video players, USB and Bluetooth connectivity, carputers, in-car entertainment, WiFi, steering wheel audio controls, hands free voice control, a heads-up display ("HUD"), HMI display **634**, a telematics device, a control panel (e.g., for controlling and/or interacting with various components, features, and/or systems), and/or other components. In at least one embodiment, infotainment SoC **630** may further be used to provide information (e.g., visual and/or audible) to user(s) of vehicle **600**, such as information from ADAS system **638**, autonomous driving information such as planned vehicle maneuvers, trajectories, surrounding environment information (e.g., intersection information, vehicle information, road information, etc.), and/or other information.

[0154] In at least one embodiment, infotainment SoC **630** may include any amount and type of GPU functionality. In at least one embodiment, infotainment SoC **630** may communicate over bus **602** with other devices, systems, and/or components of vehicle **600**. In at least one embodiment, infotainment SoC **630** may be coupled to a supervisory MCU such that a GPU of an infotainment system may perform some self-driving functions in event that primary controller(s) **636** (e.g., primary and/or backup computers of vehicle **600**) fail. In at least one embodiment, infotainment SoC **630** may put vehicle **600** into a chauffeur to safe stop mode, as described herein.

[0155] In at least one embodiment, vehicle **600** may further include instrument cluster **632** (e.g., a digital dash, an electronic instrument cluster, a digital instrument panel, etc.). In at least one embodiment, instrument cluster **632** may include, without limitation, a controller and/or supercomputer (e.g., a discrete controller or supercomputer). In at least one embodiment, instrument cluster **632** may include, without limitation, any number and combination of a set of instrumentation such as a speedometer, fuel level, oil pressure, tachometer, odometer, turn indicators, gearshift position indicator, seat belt warning light(s), parking-brake warning light(s), engine-malfunction light(s), supplemental restraint system (e.g., airbag) information, lighting controls, safety system controls, navigation information, etc. In some examples, information may be displayed and/or shared among infotainment SoC **630** and instrument cluster **632**. In at least one embodiment, instrument cluster **632** may be included as part of infotainment SoC **630**, or vice versa.

[0156] Various processing devices in FIG. **6C**, e.g., SoCs **604** and GPUs may have inference and/or training logic that is used to perform inferencing and/or training operations associated with one or more embodiments. In at least one embodiment, the inference and/or training logic may be used in system FIG. **6C** for inferencing or predicting operations based, at least in part, on weight parameters calculated using

neural network training operations, neural network functions and/or architectures, or neural network use cases described herein.

[0157] FIG. 6D is a diagram of a system **678** for communication between cloud-based server(s) and autonomous vehicle **600** of FIG. 6A, according to at least one embodiment. In at least one embodiment, system **678** may include, without limitation, server(s) **678**, network(s) **690**, and any number and type of vehicles, including vehicle **600**. In at least one embodiment, server(s) **678** may include, without limitation, a plurality of GPUs **684(A)-684(H)** (collectively referred to herein as GPUs **684**), PCIe switches **682(A)-682(D)** (collectively referred to herein as PCIe switches **682**), and/or CPUs **680(A)-680(B)** (collectively referred to herein as CPUs **680**). In at least one embodiment, GPUs **684**, CPUs **680**, and PCIe switches **682** may be interconnected with high-speed interconnects such as, for example and without limitation, NVLink interfaces **688** developed by NVIDIA and/or PCIe connections **686**. In at least one embodiment, GPUs **684** are connected via an NVLink and/or NVSwitch SoC and GPUs **684** and PCIe switches **682** are connected via PCIe interconnects. Although eight GPUs **684**, two CPUs **680**, and four PCIe switches **682** are illustrated, this is not intended to be limiting. In at least one embodiment, each of server(s) **678** may include, without limitation, any number of GPUs **684**, CPUs **680**, and/or PCIe switches **682**, in any combination. For example, in at least one embodiment, server(s) **678** could each include eight, sixteen, thirty-two, and/or more GPUs **684**.

[0158] In at least one embodiment, server(s) **678** may receive, over network(s) **690** and from vehicles, image data representative of images showing unexpected or changed road conditions, such as recently commenced road-work. In at least one embodiment, server(s) **678** may transmit, over network(s) **690** and to vehicles, neural networks **692**, updated or otherwise, and/or map information **694**, including, without limitation, information regarding traffic and road conditions. In at least one embodiment, updates to map information **694** may include, without limitation, updates for HD map **622**, such as information regarding construction sites, potholes, detours, flooding, and/or other obstructions. In at least one embodiment, neural networks **692**, and/or map information **694** may have resulted from new training and/or experiences represented in data received from any number of vehicles in an environment, and/or based at least in part on training performed at a data center (e.g., using server(s) **678** and/or other servers).

[0159] In at least one embodiment, server(s) **678** may be used to train machine learning models (e.g., neural networks) based at least in part on training data. In at least one embodiment, training data may be generated by vehicles, and/or may be generated in a simulation (e.g., using a game engine). In at least one embodiment, any amount of training data is tagged (e.g., where associated neural network benefits from supervised learning) and/or undergoes other pre-processing. In at least one embodiment, any amount of training data is not tagged and/or pre-processed (e.g., where associated neural network does not require supervised learning). In at least one embodiment, once machine learning models are trained, machine learning models may be used by vehicles (e.g., transmitted to vehicles over network(s) **690**), and/or machine learning models may be used by server(s) **678** to remotely monitor vehicles.

[0160] In at least one embodiment, server(s) **678** may receive data from vehicles and apply data to up-to-date real-time neural networks for real-time intelligent inferencing. In at least one embodiment, server(s) **678** may include deep-learning supercomputers and/or dedicated AI computers powered by GPU(s) **684**, such as a DGX and DGX Station machines developed by NVIDIA. However, in at least one embodiment, server(s) **678** may include deep learning infrastructure that uses CPU-powered data centers.

[0161] In at least one embodiment, deep-learning infrastructure of server(s) **678** may be capable of fast, real-time inferencing, and may use that capability to evaluate and verify health of processors, software, and/or associated hardware in vehicle **600**. For example, in at least one embodiment, deep-learning infrastructure may receive periodic updates from vehicle **600**, such as a sequence of images and/or objects that vehicle **600** has located in that sequence of images (e.g., via computer vision and/or other machine learning object classification techniques). In at least one embodiment, deep-learning infrastructure may run its own neural network to identify objects and compare them with objects identified by vehicle **600** and, if results do not match and deep-learning infrastructure concludes that AI in vehicle **600** is malfunctioning, then server(s) **678** may transmit a signal to vehicle **600** instructing a fail-safe computer of vehicle **600** to assume control, notify passengers, and complete a safe parking maneuver.

[0162] In at least one embodiment, server(s) **678** may include GPU(s) **684** and one or more programmable inference accelerators (e.g., NVIDIA's TensorRT 3 devices). In at least one embodiment, a combination of GPU-powered servers and inference acceleration may make real-time responsiveness possible. In at least one embodiment, such as where performance is less critical, servers powered by CPUs, FPGAs, and other processors may be used for inferencing.

Inference and Training Logic

[0163] FIG. 7A illustrates inference and/or training logic **715** used to perform inferencing and/or training operations associated with one or more embodiments.

[0164] In at least one embodiment, inference and/or training logic **715** may include, without limitation, code and/or data storage **701** to store forward and/or output weight and/or input/output data, and/or other parameters to configure neurons or layers of a neural network trained and/or used for inferencing in aspects of one or more embodiments. In at least one embodiment, training logic **715** may include, or be coupled to code and/or data storage **701** to store graph code or other software to control timing and/or order, in which weight and/or other parameter information is to be loaded to configure, logic, including integer and/or floating point units (collectively, arithmetic logic units (ALUs) or simply circuits). In at least one embodiment, code, such as graph code, loads weight or other parameter information into processor ALUs based on an architecture of a neural network to which such code corresponds. In at least one embodiment, code and/or data storage **701** stores weight parameters and/or input/output data of each layer of a neural network trained or used in conjunction with one or more embodiments during forward propagation of input/output data and/or weight parameters during training and/or inferencing using aspects of one or more embodiments. In at least one embodiment, any portion of code and/or data storage

701 may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory.

[0165] In at least one embodiment, any portion of code and/or data storage **701** may be internal or external to one or more processors or other hardware logic devices or circuits. In at least one embodiment, code and/or code and/or data storage **701** may be cache memory, dynamic randomly addressable memory ("DRAM"), static randomly addressable memory ("SRAM"), non-volatile memory (e.g., flash memory), or other storage. In at least one embodiment, a choice of whether code and/or code and/or data storage **701** is internal or external to a processor, for example, or comprising DRAM, SRAM, flash or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors.

[0166] In at least one embodiment, inference and/or training logic **715** may include, without limitation, a code and/or data storage **705** to store backward and/or output weight and/or input/output data corresponding to neurons or layers of a neural network trained and/or used for inferencing in aspects of one or more embodiments. In at least one embodiment, code and/or data storage **705** stores weight parameters and/or input/output data of each layer of a neural network trained or used in conjunction with one or more embodiments during backward propagation of input/output data and/or weight parameters during training and/or inferencing using aspects of one or more embodiments. In at least one embodiment, training logic **715** may include, or be coupled to code and/or data storage **705** to store graph code or other software to control timing and/or order, in which weight and/or other parameter information is to be loaded to configure, logic, including integer and/or floating point units (collectively, arithmetic logic units (ALUs)).

[0167] In at least one embodiment, code, such as graph code, causes the loading of weight or other parameter information into processor ALUs based on an architecture of a neural network to which such code corresponds. In at least one embodiment, any portion of code and/or data storage **705** may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory. In at least one embodiment, any portion of code and/or data storage **705** may be internal or external to one or more processors or other hardware logic devices or circuits. In at least one embodiment, code and/or data storage **705** may be cache memory, DRAM, SRAM, non-volatile memory (e.g., flash memory), or other storage. In at least one embodiment, a choice of whether code and/or data storage **705** is internal or external to a processor, for example, or comprising DRAM, SRAM, flash memory or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch size of data used in inferencing and/or training of a neural network, or some combination of these factors.

[0168] In at least one embodiment, code and/or data storage **701** and code and/or data storage **705** may be separate storage structures. In at least one embodiment, code and/or data storage **701** and code and/or data storage **705** may be a combined storage structure. In at least one embodiment, code and/or data storage **701** and code and/or data

storage **705** may be partially combined and partially separate. In at least one embodiment, any portion of code and/or data storage **701** and code and/or data storage **705** may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory.

[0169] In at least one embodiment, inference and/or training logic **715** may include, without limitation, one or more arithmetic logic unit(s) ("ALU(s)") **710**, including integer and/or floating point units, to perform logical and/or mathematical operations based, at least in part on, or indicated by, training and/or inference code (e.g., graph code), a result of which may produce activations (e.g., output values from layers or neurons within a neural network) stored in an activation storage **720** that are functions of input/output and/or weight parameter data stored in code and/or data storage **701** and/or code and/or data storage **705**. In at least one embodiment, activations stored in activation storage **720** are generated according to linear algebraic and/or matrix-based mathematics performed by ALU(s) **710** in response to performing instructions or other code, wherein weight values stored in code and/or data storage **705** and/or data storage **701** are used as operands along with other values, such as bias values, gradient information, momentum values, or other parameters or hyperparameters, any or all of which may be stored in code and/or data storage **705** or code and/or data storage **701** or another storage on or off-chip.

[0170] In at least one embodiment, ALU(s) **710** are included within one or more processors or other hardware logic devices or circuits, whereas in another embodiment, ALU(s) **710** may be external to a processor or other hardware logic device or circuit that uses them (e.g., a co-processor). In at least one embodiment, ALU(s) **710** may be included within a processor's execution units or otherwise within a bank of ALUs accessible by a processor's execution units either within same processor or distributed between different processors of different types (e.g., central processing units, graphics processing units, fixed function units, etc.). In at least one embodiment, code and/or data storage **701**, code and/or data storage **705**, and activation storage **720** may share a processor or other hardware logic device or circuit, whereas in another embodiment, they may be in different processors or other hardware logic devices or circuits, or some combination of same and different processors or other hardware logic devices or circuits. In at least one embodiment, any portion of activation storage **720** may be included with other on-chip or off-chip data storage, including a processor's L1, L2, or L3 cache or system memory. Furthermore, inferencing and/or training code may be stored with other code accessible to a processor or other hardware logic or circuit and fetched and/or processed using a processor's fetch, decode, scheduling, execution, retirement and/or other logical circuits.

[0171] In at least one embodiment, activation storage **720** may be cache memory, DRAM, SRAM, non-volatile memory (e.g., flash memory), or other storage. In at least one embodiment, activation storage **720** may be completely or partially within or external to one or more processors or other logical circuits. In at least one embodiment, a choice of whether activation storage **720** is internal or external to a processor, for example, or comprising DRAM, SRAM, flash memory or some other storage type may depend on available storage on-chip versus off-chip, latency requirements of training and/or inferencing functions being performed, batch

size of data used in inferencing and/or training of a neural network, or some combination of these factors.

[0172] In at least one embodiment, inference and/or training logic 715 illustrated in FIG. 7A may be used in conjunction with an application-specific integrated circuit (“ASIC”), such as a TensorFlow® Processing Unit from Google, an inference processing unit (IPU) from Graphcore™, or a Nervana® (e.g., “Lake Crest”) processor from Intel Corp. In at least one embodiment, inference and/or training logic 715 illustrated in FIG. 7A may be used in conjunction with central processing unit (“CPU”) hardware, graphics processing unit (“GPU”) hardware or other hardware, such as field programmable gate arrays (“FPGAs”).

[0173] FIG. 7B illustrates inference and/or training logic 715, according to at least one embodiment. In at least one embodiment, inference and/or training logic 715 may include, without limitation, hardware logic in which computational resources are dedicated or otherwise exclusively used in conjunction with weight values or other information corresponding to one or more layers of neurons within a neural network. In at least one embodiment, inference and/or training logic 715 illustrated in FIG. 7B may be used in conjunction with an application-specific integrated circuit (ASIC), such as TensorFlow® Processing Unit from Google, an inference processing unit (IPU) from Graphcore™, or a Nervana® (e.g., “Lake Crest”) processor from Intel Corp. In at least one embodiment, inference and/or training logic 715 illustrated in FIG. 7B may be used in conjunction with central processing unit (CPU) hardware, graphics processing unit (GPU) hardware or other hardware, such as field programmable gate arrays (FPGAs). In at least one embodiment, inference and/or training logic 715 includes, without limitation, code and/or data storage 701 and code and/or data storage 705, which may be used to store code (e.g., graph code), weight values and/or other information, including bias values, gradient information, momentum values, and/or other parameter or hyperparameter information. In at least one embodiment illustrated in FIG. 7B, each of code and/or data storage 701 and code and/or data storage 705 is associated with a dedicated computational resource, such as computational hardware 702 and computational hardware 706, respectively. In at least one embodiment, each of computational hardware 702 and computational hardware 706 comprises one or more ALUs that perform mathematical functions, such as linear algebraic functions, only on information stored in code and/or data storage 701 and code and/or data storage 705, respectively, result of which is stored in activation storage 720.

[0174] In at least one embodiment, each of code and/or data storage 701 and 705 and corresponding computational hardware 702 and 706, respectively, correspond to different layers of a neural network, such that resulting activation from one storage/computational pair 701/702 of code and/or data storage 701 and computational hardware 702 is provided as an input to a next storage/computational pair 705/706 of code and/or data storage 705 and computational hardware 706, in order to mirror a conceptual organization of a neural network. In at least one embodiment, each of storage/computational pairs 701/702 and 705/706 may correspond to more than one neural network layer. In at least one embodiment, additional storage/computation pairs (not

shown) subsequent to or in parallel with storage/computation pairs 701/702 and 705/706 may be included in inference and/or training logic 715.

Neural Network Training and Deployment

[0175] FIG. 8 illustrates training and deployment of a deep neural network, according to at least one embodiment. In at least one embodiment, untrained neural network 806 is trained using a training dataset 802. In at least one embodiment, training framework 804 is a PyTorch framework, whereas in other embodiments, training framework 804 is a TensorFlow, Boost, Caffe, Microsoft Cognitive Toolkit/CNTK, MXNet, Chainer, Keras, Deeplearning4j, or other training framework. In at least one embodiment, training framework 804 trains an untrained neural network 806 and enables it to be trained using processing resources described herein to generate a trained neural network 808. In at least one embodiment, weights may be chosen randomly or by pre-training using a deep belief network. In at least one embodiment, training may be performed in either a supervised, partially supervised, or unsupervised manner.

[0176] In at least one embodiment, untrained neural network 806 is trained using supervised learning, wherein training dataset 802 includes an input paired with a desired output for an input, or where training dataset 802 includes input having a known output and an output of neural network 806 is manually graded. In at least one embodiment, untrained neural network 806 is trained in a supervised manner and processes inputs from training dataset 802 and compares resulting outputs against a set of expected or desired outputs. In at least one embodiment, errors are then propagated back through untrained neural network 806. In at least one embodiment, training framework 804 adjusts weights that control untrained neural network 806. In at least one embodiment, training framework 804 includes tools to monitor how well untrained neural network 806 is converging towards a model, such as trained neural network 808, suitable to generating correct answers, such as in result 814, based on input data such as a new dataset 812. In at least one embodiment, training framework 804 trains untrained neural network 806 repeatedly while adjusting weights to refine an output of untrained neural network 806 using a loss function and adjustment algorithm, such as stochastic gradient descent. In at least one embodiment, training framework 804 trains untrained neural network 806 until untrained neural network 806 achieves a desired accuracy. In at least one embodiment, trained neural network 808 can then be deployed to implement any number of machine learning operations.

[0177] In at least one embodiment, untrained neural network 806 is trained using unsupervised learning, whereas untrained neural network 806 attempts to train itself using unlabeled data. In at least one embodiment, unsupervised learning training dataset 802 will include input data without any associated output data or “ground truth” data. In at least one embodiment, untrained neural network 806 can learn groupings within training dataset 802 and can determine how individual inputs are related to untrained dataset 802. In at least one embodiment, unsupervised training can be used to generate a self-organizing map in trained neural network 808 capable of performing operations useful in reducing dimensionality of new dataset 812. In at least one embodiment, unsupervised training can also be used to perform

anomaly detection, which allows identification of data points in new dataset **812** that deviate from normal patterns of new dataset **812**.

[0178] In at least one embodiment, semi-supervised learning may be used, which is a technique in which in training dataset **802** includes a mix of labeled and unlabeled data. In at least one embodiment, training framework **804** may be used to perform incremental learning, such as through transferred learning techniques. In at least one embodiment, incremental learning enables trained neural network **808** to adapt to new dataset **812** without forgetting knowledge instilled within trained neural network **808** during initial training.

[0179] With reference to FIG. 9, FIG. 9 is an example data flow diagram for a process **900** of generating and deploying a processing and inferencing pipeline, according to at least one embodiment. In at least one embodiment, process **900** may be deployed to perform game name recognition analysis and inferencing on user feedback data at one or more facilities **902**, such as a data center.

[0180] In at least one embodiment, process **900** may be executed within a training system **904** and/or a deployment system **906**. In at least one embodiment, training system **904** may be used to perform training, deployment, and embodiment of machine learning models (e.g., neural networks, object detection algorithms, computer vision algorithms, etc.) for use in deployment system **906**. In at least one embodiment, deployment system **906** may be configured to offload processing and compute resources among a distributed computing environment to reduce infrastructure requirements at facility **902**. In at least one embodiment, deployment system **906** may provide a streamlined platform for selecting, customizing, and implementing virtual instruments for use with computing devices at facility **902**. In at least one embodiment, virtual instruments may include software-defined applications for performing one or more processing operations with respect to feedback data. In at least one embodiment, one or more applications in a pipeline may use or call upon services (e.g., inference, visualization, compute, AI, etc.) of deployment system **906** during execution of applications.

[0181] In at least one embodiment, some applications used in advanced processing and inferencing pipelines may use machine learning models or other AI to perform one or more processing steps. In at least one embodiment, machine learning models may be trained at facility **902** using feedback data **908** (such as imaging data) stored at facility **902** or feedback data **908** from another facility or facilities, or a combination thereof. In at least one embodiment, training system **904** may be used to provide applications, services, and/or other resources for generating working, deployable machine learning models for deployment system **906**.

[0182] In at least one embodiment, a model registry **924** may be backed by object storage that may support versioning and object metadata. In at least one embodiment, object storage may be accessible through, for example, a cloud storage (e.g., a cloud **1026** of FIG. 10) compatible application programming interface (API) from within a cloud platform. In at least one embodiment, machine learning models within model registry **924** may be uploaded, listed, modified, or deleted by developers or partners of a system interacting with an API. In at least one embodiment, an API may provide access to methods that allow users with appropriate credentials to associate models with applications, such

that models may be executed as part of execution of containerized instantiations of applications.

[0183] In at least one embodiment, a training pipeline **1004** (FIG. 10) may include a scenario where facility **902** is training their own machine learning model, or has an existing machine learning model that needs to be optimized or updated. In at least one embodiment, feedback data **908** may be received from various channels, such as forums, web forms, or the like. In at least one embodiment, once feedback data **908** is received, AI-assisted annotation **910** may be used to aid in generating annotations corresponding to feedback data **908** to be used as ground truth data for a machine learning model. In at least one embodiment, AI-assisted annotation **910** may include one or more machine learning models (e.g., convolutional neural networks (CNNs)) that may be trained to generate annotations corresponding to certain types of feedback data **908** (e.g., from certain devices) and/or certain types of anomalies in feedback data **908**. In at least one embodiment, AI-assisted annotations **910** may then be used directly, or may be adjusted or fine-tuned using an annotation tool, to generate ground truth data. In at least one embodiment, in some examples, labeled data **912** may be used as ground truth data for training a machine learning model. In at least one embodiment, AI-assisted annotations **910**, labeled data **912**, or a combination thereof may be used as ground truth data for training a machine learning model, e.g., via model training **914** in FIGS. 9-10. In at least one embodiment, a trained machine learning model may be referred to as an output model **916**, and may be used by deployment system **906**, as described herein.

[0184] In at least one embodiment, training pipeline **1004** (FIG. 10) may include a scenario where facility **902** needs a machine learning model for use in performing one or more processing tasks for one or more applications in deployment system **906**, but facility **902** may not currently have such a machine learning model (or may not have a model that is optimized, efficient, or effective for such purposes). In at least one embodiment, an existing machine learning model may be selected from model registry **924**. In at least one embodiment, model registry **924** may include machine learning models trained to perform a variety of different inference tasks on imaging data. In at least one embodiment, machine learning models in model registry **924** may have been trained on imaging data from different facilities than facility **902** (e.g., facilities that are remotely located). In at least one embodiment, machine learning models may have been trained on imaging data from one location, two locations, or any number of locations. In at least one embodiment, when being trained on imaging data, which may be a form of feedback data **908**, from a specific location, training may take place at that location, or at least in a manner that protects confidentiality of imaging data or restricts imaging data from being transferred off-premises (e.g., to comply with HIPAA regulations, privacy regulations, etc.). In at least one embodiment, once a model is trained—or partially trained—at one location, a machine learning model may be added to model registry **924**. In at least one embodiment, a machine learning model may then be retrained, or updated, at any number of other facilities, and a retrained or updated model may be made available in model registry **924**. In at least one embodiment, a machine learning model may then be selected from model registry **924**—and referred to as output model **916**—and may be used in deployment system

906 to perform one or more processing tasks for one or more applications of a deployment system.

[0185] In at least one embodiment, training pipeline **1004** (FIG. **10**) may be used in a scenario that includes facility **902** requiring a machine learning model for use in performing one or more processing tasks for one or more applications in deployment system **906**, but facility **902** may not currently have such a machine learning model (or may not have a model that is optimized, efficient, or effective for such purposes). In at least one embodiment, a machine learning model selected from model registry **924** might not be fine-tuned or optimized for feedback data **908** generated at facility **902** because of differences in populations, genetic variations, robustness of training data used to train a machine learning model, diversity in anomalies of training data, and/or other issues with training data. In at least one embodiment, AI-assisted annotation **910** may be used to aid in generating annotations corresponding to feedback data **908** to be used as ground truth data for retraining or updating a machine learning model. In at least one embodiment, labeled data **912** may be used as ground truth data for training a machine learning model. In at least one embodiment, retraining or updating a machine learning model may be referred to as model training **914**. In at least one embodiment, model training **914**—e.g., AI-assisted annotations **910**, labeled data **912**, or a combination thereof—may be used as ground truth data for retraining or updating a machine learning model.

[0186] In at least one embodiment, deployment system **906** may include software **918**, services **920**, hardware **922**, and/or other components, features, and functionality. In at least one embodiment, deployment system **906** may include a software “stack,” such that software **918** may be built on top of services **920** and may use services **920** to perform some or all of processing tasks, and services **920** and software **918** may be built on top of hardware **922** and use hardware **922** to execute processing, storage, and/or other compute tasks of deployment system **906**.

[0187] In at least one embodiment, software **918** may include any number of different containers, where each container may execute an instantiation of an application. In at least one embodiment, each application may perform one or more processing tasks in an advanced processing and inferencing pipeline (e.g., inferencing, object detection, feature detection, segmentation, image enhancement, calibration, etc.). In at least one embodiment, for each type of computing device there may be any number of containers that may perform a data processing task with respect to feedback data **908** (or other data types, such as those described herein). In at least one embodiment, an advanced processing and inferencing pipeline may be defined based on selections of different containers that are desired or required for processing feedback data **908**, in addition to containers that receive and configure imaging data for use by each container and/or for use by facility **902** after processing through a pipeline (e.g., to convert outputs back to a usable data type for storage and display at facility **902**). In at least one embodiment, a combination of containers within software **918** (e.g., that make up a pipeline) may be referred to as a virtual instrument (as described in more detail herein), and a virtual instrument may leverage services **920** and hardware **922** to execute some or all processing tasks of applications instantiated in containers.

[0188] In at least one embodiment, data may undergo pre-processing as part of data processing pipeline to prepare data for processing by one or more applications. In at least one embodiment, post-processing may be performed on an output of one or more inferencing tasks or other processing tasks of a pipeline to prepare an output data for a next application and/or to prepare output data for transmission and/or use by a user (e.g., as a response to an inference request). In at least one embodiment, inferencing tasks may be performed by one or more machine learning models, such as trained or deployed neural networks, which may include output models **916** of training system **904**.

[0189] In at least one embodiment, tasks of data processing pipeline may be encapsulated in one or more container(s) that each represent a discrete, fully functional instantiation of an application and virtualized computing environment that is able to reference machine learning models. In at least one embodiment, containers or applications may be published into a private (e.g., limited access) area of a container registry (described in more detail herein), and trained or deployed models may be stored in model registry **924** and associated with one or more applications. In at least one embodiment, images of applications (e.g., container images) may be available in a container registry, and once selected by a user from a container registry for deployment in a pipeline, an image may be used to generate a container for an instantiation of an application for use by a user system.

[0190] In at least one embodiment, developers may develop, publish, and store applications (e.g., as containers) for performing processing and/or inferencing on supplied data. In at least one embodiment, development, publishing, and/or storing may be performed using a software development kit (SDK) associated with a system (e.g., to ensure that an application and/or container developed is compliant with or compatible with a system). In at least one embodiment, an application that is developed may be tested locally (e.g., at a first facility, on data from a first facility) with an SDK which may support at least some of services **920** as a system (e.g., system **1000** of FIG. **10**). In at least one embodiment, once validated by system **1000** (e.g., for accuracy, etc.), an application may be available in a container registry for selection and/or embodiment by a user (e.g., a hospital, clinic, lab, healthcare provider, etc.) to perform one or more processing tasks with respect to data at a facility (e.g., a second facility) of a user.

[0191] In at least one embodiment, developers may then share applications or containers through a network for access and use by users of a system (e.g., system **1000** of FIG. **10**). In at least one embodiment, completed and validated applications or containers may be stored in a container registry and associated machine learning models may be stored in model registry **924**. In at least one embodiment, a requesting entity that provides an inference or image processing request may browse a container registry and/or model registry **924** for an application, container, dataset, machine learning model, etc., select a desired combination of elements for inclusion in data processing pipeline, and submit a processing request. In at least one embodiment, a request may include input data that is necessary to perform a request, and/or may include a selection of application(s) and/or machine learning models to be executed in processing a request. In at least one embodiment, a request may then be passed to one or more components of deployment system

906 (e.g., a cloud) to perform processing of a data processing pipeline. In at least one embodiment, processing by deployment system **906** may include referencing selected elements (e.g., applications, containers, models, etc.) from a container registry and/or model registry **924**. In at least one embodiment, once results are generated by a pipeline, results may be returned to a user for reference (e.g., for viewing in a viewing application suite executing on a local, on-premises workstation or terminal).

[0192] In at least one embodiment, to aid in processing or execution of applications or containers in pipelines, services **920** may be leveraged. In at least one embodiment, services **920** may include compute services, collaborative content creation services, simulation services, artificial intelligence (AI) services, visualization services, and/or other service types. In at least one embodiment, services **920** may provide functionality that is common to one or more applications in software **918**, so functionality may be abstracted to a service that may be called upon or leveraged by applications. In at least one embodiment, functionality provided by services **920** may run dynamically and more efficiently, while also scaling well by allowing applications to process data in parallel, e.g., using a parallel computing platform **1030** (FIG. 10). In at least one embodiment, rather than each application that shares a same functionality offered by a service **920** being required to have a respective instance of service **920**, service **920** may be shared between and among various applications. In at least one embodiment, services may include an inference server or engine that may be used for executing detection or segmentation tasks, as non-limiting examples. In at least one embodiment, a model training service may be included that may provide machine learning model training and/or retraining capabilities.

[0193] In at least one embodiment, where a service **920** includes an AI service (e.g., an inference service), one or more machine learning models associated with an application for anomaly detection (e.g., tumors, growth abnormalities, scarring, etc.) may be executed by calling upon (e.g., as an API call) an inference service (e.g., an inference server) to execute machine learning model(s), or processing thereof, as part of application execution. In at least one embodiment, where another application includes one or more machine learning models for segmentation tasks, an application may call upon an inference service to execute machine learning models for performing one or more of processing operations associated with segmentation tasks. In at least one embodiment, software **918** implementing advanced processing and inferencing pipeline may be streamlined because each application may call upon the same inference service to perform one or more inferencing tasks.

[0194] In at least one embodiment, hardware **922** may include GPUs, CPUs, graphics cards, an AI/deep learning system (e.g., an AI supercomputer, such as NVIDIA's DGX™ supercomputer system), a cloud platform, or a combination thereof. In at least one embodiment, different types of hardware **922** may be used to provide efficient, purpose-built support for software **918** and services **920** in deployment system **906**. In at least one embodiment, use of GPU processing may be implemented for processing locally (e.g., at facility **902**), within an AI/deep learning system, in a cloud system, and/or in other processing components of deployment system **906** to improve efficiency, accuracy, and efficacy of game name recognition.

[0195] In at least one embodiment, software **918** and/or services **920** may be optimized for GPU processing with respect to deep learning, machine learning, and/or high-performance computing, simulation, and visual computing, as non-limiting examples. In at least one embodiment, at least some of the computing environment of deployment system **906** and/or training system **904** may be executed in a datacenter or one or more supercomputers or high performance computing systems, with GPU-optimized software (e.g., hardware and software combination of NVIDIA's DGX™ system). In at least one embodiment, hardware **922** may include any number of GPUs that may be called upon to perform processing of data in parallel, as described herein. In at least one embodiment, cloud platform may further include GPU processing for GPU-optimized execution of deep learning tasks, machine learning tasks, or other computing tasks. In at least one embodiment, cloud platform (e.g., NVIDIA's NGC™) may be executed using an AI/deep learning supercomputer(s) and/or GPU-optimized software (e.g., as provided on NVIDIA's DGX™ systems) as a hardware abstraction and scaling platform. In at least one embodiment, cloud platform may integrate an application container clustering system or orchestration system (e.g., KUBERNETES) on multiple GPUs to enable seamless scaling and load balancing.

[0196] FIG. 10 is a system diagram for an example system **1000** for generating and deploying a deployment pipeline, according to at least one embodiment. In at least one embodiment, system **1000** may be used to implement process **900** of FIG. 9 and/or other processes including advanced processing and inferencing pipelines. In at least one embodiment, system **1000** may include training system **904** and deployment system **906**. In at least one embodiment, training system **904** and deployment system **906** may be implemented using software **918**, services **920**, and/or hardware **922**, as described herein.

[0197] In at least one embodiment, system **1000** (e.g., training system **904** and/or deployment system **906**) may be implemented in a cloud computing environment (e.g., using cloud **1026**). In at least one embodiment, system **1000** may be implemented locally with respect to a facility, or as a combination of both cloud and local computing resources. In at least one embodiment, access to APIs in cloud **1026** may be restricted to authorized users through enacted security measures or protocols. In at least one embodiment, a security protocol may include web tokens that may be signed by an authentication (e.g., AuthN, AuthZ, Gluecon, etc.) service and may carry appropriate authorization. In at least one embodiment, APIs of virtual instruments (described herein), or other instantiations of system **1000**, may be restricted to a set of public internet service providers (ISPs) that have been vetted or authorized for interaction.

[0198] In at least one embodiment, various components of system **1000** may communicate between and among one another using any of a variety of different network types, including but not limited to local area networks (LANs) and/or wide area networks (WANs) via wired and/or wireless communication protocols. In at least one embodiment, communication between facilities and components of system **1000** (e.g., for transmitting inference requests, for receiving results of inference requests, etc.) may be communicated over a data bus or data busses, wireless data protocols (Wi-Fi), wired data protocols (e.g., Ethernet), etc.

[0199] In at least one embodiment, training system 904 may execute training pipelines 1004, similar to those described herein with respect to FIG. 9. In at least one embodiment, where one or more machine learning models are to be used in deployment pipelines 1010 by deployment system 906, training pipelines 1004 may be used to train or retrain one or more (e.g., pre-trained) models, and/or implement one or more of pre-trained models 1006 (e.g., without a need for retraining or updating). In at least one embodiment, as a result of training pipelines 1004, output model(s) 916 may be generated. In at least one embodiment, training pipelines 1004 may include any number of processing steps, AI-assisted annotation 910, labeling or annotating of feedback data 908 to generate labeled data 912, model selection from a model registry, model training 914, training, retraining, or updating models, and/or other processing steps. In at least one embodiment, for different machine learning models used by deployment system 906, different training pipelines 1004 may be used. In at least one embodiment, training pipeline 1004, similar to a first example described with respect to FIG. 9, may be used for a first machine learning model, training pipeline 1004, similar to a second example described with respect to FIG. 9, may be used for a second machine learning model, and training pipeline 1004, similar to a third example described with respect to FIG. 9, may be used for a third machine learning model. In at least one embodiment, any combination of tasks within training system 904 may be used depending on what is required for each respective machine learning model. In at least one embodiment, one or more of machine learning models may already be trained and ready for deployment so machine learning models may not undergo any processing by training system 904, and may be implemented by deployment system 906.

[0200] In at least one embodiment, output model(s) 916 and/or pre-trained model(s) 1006 may include any types of machine learning models depending on embodiment. In at least one embodiment, and without limitation, machine learning models used by system 1000 may include machine learning model(s) using linear regression, logistic regression, decision trees, support vector machines (SVM), Naïve Bayes, k-nearest neighbor (Knn), K means clustering, random forest, dimensionality reduction algorithms, gradient boosting algorithms, neural networks (e.g., auto-encoders, convolutional, recurrent, perceptrons, Long/Short Term Memory (LSTM), Bi-LSTM, Hopfield, Boltzmann, deep belief, deconvolutional, generative adversarial, liquid state machine, etc.), and/or other types of machine learning models.

[0201] In at least one embodiment, training pipelines 1004 may include AI-assisted annotation. In at least one embodiment, labeled data 912 (e.g., traditional annotation) may be generated by any number of techniques. In at least one embodiment, labels or other annotations may be generated within a drawing program (e.g., an annotation program), a computer aided design (CAD) program, a labeling program, another type of program suitable for generating annotations or labels for ground truth, and/or may be hand drawn, in some examples. In at least one embodiment, ground truth data may be synthetically produced (e.g., generated from computer models or renderings), real produced (e.g., designed and produced from real-world data), machine-automated (e.g., using feature analysis and learning to extract features from data and then generate labels), human annotated (e.g., labeler, or annotation expert, defines loca-

tion of labels), and/or a combination thereof. In at least one embodiment, for each instance of feedback data 908 (or other data type used by machine learning models), there may be corresponding ground truth data generated by training system 904. In at least one embodiment, AI-assisted annotation may be performed as part of deployment pipelines 1010; either in addition to, or in lieu of, AI-assisted annotation included in training pipelines 1004. In at least one embodiment, system 1000 may include a multi-layer platform that may include a software layer (e.g., software 918) of diagnostic applications (or other application types) that may perform one or more medical imaging and diagnostic functions.

[0202] In at least one embodiment, a software layer may be implemented as a secure, encrypted, and/or authenticated API through which applications or containers may be invoked (e.g., called) from an external environment(s), e.g., facility 902. In at least one embodiment, applications may then call or execute one or more services 920 for performing compute, AI, or visualization tasks associated with respective applications, and software 918 and/or services 920 may leverage hardware 922 to perform processing tasks in an effective and efficient manner.

[0203] In at least one embodiment, deployment system 906 may execute deployment pipelines 1010. In at least one embodiment, deployment pipelines 1010 may include any number of applications that may be sequentially, non-sequentially, or otherwise applied to feedback data (and/or other data types), including AI-assisted annotation, as described above. In at least one embodiment, as described herein, a deployment pipeline 1010 for an individual device may be referred to as a virtual instrument for a device. In at least one embodiment, for a single device, there may be more than one deployment pipeline 1010 depending on information desired from data generated by a device.

[0204] In at least one embodiment, applications available for deployment pipelines 1010 may include any application that may be used for performing processing tasks on feedback data or other data from devices. In at least one embodiment, because various applications may share common image operations, in some embodiments, a data augmentation library (e.g., as one of services 920) may be used to accelerate these operations. In at least one embodiment, to avoid bottlenecks of conventional processing approaches that rely on CPU processing, parallel computing platform 1030 may be used for GPU acceleration of these processing tasks.

[0205] In at least one embodiment, deployment system 906 may include a user interface (UI) 1014 (e.g., a graphical user interface, a web interface, etc.) that may be used to select applications for inclusion in deployment pipeline(s) 1010, arrange applications, modify or change applications or parameters or constructs thereof, use and interact with deployment pipeline(s) 1010 during set-up and/or deployment, and/or to otherwise interact with deployment system 906. In at least one embodiment, although not illustrated with respect to training system 904, UI 1014 (or a different user interface) may be used for selecting models for use in deployment system 906, for selecting models for training, or retraining, in training system 904, and/or for otherwise interacting with training system 904.

[0206] In at least one embodiment, pipeline manager 1012 may be used, in addition to an application orchestration system 1028, to manage interaction between applications or

containers of deployment pipeline(s) **1010** and services **920** and/or hardware **922**. In at least one embodiment, pipeline manager **1012** may be configured to facilitate interactions from application to application, from application to service **920**, and/or from application or service to hardware **922**. In at least one embodiment, although illustrated as included in software **918**, this is not intended to be limiting, and in some examples pipeline manager **1012** may be included in services **920**. In at least one embodiment, application orchestration system **1028** (e.g., Kubernetes, DOCKER, etc.) may include a container orchestration system that may group applications into containers as logical units for coordination, management, scaling, and deployment. In at least one embodiment, by associating applications from deployment pipeline(s) **1010** (e.g., a reconstruction application, a segmentation application, etc.) with individual containers, each application may execute in a self-contained environment (e.g., at a kernel level) to increase speed and efficiency.

[0207] In at least one embodiment, each application and/or container (or image thereof) may be individually developed, modified, and deployed (e.g., a first user or developer may develop, modify, and deploy a first application and a second user or developer may develop, modify, and deploy a second application separate from a first user or developer), which may allow for focus on, and attention to, a task of a single application and/or container(s) without being hindered by tasks of other application(s) or container(s). In at least one embodiment, communication, and cooperation between different containers or applications may be aided by pipeline manager **1012** and application orchestration system **1028**. In at least one embodiment, so long as an expected input and/or output of each container or application is known by a system (e.g., based on constructs of applications or containers), application orchestration system **1028** and/or pipeline manager **1012** may facilitate communication among and between, and sharing of resources among and between, each of applications or containers. In at least one embodiment, because one or more of applications or containers in deployment pipeline(s) **1010** may share the same services and resources, application orchestration system **1028** may orchestrate, load balance, and determine sharing of services or resources between and among various applications or containers. In at least one embodiment, a scheduler may be used to track resource requirements of applications or containers, current usage or planned usage of these resources, and resource availability. In at least one embodiment, the scheduler may thus allocate resources to different applications and distribute resources between and among applications in view of requirements and availability of a system. In some examples, the scheduler (and/or other component of application orchestration system **1028**) may determine resource availability and distribution based on constraints imposed on a system (e.g., user constraints), such as quality of service (QoS), urgency of need for data outputs (e.g., to determine whether to execute real-time processing or delayed processing), etc.

[0208] In at least one embodiment, services **920** leveraged and shared by applications or containers in deployment system **906** may include compute services **1016**, collaborative content creation services **1017**, AI services **1018**, simulation services **1019**, visualization services **1020**, and/or other service types. In at least one embodiment, applications may call (e.g., execute) one or more of services **920** to perform processing operations for an application. In at least

one embodiment, compute services **1016** may be leveraged by applications to perform super-computing or other high-performance computing (HPC) tasks. In at least one embodiment, compute service(s) **1016** may be leveraged to perform parallel processing (e.g., using a parallel computing platform **1030**) for processing data through one or more of applications and/or one or more tasks of a single application, substantially simultaneously. In at least one embodiment, parallel computing platform **1030** (e.g., NVIDIA's CUDA®) may enable general purpose computing on GPUs (GPGPU) (e.g., GPUs **1022**). In at least one embodiment, a software layer of parallel computing platform **1030** may provide access to virtual instruction sets and parallel computational elements of GPUs, for execution of compute kernels. In at least one embodiment, parallel computing platform **1030** may include memory and, in some embodiments, a memory may be shared between and among multiple containers, and/or between and among different processing tasks within a single container. In at least one embodiment, inter-process communication (IPC) calls may be generated for multiple containers and/or for multiple processes within a container to use same data from a shared segment of memory of parallel computing platform **1030** (e.g., where multiple different stages of an application or multiple applications are processing same information). In at least one embodiment, rather than making a copy of data and moving data to different locations in memory (e.g., a read/write operation), same data in the same location of a memory may be used for any number of processing tasks (e.g., at the same time, at different times, etc.). In at least one embodiment, as data is used to generate new data as a result of processing, this information of a new location of data may be stored and shared between various applications. In at least one embodiment, location of data and a location of updated or modified data may be part of a definition of how a payload is understood within containers.

[0209] In at least one embodiment, AI services **1018** may be leveraged to perform inferencing services for executing machine learning model(s) associated with applications (e.g., tasked with performing one or more processing tasks of an application). In at least one embodiment, AI services **1018** may leverage AI system **1024** to execute machine learning model(s) (e.g., neural networks, such as CNNs) for segmentation, reconstruction, object detection, feature detection, classification, and/or other inferencing tasks. In at least one embodiment, applications of deployment pipeline(s) **1010** may use one or more of output models **916** from training system **904** and/or other models of applications to perform inference on imaging data (e.g., DICOM data, RIS data, CIS data, REST compliant data, RPC data, raw data, etc.). In at least one embodiment, two or more examples of inferencing using application orchestration system **1028** (e.g., a scheduler) may be available. In at least one embodiment, a first category may include a high priority/low latency path that may achieve higher service level agreements, such as for performing inference on urgent requests during an emergency, or for a radiologist during diagnosis. In at least one embodiment, a second category may include a standard priority path that may be used for requests that may be non-urgent or where analysis may be performed at a later time. In at least one embodiment, application orchestration system **1028** may distribute resources (e.g., services **920** and/or hardware **922**) based on priority paths for different inferencing tasks of AI services **1018**.

[0210] In at least one embodiment, shared storage may be mounted to AI services **1018** within system **1000**. In at least one embodiment, shared storage may operate as a cache (or other storage device type) and may be used to process inference requests from applications. In at least one embodiment, when an inference request is submitted, a request may be received by a set of API instances of deployment system **906**, and one or more instances may be selected (e.g., for best fit, for load balancing, etc.) to process a request. In at least one embodiment, to process a request, a request may be entered into a database, a machine learning model may be located from model registry **924** if not already in a cache, a validation step may ensure appropriate machine learning model is loaded into a cache (e.g., shared storage), and/or a copy of a model may be saved to a cache. In at least one embodiment, the scheduler (e.g., of pipeline manager **1012**) may be used to launch an application that is referenced in a request if an application is not already running or if there are not enough instances of an application. In at least one embodiment, if an inference server is not already launched to execute a model, an inference server may be launched. In at least one embodiment, any number of inference servers may be launched per model. In at least one embodiment, in a pull model, in which inference servers are clustered, models may be cached whenever load balancing is advantageous. In at least one embodiment, inference servers may be statically loaded in corresponding, distributed servers.

[0211] In at least one embodiment, inferencing may be performed using an inference server that runs in a container. In at least one embodiment, an instance of an inference server may be associated with a model (and optionally a plurality of versions of a model). In at least one embodiment, if an instance of an inference server does not exist when a request to perform inference on a model is received, a new instance may be loaded. In at least one embodiment, when starting an inference server, a model may be passed to an inference server such that a same container may be used to serve different models so long as the inference server is running as a different instance.

[0212] In at least one embodiment, during application execution, an inference request for a given application may be received, and a container (e.g., hosting an instance of an inference server) may be loaded (if not already loaded), and a start procedure may be called. In at least one embodiment, pre-processing logic in a container may load, decode, and/or perform any additional pre-processing on incoming data (e.g., using a CPU(s) and/or GPU(s)). In at least one embodiment, once data is prepared for inference, a container may perform inference as necessary on data. In at least one embodiment, this may include a single inference call on one image (e.g., a hand X-ray), or may require inference on hundreds of images (e.g., a chest CT). In at least one embodiment, an application may summarize results before completing, which may include, without limitation, a single confidence score, pixel level-segmentation, voxel-level segmentation, generating a visualization, or generating text to summarize findings. In at least one embodiment, different models or applications may be assigned different priorities. For example, some models may have a real-time (turn-around time less than one minute) priority while others may have lower priority (e.g., turnaround less than 10 minutes). In at least one embodiment, model execution times may be

measured from requesting institution or entity and may include partner network traversal time, as well as execution on an inference service.

[0213] In at least one embodiment, transfer of requests between services **920** and inference applications may be hidden behind a software development kit (SDK), and robust transport may be provided through a queue. In at least one embodiment, a request is placed in a queue via an API for an individual application/tenant ID combination and an SDK pulls a request from a queue and gives a request to an application. In at least one embodiment, a name of a queue may be provided in an environment from where an SDK picks up the request. In at least one embodiment, asynchronous communication through a queue may be useful as it may allow any instance of an application to pick up work as it becomes available. In at least one embodiment, results may be transferred back through a queue, to ensure no data is lost. In at least one embodiment, queues may also provide an ability to segment work, as highest priority work may go to a queue with most instances of an application connected to it, while lowest priority work may go to a queue with a single instance connected to it that processes tasks in an order received. In at least one embodiment, an application may run on a GPU-accelerated instance generated in cloud **1026**, and an inference service may perform inferencing on a GPU.

[0214] In at least one embodiment, visualization services **1020** may be leveraged to generate visualizations for viewing outputs of applications and/or deployment pipeline(s) **1010**. In at least one embodiment, GPUs **1022** may be leveraged by visualization services **1020** to generate visualizations. In at least one embodiment, rendering effects, such as ray-tracing or other light transport simulation techniques, may be implemented by visualization services **1020** to generate higher quality visualizations. In at least one embodiment, visualizations may include, without limitation, 2D image renderings, 3D volume renderings, 3D volume reconstruction, 2D tomographic slices, virtual reality displays, augmented reality displays, etc. In at least one embodiment, virtualized environments may be used to generate a virtual interactive display or environment (e.g., a virtual environment) for interaction by users of a system (e.g., doctors, nurses, radiologists, etc.). In at least one embodiment, visualization services **1020** may include an internal visualizer, cinematics, and/or other rendering or image processing capabilities or functionality (e.g., ray tracing, rasterization, internal optics, etc.).

[0215] In at least one embodiment, hardware **922** may include GPUs **1022**, AI system **1024**, cloud **1026**, and/or any other hardware used for executing training system **904** and/or deployment system **906**. In at least one embodiment, GPUs **1022** (e.g., NVIDIA's TESLA® and/or QUADRO® GPUs) may include any number of GPUs that may be used for executing processing tasks of compute services **1016**, collaborative content creation services **1017**, AI services **1018**, simulation services **1019**, visualization services **1020**, other services, and/or any of features or functionality of software **918**. For example, with respect to AI services **1018**, GPUs **1022** may be used to perform pre-processing on imaging data (or other data types used by machine learning models), post-processing on outputs of machine learning models, and/or to perform inferencing (e.g., to execute machine learning models). In at least one embodiment, cloud **1026**, AI system **1024**, and/or other components of

system **1000** may use GPUs **1022**. In at least one embodiment, cloud **1026** may include a GPU-optimized platform for deep learning tasks. In at least one embodiment, AI system **1024** may use GPUs, and cloud **1026**—or at least a portion tasked with deep learning or inferencing—may be executed using one or more AI systems **1024**. As such, although hardware **922** is illustrated as discrete components, this is not intended to be limiting, and any components of hardware **922** may be combined with, or leveraged by, any other components of hardware **922**.

[0216] In at least one embodiment, AI system **1024** may include a purpose-built computing system (e.g., a supercomputer or an HPC) configured for inferencing, deep learning, machine learning, and/or other artificial intelligence tasks. In at least one embodiment, AI system **1024** (e.g., NVIDIA's DGX™) may include GPU-optimized software (e.g., a software stack) that may be executed using a plurality of GPUs **1022**, in addition to CPUs, RAM, storage, and/or other components, features, or functionality. In at least one embodiment, one or more AI systems **1024** may be implemented in cloud **1026** (e.g., in a data center) for performing some or all of AI-based processing tasks of system **1000**.

[0217] In at least one embodiment, cloud **1026** may include a GPU-accelerated infrastructure (e.g., NVIDIA's NGC™) that may provide a GPU-optimized platform for executing processing tasks of system **1000**. In at least one embodiment, cloud **1026** may include an AI system(s) **1024** for performing one or more of AI-based tasks of system **1000** (e.g., as a hardware abstraction and scaling platform). In at least one embodiment, cloud **1026** may integrate with application orchestration system **1028** leveraging multiple GPUs to enable seamless scaling and load balancing between and among applications and services **920**. In at least one embodiment, cloud **1026** may be tasked with executing at least some of services **920** of system **1000**, including compute services **1016**, AI services **1018**, and/or visualization services **1020**, as described herein. In at least one embodiment, cloud **1026** may perform small and large batch inference (e.g., executing NVIDIA's TensorRT™), provide an accelerated parallel computing API and platform **1030** (e.g., NVIDIA's CUDA®), execute application orchestration system **1028** (e.g., KUBERNETES), provide a graphics rendering API and platform (e.g., for ray-tracing, 2D graphics, 3D graphics, and/or other rendering techniques to produce higher quality cinematics), and/or may provide other functionality for system **1000**.

[0218] In at least one embodiment, in an effort to preserve patient confidentiality (e.g., where patient data or records are to be used off-premises), cloud **1026** may include a registry, such as a deep learning container registry. In at least one embodiment, a registry may store containers for instantiations of applications that may perform pre-processing, post-processing, or other processing tasks on patient data. In at least one embodiment, cloud **1026** may receive data that includes patient data as well as sensor data in containers, perform requested processing for just sensor data in those containers, and then forward a resultant output and/or visualizations to appropriate parties and/or devices (e.g., on-premises medical devices used for visualization or diagnoses), all without having to extract, store, or otherwise access patient data. In at least one embodiment, confidentiality of patient data is preserved in compliance with HIPAA and/or other data regulations.

[0219] Some portions of the detailed descriptions above are presented in terms of algorithms and symbolic representations of operations on data bits within a computer memory. These algorithmic descriptions and representations are the means used by those skilled in the data processing arts to most effectively convey the substance of their work to others skilled in the art. An algorithm is here, and generally, conceived to be a self-consistent sequence of steps leading to a desired result. The steps are those requiring physical manipulations of physical quantities. Usually, though not necessarily, these quantities take the form of electrical or magnetic signals capable of being stored, transferred, combined, compared, and otherwise manipulated. It has proven convenient at times, principally for reasons of common usage, to refer to these signals as bits, values, elements, symbols, characters, terms, numbers, or the like.

[0220] It should be borne in mind, however, that all of these and similar terms are to be associated with the appropriate physical quantities and are merely convenient labels applied to these quantities. Unless specifically stated otherwise, as apparent from the following discussion, it is appreciated that throughout the description, discussions utilizing terms such as “identifying,” “determining,” “storing,” “adjusting,” “causing,” “returning,” “comparing,” “creating,” “stopping,” “loading,” “copying,” “throwing,” “replacing,” “performing,” or the like, refer to the action and processes of a computer system, or similar electronic computing device, that manipulates and transforms data represented as physical (electronic) quantities within the computer system's registers and memories into other data similarly represented as physical quantities within the computer system memories or registers or other such information storage, transmission or display devices.

[0221] Examples of the present disclosure also relate to an apparatus for performing the methods described herein. This apparatus can be specially constructed for the required purposes, or it can be a general purpose computer system selectively programmed by a computer program stored in the computer system. Such a computer program can be stored in a computer readable storage medium, such as, but not limited to, any type of disk including optical disks, CD-ROMs, and magnetic-optical disks, read-only memories (ROMs), random access memories (RAMs), EPROMs, EEPROMs, magnetic disk storage media, optical storage media, flash memory devices, other type of machine-accessible storage media, or any type of media suitable for storing electronic instructions, each coupled to a computer system bus.

[0222] The methods and displays presented herein are not inherently related to any particular computer or other apparatus. Various general purpose systems can be used with programs in accordance with the teachings herein, or it may prove convenient to construct a more specialized apparatus to perform the required method steps. The required structure for a variety of these systems will appear as set forth in the description below. In addition, the scope of the present disclosure is not limited to any particular programming language. It will be appreciated that a variety of programming languages can be used to implement the teachings of the present disclosure.

[0223] It is to be understood that the above description is intended to be illustrative, and not restrictive. Many other implementation examples will be apparent to those of skill in the art upon reading and understanding the above descrip-

tion. Although the present disclosure describes specific examples, it will be recognized that the systems and methods of the present disclosure are not limited to the examples described herein, but can be practiced with modifications within the scope of the appended claims. Accordingly, the specification and drawings are to be regarded in an illustrative sense rather than a restrictive sense. The scope of the present disclosure should, therefore, be determined with reference to the appended claims, along with the full scope of equivalents to which such claims are entitled.

[0224] Other variations are within the spirit of present disclosure. Thus, while disclosed techniques are susceptible to various modifications and alternative constructions, certain illustrated embodiments thereof are shown in drawings and have been described above in detail. It should be understood, however, that there is no intention to limit disclosure to specific form or forms disclosed, but on contrary, intention is to cover all modifications, alternative constructions, and equivalents falling within spirit and scope of disclosure, as defined in appended claims.

[0225] Use of terms “a” and “an” and “the” and similar referents in context of describing disclosed embodiments (especially in context of following claims) are to be construed to cover both singular and plural, unless otherwise indicated herein or clearly contradicted by context, and not as a definition of a term. Terms “comprising,” “having,” “including,” and “containing” are to be construed as open-ended terms (meaning “including, but not limited to,”) unless otherwise noted. “Connected,” when unmodified and referring to physical connections, is to be construed as partly or wholly contained within, attached to, or joined together, even if there is something intervening. Recitation of ranges of values herein are merely intended to serve as a shorthand method of referring individually to each separate value falling within range, unless otherwise indicated herein and each separate value is incorporated into specification as if it were individually recited herein. In at least one embodiment, use of term “set” (e.g., “a set of items”) or “subset” unless otherwise noted or contradicted by context, is to be construed as a nonempty collection comprising one or more members. Further, unless otherwise noted or contradicted by context, term “subset” of a corresponding set does not necessarily denote a proper subset of corresponding set, but subset and corresponding set may be equal.

[0226] Conjunctive language, such as phrases of form “at least one of A, B, and C,” or “at least one of A, B and C,” unless specifically stated otherwise or otherwise clearly contradicted by context, is otherwise understood with context as used in general to present that an item, term, etc., may be either A or B or C, or any nonempty subset of set of A and B and C. For instance, in illustrative example of a set having three members, conjunctive phrases “at least one of A, B, and C” and “at least one of A, B and C” refer to any of following sets: {A}, {B}, {C}, {A, B}, {A, C}, {B, C}, {A, B, C}. Thus, such conjunctive language is not generally intended to imply that certain embodiments require at least one of A, at least one of B and at least one of C each to be present. In addition, unless otherwise noted or contradicted by context, term “plurality” indicates a state of being plural (e.g., “a plurality of items” indicates multiple items). In at least one embodiment, number of items in a plurality is at least two, but can be more when so indicated either explicitly or by context. Further, unless stated otherwise or oth-

erwise clear from context, phrase “based on” means “based at least in part on” and not “based solely on.”

[0227] Operations of processes described herein can be performed in any suitable order unless otherwise indicated herein or otherwise clearly contradicted by context. In at least one embodiment, a process such as those processes described herein (or variations and/or combinations thereof) is performed under control of one or more computer systems configured with executable instructions and is implemented as code (e.g., executable instructions, one or more computer programs or one or more applications) executing collectively on one or more processors, by hardware or combinations thereof. In at least one embodiment, code is stored on a computer-readable storage medium, for example, in form of a computer program comprising a plurality of instructions executable by one or more processors. In at least one embodiment, a computer-readable storage medium is a non-transitory computer-readable storage medium that excludes transitory signals (e.g., a propagating transient electric or electromagnetic transmission) but includes non-transitory data storage circuitry (e.g., buffers, cache, and queues) within transceivers of transitory signals. In at least one embodiment, code (e.g., executable code or source code) is stored on a set of one or more non-transitory computer-readable storage media having stored thereon executable instructions (or other memory to store executable instructions) that, when executed (i.e., as a result of being executed) by one or more processors of a computer system, cause computer system to perform operations described herein. In at least one embodiment, set of non-transitory computer-readable storage media comprises multiple non-transitory computer-readable storage media and one or more of individual non-transitory storage media of multiple non-transitory computer-readable storage media lack all of code while multiple non-transitory computer-readable storage media collectively store all of code. In at least one embodiment, executable instructions are executed such that different instructions are executed by different processors—for example, a non-transitory computer-readable storage medium store instructions and a main central processing unit (“CPU”) executes some of instructions while a graphics processing unit (“GPU”) executes other instructions. In at least one embodiment, different components of a computer system have separate processors and different processors execute different subsets of instructions.

[0228] Accordingly, in at least one embodiment, computer systems are configured to implement one or more services that singly or collectively perform operations of processes described herein and such computer systems are configured with applicable hardware and/or software that enable performance of operations. Further, a computer system that implements at least one embodiment of present disclosure is a single device and, in another embodiment, is a distributed computer system comprising multiple devices that operate differently such that distributed computer system performs operations described herein and such that a single device does not perform all operations.

[0229] Use of any and all examples, or exemplary language (e.g., “such as”) provided herein, is intended merely to better illuminate embodiments of the disclosure and does not pose a limitation on scope of disclosure unless otherwise claimed. No language in specification should be construed as indicating any non-claimed element as essential to practice of disclosure.

[0230] All references, including publications, patent applications, and patents, cited herein are hereby incorporated by reference to same extent as if each reference were individually and specifically indicated to be incorporated by reference and were set forth in its entirety herein.

[0231] In description and claims, terms “coupled” and “connected,” along with their derivatives, may be used. It should be understood that these terms may be not intended as synonyms for each other. Rather, in particular examples, “connected” or “coupled” may be used to indicate that two or more elements are in direct or indirect physical or electrical contact with each other. “Coupled” may also mean that two or more elements are not in direct contact with each other, but yet still co-operate or interact with each other.

[0232] Unless specifically stated otherwise, it may be appreciated that throughout specification terms such as “processing,” “computing,” “calculating,” “determining,” or like, refer to action and/or processes of a computer or computing system, or similar electronic computing device, that manipulate and/or transform data represented as physical, such as electronic, quantities within computing system’s registers and/or memories into other data similarly represented as physical quantities within computing system’s memories, registers or other such information storage, transmission or display devices.

[0233] In a similar manner, term “processor” may refer to any device or portion of a device that processes electronic data from registers and/or memory and transform that electronic data into other electronic data that may be stored in registers and/or memory. As non-limiting examples, “processor” may be a CPU or a GPU. A “computing platform” may comprise one or more processors. As used herein, “software” processes may include, for example, software and/or hardware entities that perform work over time, such as tasks, threads, and intelligent agents. Also, each process may refer to multiple processes, for carrying out instructions in sequence or in parallel, continuously or intermittently. In at least one embodiment, terms “system” and “method” are used herein interchangeably insofar as system may embody one or more methods and methods may be considered a system.

[0234] In present document, references may be made to obtaining, acquiring, receiving, or inputting analog or digital data into a subsystem, computer system, or computer-implemented machine. In at least one embodiment, process of obtaining, acquiring, receiving, or inputting analog and digital data can be accomplished in a variety of ways such as by receiving data as a parameter of a function call or a call to an application programming interface. In at least one embodiment, processes of obtaining, acquiring, receiving, or inputting analog or digital data can be accomplished by transferring data via a serial or parallel interface. In at least one embodiment, processes of obtaining, acquiring, receiving, or inputting analog or digital data can be accomplished by transferring data via a computer network from providing entity to acquiring entity. In at least one embodiment, references may also be made to providing, outputting, transmitting, sending, or presenting analog or digital data. In various examples, processes of providing, outputting, transmitting, sending, or presenting analog or digital data can be accomplished by transferring data as an input or output parameter of a function call, a parameter of an application programming interface or interprocess communication mechanism.

[0235] Although descriptions herein set forth example embodiments of described techniques, other architectures may be used to implement described functionality, and are intended to be within scope of this disclosure. Furthermore, although specific distributions of responsibilities may be defined above for purposes of description, various functions and responsibilities might be distributed and divided in different ways, depending on circumstances.

[0236] Furthermore, although subject matter has been described in language specific to structural features and/or methodological acts, it is to be understood that subject matter claimed in appended claims is not necessarily limited to specific features or acts described. Rather, specific features and acts are disclosed as exemplary forms of implementing the claims.

What is claimed is:

1. A method comprising:

obtaining digital representations of an object depicted in a plurality of video frames; and

performing a bidirectional tracking of the object across the plurality of video frames, wherein the bidirectional tracking comprises:

for each of a forward direction (FD) of the bidirectional tracking and a reverse direction (RD) of the bidirectional tracking, obtaining, using (i) a current state of the object associated with an upstream video frame and (ii) the digital representation of the object for a downstream video frame, an updated state of the object associated with the downstream video frame; obtaining, using at least one of the updated state of the object for the FD or the updated state of the object for the RD, a bidirectional state of the object; and determining, using the bidirectional state of the object, a trajectory of the object across the plurality of video frames.

2. The method of claim 1, wherein the digital representations of the object comprise one or more feature vectors generated by a machine learning model for respective video frames of the plurality of video frames.

3. The method of claim 1, wherein the current state of the object comprises:

a feature vector representative of a depiction of the object in at least the upstream video frame, and

a location of the object associated with the upstream video frame.

4. The method of claim 3, wherein the current state of the object further comprises:

a size of the object, and

a velocity of the object associated with the upstream video frame, and

wherein the location of the object, the size of the object and the velocity of the object are determined using a statistical filter.

5. The method of claim 1, wherein obtaining the bidirectional state of the object comprises:

computing an FD similarity between an FD feature vector associated with the current state of the object for the FD of the bidirectional tracking and a feature vector associated with the digital representation of the object for the downstream frame;

computing an RD similarity between an RD feature vector associated with the current state of the object for the RD of the bidirectional tracking and the feature vector

associated with the digital representation of the object for the downstream frame; and
obtaining the bidirectional state of the object using the FD similarity and the RD similarity.

6. The method of claim 5, wherein obtaining the bidirectional state of the object using the FD similarity and the RD similarity comprises:

- determining that each of the FD similarity and the RD similarity is above a threshold similarity;
- modifying the updated state for the FD using the updated state for the RD; and
- modifying the updated state for the RD using the updated state for the FD.

7. The method of claim 5, wherein obtaining the bidirectional state of the object using the FD similarity and the RD similarity comprises:

- determining that the FD similarity is above a threshold similarity and that the RD similarity is below the threshold similarity;
- resetting the state of the object for the RD;
- assigning a new object identification (ID) as an object ID for the RD of the bidirectional tracking; and
- mapping the object ID for the RD of the bidirectional tracking to an object ID for the FD of the bidirectional tracking.

8. The method of claim 5, wherein obtaining the bidirectional state of the object using the FD similarity and the RD similarity comprises:

- determining that the RD similarity is above a threshold similarity and that the FD similarity is below the threshold similarity;
- resetting the state of the object for the FD;
- assigning a new object ID as the object for the FD of the bidirectional tracking; and
- mapping the object ID for the FD of the bidirectional tracking to an object ID for the RD of the bidirectional tracking.

9. The method of claim 5, wherein obtaining the bidirectional state of the object using the FD similarity and the RD similarity comprises:

- determining that each of the FD similarity and the RD similarity is below a threshold similarity;
- resetting the updated state of the object for the FD;
- resetting the updated state of the object for the RD; and
- assigning new object IDs to each of the FD of the bidirectional tracking and the RD of the bidirectional tracking.

10. The method of claim 1, further comprising:

generating at least one of:

- a human-perceivable report that is based, at least in part, on the trajectory of the object,
- a statistical report that is based, at least in part, on the trajectory of the object, or
- a video that depicts, at least a portion of the trajectory of the object.

11. A system comprising:

a processing device to:

- obtain digital representations of an object depicted in a plurality of video frames; and perform a bidirectional tracking of the object across the plurality of video frames, wherein to perform the bidirectional tracking, the processing device is to:
- for each of a forward direction (FD) of the bidirectional tracking and a reverse direction (RD) of the

- bidirectional tracking, obtaining, using (i) a current state of the object associated with an upstream video frame and (ii) the digital representation of the object for a downstream video frame, an updated state of the object associated with the downstream video frame;

- obtain, using at least one of the updated state of the object for the FD or the updated state of the object for the RD, a bidirectional state of the object; and
- determine, using the bidirectional state of the object, a trajectory of the object across the plurality of video frames.

12. The system of claim 11, wherein the current state of the object comprises:

- a feature vector representative of a depiction of the object in at least the upstream video frame,
- a size of the object, and
- a location of the object associated with the upstream video frame.

13. The system of claim 11, wherein to obtain the bidirectional state of the object, the processing device is to:

- compute an FD similarity between an FD feature vector associated with the current state of the object for the FD of the bidirectional tracking and a feature vector associated with the digital representation of the object for the downstream frame;

- compute an RD similarity between an RD feature vector associated with the current state of the object for the RD of the bidirectional tracking and the feature vector associated with the digital representation of the object for the downstream frame; and

- obtain the bidirectional state of the object using the FD similarity and the RD similarity.

14. The system of claim 13, wherein to obtain the bidirectional state of the object using the FD similarity and the RD similarity, the processing device is to:

- determine that each of the FD similarity and the RD similarity is above a threshold similarity;

- modify the updated state for the FD using the updated state for the RD; and

- modifying the updated state for the RD using the updated state for the FD.

15. The system of claim 13, wherein to obtain the bidirectional state of the object using the FD similarity and the RD similarity, the processing device is to:

- determine that the FD similarity is above a threshold similarity and that the RD similarity is below the threshold similarity;

- reset the state of the object for the RD;

- assign a new object identification (ID) as an object ID for the RD of the bidirectional tracking; and

- map the object ID for the RD of the bidirectional tracking to an object ID for the FD of the bidirectional tracking.

16. The system of claim 13, wherein to obtain the bidirectional state of the object using the FD similarity and the RD similarity, the processing device is to:

- determine that the RD similarity is above a threshold similarity and that the FD similarity is below the threshold similarity;

- reset the state of the object for the FD;

- assign a new object ID as an object for the FD of the bidirectional tracking; and

- map the object ID for the FD of the bidirectional tracking to an object ID for the RD of the bidirectional tracking.

17. The system of claim **13**, wherein to obtain the bidirectional state of the object using the FD similarity and the RD similarity, the processing device is to:

determine that each of the FD similarity and the RD similarity is below a threshold similarity;
reset the updated state of the object for the FD;
reset the updated state of the object for the RD; and
assign new object IDs to at least one of the FD of the bidirectional tracking or the RD of the bidirectional tracking.

18. The system of claim **11**, wherein the processing device is further to:

generate at least one of:

a human-perceivable report that is based, at least in part, on the trajectory of the object,
a statistical report that is based, at least in part, on the trajectory of the object, or
a video that depicts, at least a portion of the trajectory of the object.

19. The system of claim **11**, wherein the system is comprised in at least one of:

a control system for an autonomous or semi-autonomous machine;
a perception system for an autonomous or semi-autonomous machine;
a system for performing simulation operations;
a system for performing digital twin operations;
a system for performing light transport simulation;
a system for performing collaborative content creation for 3D assets;
a system for performing deep learning operations;
a system implemented using an edge device;

a system for generating or presenting at least one of augmented reality content, virtual reality content, or mixed reality content;

a system implemented using a robot;

a system for performing conversational AI operations;

a system for generating synthetic data using AI operations;

a system incorporating one or more virtual machines (VMs);

a system implemented at least partially in a data center; or

a system implemented at least partially using cloud computing resources.

20. A processor comprising one or more processing devices to:

obtain digital representations of an object depicted in a plurality of video frames; and

perform a bidirectional tracking of the object across the plurality of video frames, wherein to perform the bidirectional tracking, the processing device is to:

for each of a forward direction (FD) of the bidirectional tracking and a reverse direction (RD) of the bidirectional tracking, obtaining, using (i) a current state of the object associated with an upstream video frame and (ii) the digital representation of the object for a downstream video frame, an updated state of the object associated with the downstream video frame;

obtain, using at least one of the updated state of the object for the FD or the updated state of the object for the RD, a bidirectional state of the object; and

determine, using the bidirectional state of the object, a trajectory of the object across the plurality of video frames.

* * * * *